

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-a36a8a54ee9300fc8.jsonl`  
- SHA-256: `1d97a06e57ebcf5b148e8e9c3da7c6847b49d165488afd07c2f53fa931b197e7`  
- Source modified: `2026-06-06T15:32:51+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d4359b2a-456`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:26:29.865Z)

Adversarially VERIFY this CODE_MAP against the actual repo (cwd is repo root). For a representative sample across every section: confirm each cited @path exists (Glob/Read) and each named symbol (class/function/const/registry) actually appears in that file (Grep). Flag: fabricated or misspelled paths/symbols, dataflow arrows pointing the wrong way, registry/plugin claims that don't match the code, and any CRITICAL omission. Then return the FINAL map: fix every error inline and keep the dense format; if already accurate return it essentially unchanged. Set accurate=true only if you found no substantive errors. List what you checked/changed in issues.

=== MAP TO VERIFY ===

I have everything I need. Producing the CODE_MAP.md body.

```markdown

## CODE\_MAP.md

**\*\*Purpose:\*\*** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack.

**\*\*Regenerate:\*\*** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change.

**\*\*LEGEND\*\***

- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `\$` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

## 1. PIPELINE

### 1A. Runtime: video file -> highlight reel

```

config.json (json.load; absent -> {})

-> video_id = MD5(whole file) @backend/main.py::get_file_md5 (? chunk size varies per entrypoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE file so values agree)

-> validate

@backend/validators/base.py::registry.run_all(video_path, config)

```

    FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> Blur -> SceneCut ->
PTSContinuity -> Relevance
    -> db.add_video(video_id, filepath, status, [r.dict()]) @backend/database.py
    -> seg = segmenter_registry.get(name)(db, config) ?
@backend/indexer/base.py::segmenter_registry
    -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
    [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without
stitching]
    -> select segment ids -> [(start,end)] (+ stitching padding)
    -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/stitcher/compiler.py
    per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
    ...
Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
    ...
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
    -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted via
db.add_candidate_segment]
    -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip,
prompt) ? provider_registry
    -> P4 db.add_play_segment + db.link_candidate_to_segment
    ...
Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) ->
add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

```

1B. WASB shuttle substrate (P2 of wasb_hybrid)

```

    ...
WasbHybridSegmenter.process_video @backend/indexer/wasb_hybrid.py
    -> WasbRunner.healthcheck() (gate; not ok -> return []) @backend/detectors/wasb_runner.py
    -> WasbRunner.run_predict(video, out_dir)
        stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
    -> @backend/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass
(CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
    -> TrackNetRunner.parse_trajectory_csv(csv) @backend/detectors/tracknet_runner.py
(shared, re-exported by WasbRunner)
    -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, velocity_thresh,
merge_gap, min_window_duration) [the model-agnostic WINDOWING BRAIN]
    ...
(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback
(30.0, 1280.0).)

```

1C. Calibration / eval flow (NO pipeline run unless told)

```

    ...
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage ?
{candidates, final}
    -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
    -> harness.evaluate -> EvalReport -> report_to_dict
        -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
        -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py [CI
gate]
Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate}
@backend/eval/calibration.py
Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
    ...
Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can
segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
    ...

```

2. SUBSYSTEMS

2.0 Orchestration & config

```
- `@backend/main.py` ? FastAPI app + argparse CLI (`server`/`process`/`debug-clip`/`trim`/`--setup`).
- `::app` FastAPI; `::db_instance`, `::global_config` module globals set ONLY in `main()`. ?
importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
- `::process_video` `@POST /api/process` -> validate -> add_video -> segment.
`::compile_highlights` `@POST /api/compile`. `::run_cli_diagnose_clip` lazy `import cv2`,
samples ±3s vs `indexing.motion_threshold`.
- ? uvicorn host hardcoded `127.0.0.1`; CLI default_segmenter fallback `motion`.
- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks
automation), `default_segmenter` fallback `yolo_hybrid`, skips validation.
- `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`.
Exit 2=arg/IO. ? pure scorer; errors if video never processed.
- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (label only).
- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok,
1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even
a regressing run).
- `@config.json` ? single config. Sections: `validation`, `indexing` (+ `tracknet`, `wasb`),
`output`, `stitching`, root `sport/ai_provider/ai_model`. ? **three default segmenters
diverge**: config ships `gemini`, setup.py writes `yolo_hybrid`, main.py fallback `motion`.
- `@setup.py` ? write default config (skips if present), ffmpeg/torch/CUDA diagnostics, GPU-
aware pip (`nvidia-smi` -> cu124 else cpu). ? generated config diverges from checked-in (no
`relevance`/`tracknet` blocks, blur 100.0 vs 20.0).
```

2.1 Segmenters (indexer) ?

```
- `@backend/indexer/base.py`
- `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None)` -> List[dict{id,start_time,end_time,duration,metadata}].
- `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**:
subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND
import the module in `@backend/indexer/__init__.py` (registration is an IMPORT side-effect). ?
`register()` builds `cls(None, {})` just to read `name` -> name/description MUST NOT touch
`self.db`/`self.config`.
- `@backend/indexer/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest.
- `@backend/indexer/wasb_hybrid.py::WasbHybridSegmenter` (name `wasb_hybrid`) ? WSL WASB
substrate. Config `indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; hardcoded
relative out dirs `output/wasb`; needs `{PROVIDER}_API_KEY` (missing -> []).
- `@backend/indexer/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `tracknet_hybrid`) ?
copy-paste twin of wasb_hybrid (swaps runner; `detection_params` adds `queue_length`). ? any
P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
- `@backend/indexer/yolo_hybrid.py::HybridYoloSegmenter` (name `yolo_hybrid`, no YOLO remains
? torchvision detector + supervision ByteTrack). `::PERSON_CLASS_ID=1` ? (ultralytics used 0;
wrong value -> zero detections). `::lazy_load_model`, `::extract_action_windows_from_chunk`. ?
velocity computed at `fps/frame_skip` -> `velocity_threshold` coupled to `frame_skip`; trackers
per-chunk (windows merged by TIME not ID); `import torch` at module top.
- `@backend/indexer/gemini.py::GeminiPlaySegmenter` (name `gemini`) ? pure-AI, no P2. ? chunks
only if `duration>chunk_duration`; `max_workers=5` hardcoded (ignores `ai_max_workers`);
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates analysis;
`parse_time` accepts colon timestamps (warns).
- `@backend/indexer/motion.py::MotionPlaySegmenter` (name `motion`) ? pixel absdiff baseline.
? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint); only
segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
```

2.2 Detectors & windowing (WSL-quarantined)

```
- `@backend/detectors/__init__.py` ? docstring only (refs `docs/DECISIONS.md` ADR-001). ? does
NOT re-export runners; import submodules directly.
- `@backend/detectors/tracknet_runner.py`
```

```

- `::TrackNetConfig`(+`.from_indexing_cfg`), `::TrackNetRunner` ? runs TrackNet `predict.py`
in WSL.
- `::to_wsl_mnt_path`(`C:\x`->`/mnt/c/x`), `::TrajectoryPoint`(frame,visible,x,y).
- **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `` -> `run_predict` returns None; predict.py only `.mp4/.avi`; `visible==False`
resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent;
`track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever).
- `@backend/detectors/wasb_runner.py::WasbRunner` ? stages `wasb_infer.py` + video into WSL,
runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)`. ? `run_predict` does NOT pass cache flags (uses wasb_infer
defaults flush=1000/batch=8/workers=4); `wasb_infer.py` MUST be re-staged each run (editing
Windows copy is inert until staged); `_wsl_bash` duplicated (drift risk).
- `@backend/detectors/wasb_infer.py` (WSL) ? `::run` core. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.jsonl'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible`,
`::atomic_write_text/atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be np.array). ?
GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `runner.gpus=[0]` hardcoded
single-GPU; `extract_frames` cv2-PNG SLOW (prefer host ffmpeg).

```

2.3 Eval & calibration (all pure; I/O lives in CLI drivers)

```

- `@backend/eval/metrics.py` ? scoring primitive. `::interval_iou`, `::match_intervals` (greedy,
deterministic), `::score`->`::Score`(`.as_dict`() = wire shape), `::pr_curve`. ? greedy not
Hungarian; `score`() indexes the SAME preds/gts used to build `match_result` or boundary errors
are silently wrong.
- `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV;
RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
- `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ?
defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2
labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`,
`::kfold_indices`. ? `generalization_gap >=0.1` = overfit alarm.
- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)` (? hardcoded
dup of real defaults), `::default_grid` (45 cfgs), `::make_predict_fn`, `::load_golden_gts` (?
SILENTLY skips bad rows ? opposite of annotations.load_annotations). ? CLI defaults
`fps=59.94/frame-width=1920` baked in.
- `@backend/eval/export_wasb_segments.py` ? trajectory+cfg -> golden/slicer CSV + comparison.
`::TUNED=(0.05,1.0,1.0)` (? hardcoded from ONE video), `::SEG_FIELDS`, `::COMP_FIELDS`,
`::build_comparison`. `--slice` requires `--video`.
- `@backend/eval/batch.py` ? corpus aggregation. `::aggregate` (micro pool TP/FP/FN, TP-weighted
mean_iou, macro F1), `::default_stages_for` (`auto`;
`::FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path`.
- `@backend/eval/harness.py` ? DB-pred scorer. `::STAGES=('candidates','final')`,
`::get_predictions` (candidates->`query_candidate_segments(detector=)`;
final->`query_play_segments({})` ALL segs, detector arg ignored),
`::evaluate`->`::EvalReport`->`::report_to_dict` (input contract for batch.aggregate).
- `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES` (5-d),
`::extract_yolo_features` (? assumes `row['stats']` already dict; raw JSON str -> all-zero),
`::label_candidates` (same IoU matching as evaluator), `::build_training_set`->(X,y,intervals).
- `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
feature ORDER at predict must match fit (names are metadata only); fixed 1000 epochs.
- `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider`. ? imports `annotation_registry` from **`backend.annotations`** NOT
`backend.eval.annotations` (two different "annotations"); gate fires on precision OR recall drop
(F1 reported, not gated); first run on a video silently passes (no baseline).

```

2.4 Stitcher / tools / utils

```

- `@backend/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. $ skipped tuple
(idx,start,end,reason). ? if duration probe fails (0.0) NO boundary validation; `padded_start`
uses `begin_padding`; temp clips in `output_dir`; raises
ValueError/FileNotFoundError/RuntimeError; uses `print()`.

```

- `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0`` (? duplicate compiler's by copy, kept in sync manually). `_read_time` -> `backend/eval/annotations.parse_timestamp`.
- `@backend/utils/video.py` ? `::get_video_duration` (ffprobe), `::extract_video_chunk(...,reencode=False)`. ? uses `-t DURATION` (length, not end ts); copy mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips.
- `@backend/utils/geometry.py` ? `::get_velocity_normalised` (used by yolo_hybrid), `::calculate_iou` (? +1 px convention inflates IoU), `::get_center/get_distance/get_velocity`.

2.5 Validators ?

- `@backend/validators/base.py::VideoValidator` ABC, `::ValidationResult` (pydantic), `::ValidationRegistry`, `::registry` (singleton). `$ validate(video_path, config, context) -> ValidationResult`. ? ****add a validator****: subclass + `registry.register(MyValidator())` (registration order = execution order). ? validators share a per-run `context` dict; exceptions wrapped as `passed=False` (never abort batch).
- `@backend/validators/format.py::FormatValidator` (`format_check`) ? ****PRODUCER**** of `context['ffprobe_meta']`; ? MUST run before `resolution_fps`; container check is substring `'mp4'`.
- `@backend/validators/resolution_fps.py::ResolutionFPSValidator` ? CONSUMES `ffprobe_meta` (? hard-fails if `format_check` didn't run first). `min_fps` default 29.9.
- `@backend/validators/blur.py` (`blur_check`, Laplacian var < threshold = fail), `@../scene_cut.py` (? `cut_threshold=0.6` hardcoded, only first ~100 frames analyzed), `@../pts_continuity.py` (? 10.0s gap hardcoded, comment says 8.0s ? drift), `@../relevance.py` (HSV court-green; lazy `import backend.sports`; ? unknown sport -> empty cfg, does NOT fail).

2.6 Sports / Providers / Annotations / DB ?

- `@backend/sports/base.py::SportProfile` ABC;
- `@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
- `@backend/sports/__init__.py::sport_registry` (auto-registers badminton). ? ****add a sport****: subclass `SportProfile`, `sport_registry.register(MyProfile)`. `$ prompt` emits JSON array `{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS timestamps; `get()` returns a fresh instance each call.
- `@backend/providers/base.py::AIVideoProvider` ABC;
- `@backend/providers/gemini.py::GeminiProvider` (upload->poll->generate JSON temp 0->delete);
- `@backend/providers/__init__.py::provider_registry` (auto-registers `'gemini'`). ? ****add a provider****: subclass, `provider_registry.register('name', MyCls)` (key passed EXPLICITLY, unlike `sport_registry`). `$ analyze_video(path, prompt, chunk_duration=0, label='') -> (segments:list, metrics:dict{upload_time,process_time,gen_time})`. ? blocking `time.sleep(10)` poll; returns `([], metrics)` on ANY failure (empty != error); lazy client (auth surfaces on first use).
- `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`), `::Interval` (= metrics.Interval), `::STATUSES`, `::annotation_registry`. ? ****add a tier****: subclass, register at module bottom + import in `__init__.py`. `$ fetch -> List[Interval]` sorted; contract says RAISE (not `[]`) on unavailable (`[]` silently zeroes recall).
- `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the CSV path; -> `backend.eval.annotations.load_annotations`).
- `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; injectable `labeler`, `::warn_if_same_lineage` ORACLE RULE, default `not_configured` RAISES). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`.
- `@backend/annotations/__init__.py` ? ? `noqa F401` imports are load-bearing (removing de-registers providers).
- `@backend/database.py::Database` ? SQLite, 4 tables (`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder (currently =1). Methods: `add_video`, `add_play_segment`, `add_event`, `add_candidate_segment`, `link_candidate_to_segment`, `query_candidate_segments`, `query_play_segments`, `get_video`, `clear_video_data`. ? every conn re-issues `PRAGMA foreign_keys=ON` (else CASCADE silently skipped); `add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest; write helpers swallow exceptions (return False/None); events column is `type` not `event_type`; `duration` stored not derived.

3. DATA CONTRACTS (canonical shapes ? one place)

```

$ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`):
`Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords.
Row = `(fid, 1|0, x or -1, y or -1)`.
-> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).

$ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
`{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
`start/end = group_frame/fps + chunk_start`.

$ **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
`{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL,
confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count}, detection_params:JSON
{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path[,queue_length(tracknet)][,frame_skip,tracker,detector_model,detector_score_threshold(yolo)]}, confirmed_segment_id,
human_label, notes, created_at}` (stats/detection_params deserialized on read).

$ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
`{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:'f{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.

$ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time, end_time,
shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start; `_candidate_id`
stamped internally).

$ **GT / golden CSVs** (TWO conventions ? do not conflate):
- generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal or
MM:SS/HH:MM:SS; RAISES on bad row.
- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.

$ **det_raw.jsonl** (one window/line): `{ "w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
$ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
$ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv}, ...]}`; ?
DB key = file MD5, manifest `video_id` is a label.

$ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches,unmatched_pred(=FP),unmatched_gt(=FN)}`.
`Score{iou_threshold,num_pred,num_gt,true_positives,false_positives,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.

$ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]), 'score':float,
'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST rebuild `xy` as
np.array (tracker does `np.linalg.norm`).

---
```

4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; verified ? authoritative)

```

- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval',
overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda,
runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU.
- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize (mean .485/.456/.406, std .229/.224/.225); affine resize
(`utils.image.get_affine_transform`); affine matrix maps heatmap coords -> original-frame
coords.
- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
```

```

hms_vis)` results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. `eid`
= frames_out index, `bid` = batch element.
- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential `self._fid++` per `update()`
(assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`;
`refresh()` resets. `_select_not_too_far` uses `cfg.tracker.max_disp`; `det['xy']` MUST be
np.array. Full ordered re-run is deterministic.
- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform,
seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`
- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed (~6MB), trained on
academic NYCU/TrackNetV2 data -> ? confirm terms before COMMERCIAL deploy. monotrack weights =
noncommercial, avoided.

---
```

5. CROSS-REPO (sibling private repo `sports-data-collector`)

```

- Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-encoding/re-
downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact
file used.
- `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer
`start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
- Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) -> consumed
by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.

---
```

6. GOTCHAS (cross-cutting footguns)

```

- **MD5 keying:** `video_id` = MD5 of WHOLE file everywhere ? but each entriypoint uses a
different chunk SIZE (64KB/1MB/8KB); values agree, code does not. Manifest `video_id` is a
label, NOT the DB key.
- **Three default segmenters:** config.json `gemini` vs setup.py `yolo_hybrid` vs main.py
`motion`. You may run a different detector than expected.
- **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op for callers.
`analyze_video` also returns `[]` on real failure -> empty != "no rallies".
- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0,
1280.0/1920.0) silently mis-scale velocity thresholds if probing is skipped; calibrate_wasb CLI
bakes `59.94/1920`.
- **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs
(`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL
only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
`--frames_dir`.
- **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
- **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in
`output/<db_name>` before any eval/regression.
- **Validator ordering coupling:** `resolution_fps_check` hard-fails unless `format_check` ran
earlier (shared `context['ffprobe_meta']`).
- **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-deletes
its segments. Use `clear_video_data` deliberately.
- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to P3/P4
must land in both.
- **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
- **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler
padding ? duplicated by copy, not import; re-tuning won't propagate.
- **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
- **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe
fails (0.0) no boundary validation.

---
```

7. RAMP PATHS (to do X -> open Y::symbol)

```
| Task | Open |
|---|---|
| Tune candidate windowing (velocity/merge/min-dur) |
`@backend/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
| Add a segmenter | subclass `@backend/indexer/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in `@backend/indexer/__init__.py` |
| Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
| Add a sport | subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)`
in `@backend/sports/__init__.py`; define prompt+HSV in new file |
| Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', Cls)` |
| Add an annotation/GT tier | subclass `@backend/annotations/base.py::AnnotationProvider`;
register + import in `@backend/annotations/__init__.py` |
| Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory
--labels --fps --frame-width`); trust `cv` recall not `fit` |
| Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main` (`--slice`
needs `--video`) |
| Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
| Score DB preds vs GT (one video) | `@run_eval.py::main` |
| Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
| Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress, 2=missing
DB; snapshot via `--update-baseline` |
| Resume a crashed WASB pass | nothing to do ? `@backend/detectors/wasb_infer.py::run` auto-
resumes from `det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart; cache
invalidates on weights/sport/frames change |
| Add a validator | subclass `@backend/validators/base.py::VideoValidator`;
`registry.register(instance)` (mind order; producers before consumers) |
| Add a DB column/table | `@backend/database.py::Database._init_db` migration ladder + bump
`PRAGMA user_version` (else version-assert tests fail) |
| Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
```

8. TEST MAP (`tests/test\_\*.py` -> subsystem)

```
| Test | Covers |
|---|---|
| `test_base.py` | `indexer.base.SegmenterRegistry` (register/get/list via DummySegmenter) |
| `test_yolo_hybrid.py` / `test_tracknet_hybrid.py` | hybrid segmenters (yolo
torchvision+ByteTrack; tracknet 4-phase) |
| `test_gemini.py` | `indexer.gemini.GeminiPlaySegmenter` (parse/validate/chunk) |
| `test_tracknet_runner.py` | `detectors.tracknet_runner` parse_trajectory_csv +
trajectory_to_action_windows (shared brain) |
| `test_wasb_cache.py` | `detectors.wasb_infer` resumable cache (serialize, contiguity heal,
manifest invalidation, atomic writes) |
| `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
| `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate |
| `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict |
| `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate |
| `test_calibration.py` | `eval.calibration` kfold/select_best/cross_validate/LOVO |
| `test_calibrate_wasb.py` | `eval.calibrate_wasb` CLI/run/grid |
| `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment+comparison export |
| `test_eval_training.py` | `eval.training` feature extraction + labeling |
| `test_eval_classifier.py` | `eval.classifier.LogisticRegression` fit/predict/save/load |
| `test_eval_regression.py` | `eval.regression` baseline compare/gate |
| `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label parse |
| `test_video.py` | `utils.video` duration/chunk |
| `test_geometry.py` | `utils.geometry` center/distance/velocity/IoU |
```



```
| `test_database.py` | `database.Database` schema/CRUD/migrations/version |
| `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile |
| `test_providers.py` / `test_provider_gemini.py` / `test_gemini.py` | `providers` registry +
GeminiProvider lifecycle |
| `test_annotation_providers.py` / `test_automated_provider.py` | `annotations` registry,
FileProvider, AutomatedProvider (oracle/lineage) |
| (no dedicated test) | `validators/*`, `stitcher.compiler`, `main.py`, `run_*.py` orchestrators
?? exercised only indirectly |
```
```

## 2. assistant (2026-06-06T15:26:33.516Z)

I'll verify this CODE\_MAP against the actual repo. Let me start by exploring the structure and confirming key paths exist.

## 3. user (2026-06-06T15:26:33.615Z)

```
backend\database.py
backend\detectors__init__.py
backend\detectors\tracknet_runner.py
backend\eval__init__.py
backend\eval\annotations.py
backend\eval\batch.py
backend\eval\classifier.py
backend\eval\harness.py
backend\eval\metrics.py
backend\eval\training.py
backend\indexer\base.py
backend\indexer\gemini.py
backend\indexer\motion.py
backend\indexer\tracknet_hybrid.py
backend\main.py
backend\providers__init__.py
backend\providers\base.py
backend\providers\gemini.py
backend\sports__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\stitcher\compiler.py
backend\utils__init__.py
backend\utils\geometry.py
backend\utils\video.py
backend\validators__init__.py
backend\validators\base.py
backend\validators\blur.py
backend\validators\format.py
backend\validators\pts_continuity.py
backend\validators\relevance.py
backend\validators\resolution_fps.py
backend\validators\scene_cut.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
scratch\analyze_motion.py
scratch\clean_state.py
scratch\diagnose_rally.py
scratch\gemini_video_debugger.py
scratch\list_models.py
scratch\test_ai_segmenter.py
scratch\test_thresholds.py
tests__init__.py
tests\test_base.py
tests\test_database.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
```

```

tests\test_eval_classifier.py
tests\test_eval_harness.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_gemini.py
tests\test_geometry.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_video.py
backend\indexer\yolo_hybrid.py
scripts\generate_khelacharya_flyer.py
setup.py
tests\test_yolo_hybrid.py
backend\annotations\base.py
backend\annotations\file_provider.py
tests\test_annotation_providers.py
backend\annotations\automated_provider.py
backend\annotations__init__.py
backend\eval\regression.py
run_regression.py
tests\test_automated_provider.py
tests\test_eval_regression.py
backend\tools\rally_slicer.py
backend\tools__init__.py
tests\test_rally_slicer.py
backend\detectors\wasb_runner.py
backend\eval\calibration.py
backend\indexer\wasb_hybrid.py
backend\indexer__init__.py
tests\test_calibration.py
backend\eval\calibrate_wasb.py
tests\test_calibrate_wasb.py
backend\detectors\wasb_infer.py
tests\test_wasb_cache.py
backend\eval\export_wasb_segments.py
tests\test_export_wasb_segments.py

```

#### 4. assistant (2026-06-06T15:26:36.666Z)

The repo structure matches. Let me verify key files and symbols across sections in parallel. Starting with the pipeline core and registries.

#### 5. user (2026-06-06T15:26:37.608Z)

```

1 from abc import ABC, abstractmethod
2 from typing import List, Dict, Any, Type, Optional
3 from backend.database import Database
4
5 class BasePlaySegmenter(ABC):
6 def __init__(self, db: Database, config: Dict[str, Any]):
7 self.db = db
8 self.config = config
9
10 @property
11 @abstractmethod
12 def name(self) -> str:
13 """Unique identifier for this segmenter."""
14 pass
15
16 @property
17 @abstractmethod
18 def description(self) -> str:

```

```

19 """Human-readable explanation of this segmenter."""
20 pass
21
22 @abstractmethod
23 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
24 """
25 Processes a video to detect play segments, populates SQLite database,
26 and returns list of segment representations.
27 """
28 pass
29
30 class SegmenterRegistry:
31 def __init__(self):
32 self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
33
34 def register(self, segmenter_cls: Type[BasePlaySegmenter]):
35 """Registers a new segmenter class under its name property."""
36 try:
37 temp_inst = segmenter_cls(None, {})
38 name = temp_inst.name
39 except Exception as e:
40 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
41 self._segmenters[name] = segmenter_cls
42
43 def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
44 return self._segmenters.get(name)
45
46 def list_available(self) -> Dict[str, str]:
47 """Returns a dict of registered segmenter names and their descriptions."""
48 res = {}
49 for name, cls in self._segmenters.items():
50 try:
51 temp_inst = cls(None, {})
52 desc = temp_inst.description
53 except Exception:
54 desc = "No description available."
55 res[name] = desc
56 return res
57
58 segmenter_registry = SegmenterRegistry()
59

```

## 6. user (2026-06-06T15:26:38.078Z)

```

1 from backend.indexer.base import segmenter_registry, BasePlaySegmenter
2 from backend.indexer.motion import MotionPlaySegmenter
3 from backend.indexer.gemini import GeminiPlaySegmenter
4 from backend.indexer.yolo_hybrid import HybridYoloSegmenter
5 from backend.indexer.tracknet_hybrid import TrackNetHybridSegmenter
6 from backend.indexer.wasb_hybrid import WasbHybridSegmenter
7
8 __all__ = [
9 "segmenter_registry",
10 "BasePlaySegmenter",
11 "MotionPlaySegmenter",
12 "GeminiPlaySegmenter",
13 "HybridYoloSegmenter",
14 "TrackNetHybridSegmenter",
15 "WasbHybridSegmenter",
16]
17

```

## 7. user (2026-06-06T15:26:38.648Z)

```

1 """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3 Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4 1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5 2. copies wasb_infer.py into the WASB repo's src/ dir,
6 3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7 4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9 So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10 with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11 """
12 import os
13 import shlex
14 import logging
15 import subprocess
16 from dataclasses import dataclass
17 from typing import Dict, Any, Optional
18
19 # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20 # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21 TrackNetRunner.
22 from backend.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23
24 logger = logging.getLogger(__name__)
25
26 # Windows path to the inference wrapper we stage into WSL.
27 _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29 @dataclass
30 class WasbConfig:
31 """Resolved settings for a WASB WSL run (built from config.json ->
32 indexing.wasb)."""
33 distro: str = "Ubuntu"
34 repo_dir: str = "~/models/WASB-SBDT"
35 conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36 conda_env: str = "wasb"
37 weights_path: str = "~/models/WASB-
38 SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
39 sport: str = "badminton"
40 wsl_stage_dir: str = "~/clips"
41 timeout_sec: int = 0 # 0 = no timeout
42
43 @classmethod
44 def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
45 w = (idx_cfg or {}).get("wasb", {}) or {}
46 return cls(
47 distro=w.get("wsl_distro", "Ubuntu"),
48 repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
49 conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
50 conda_env=w.get("conda_env", "wasb"),
51 weights_path=w.get("weights_path",
52 "~/models/WASB-
53 SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54 sport=w.get("sport", "badminton"),
55 wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56 timeout_sec=int(w.get("timeout_sec", 0)),
57)
58
59 class WasbRunner:
60 def __init__(self, cfg: WasbConfig):
61 self.cfg = cfg
62
63 def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:

```

```

62 full = f"source {self.cfg.conda_sh} && {inner_cmd}"
63 argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
64 logger.debug("WSL exec: %s", full)
65 return subprocess.run(argv, capture_output=True, text=True,
66 timeout=(self.cfg.timeout_sec or None))
67
68 def healthcheck(self) -> tuple:
69 """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
70 cmd = (f"conda activate {self.cfg.conda_env} && "
71 f"python -c \"import torch; print('torch', torch.__version__, "
72 f"'cuda', torch.cuda.is_available())\"")
73 try:
74 res = self._wsl_bash(cmd)
75 except FileNotFoundError:
76 return False, "`wsl` not found ? Windows + WSL2 required."
77 except subprocess.TimeoutExpired:
78 return False, "WSL healthcheck timed out."
79 out = (res.stdout or "").strip()
80 if res.returncode != 0:
81 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
82 return True, out
83
84 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
85 src_mnt = to_wsl_mnt_path(video_win_path)
86 dest = f"{self.cfg.wsl_stage_dir}/{base}"
87 cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
88 {shlex.quote(dest)} && echo OK"
89 try:
90 res = self._wsl_bash(cmd)
91 except subprocess.TimeoutExpired:
92 logger.error("Staging video into WSL timed out.")
93 return None
94 if res.returncode != 0 or "OK" not in (res.stdout or ""):
95 logger.error("Failed to stage video into WSL: %s", (res.stderr or
96 res.stdout).strip())
97 return None
98 return dest
99
100 def _stage_infer_script(self) -> bool:
101 """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
102 src_mnt = to_wsl_mnt_path(_INFER_WIN)
103 cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
104 OK"
105 res = self._wsl_bash(cmd)
106 return res.returncode == 0 and "OK" in (res.stdout or "")
107
108 def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
109 """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
110 None."""
111 os.makedirs(output_win_dir, exist_ok=True)
112 base = os.path.basename(video_win_path)
113 stem = os.path.splitext(base)[0]
114 wsl_out_dir = to_wsl_mnt_path(output_win_dir)
115 expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
116
117 if not self._stage_infer_script():
118 logger.error("Could not stage wasb_infer.py into WSL.")
119 return None
120 staged_video = self._stage_video(video_win_path, base)
121 if staged_video is None:
122 return None
123 frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
124
125 cmd = (
126 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "

```

```

123 f"python wasb_infer.py "
124 f"--video {shlex.quote(staged_video)} "
125 f"--frames_out_dir {shlex.quote(frames_dir)} "
126 f"--weights {shlex.quote(self.cfg.weights_path)} "
127 f"--sport {shlex.quote(self.cfg.sport)} "
128 f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}}"
129)
130 logger.info("Running WASB inference on %s ...", base)
131 try:
132 res = self._wsl_bash(cmd)
133 except subprocess.TimeoutExpired:
134 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
135 return None
136 if res.returncode != 0:
137 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
138 return None
139 if not os.path.exists(expected_csv_win):
140 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
141 return None
142 logger.info("WASB trajectory CSV: %s", expected_csv_win)
143 return expected_csv_win
144
145 # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
146 parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
147 trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
148

```

## 8. assistant (2026-06-06T15:26:42.838Z)

Base segmenter, registry, and WasbRunner all confirmed. Note: the map says `run\_predict` does NOT pass cache flags ? confirmed (no flush/batch/workers flags in the command). Let me check the main.py, tracknet\_runner, and wasb\_infer files.

## 9. user (2026-06-06T15:26:43.639Z)

```

1 """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3 This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4 inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6 1. Translate the Windows video path to its WSL-visible form (and optionally
7 stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8 2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9 3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10 4. Convert that trajectory into high-motion "action windows" ? the same
11 candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12 pipeline (AI handoff, DB population) is unchanged.
13
14 Nothing here imports TensorFlow; all model code stays in WSL.
15 """
16
17 import os
18 import csv
19 import shlex
20 import logging
21 import subprocess
22 from dataclasses import dataclass
23 from typing import List, Dict, Any, Optional, Tuple
24
25 logger = logging.getLogger(__name__)
26

```

```

27
28 @dataclass
29 class TrackNetConfig:
30 """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
31 distro: str = "Ubuntu"
32 repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
33 conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
34 conda_env: str = "TrackNetV4"
35 weights_path: str = "" # WSL path to the .h5/.keras weights
(REQUIRED)
36 queue_length: int = 5
37 stage_in_wsl: bool = True # copy video into WSL fs first
(perf)
38 wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
in WSL
39 timeout_sec: int = 0 # 0 = no timeout
40
41 @classmethod
42 def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
43 tn = (idx_cfg or {}).get("tracknet", {}) or {}
44 return cls(
45 distro=tn.get("wsl_distro", "Ubuntu"),
46 repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
47 conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
48 conda_env=tn.get("conda_env", "TrackNetV4"),
49 weights_path=tn.get("weights_path", ""),
50 queue_length=int(tn.get("queue_length", 5)),
51 stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
52 wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
53 timeout_sec=int(tn.get("timeout_sec", 0)),
54)
55
56
57 def to_wsl_mnt_path(win_path: str) -> str:
58 """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
59
60 Already-POSIX paths (starting with / or ~) are returned unchanged so the
61 same helper is safe to call on either kind of path.
62 """
63 p = str(win_path)
64 if p.startswith("/") or p.startswith("~"):
65 return p
66 p = p.replace("\\", "/")
67 if len(p) >= 2 and p[1] == ":":
68 drive = p[0].lower()
69 return f"/mnt/{drive}{p[2:]}"
70 return p
71
72
73 @dataclass
74 class TrajectoryPoint:
75 frame: int
76 visible: bool
77 x: float
78 y: float
79
80
81 class TrackNetRunner:
82 """Runs TrackNet predict.py in WSL and turns the output CSV into action windows."""
83
84 def __init__(self, cfg: TrackNetConfig):
85 self.cfg = cfg
86
87 # ----- WSL exec

```

```

88
89 def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
90 """Run a bash command string inside the configured WSL distro."""
91 full = f"source {self.cfg.conda_sh} && {inner_cmd}"
92 argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
93 logger.debug("WSL exec: %s", full)
94 return subprocess.run(
95 argv, capture_output=True, text=True,
96 timeout=(self.cfg.timeout_sec or None),
97)
98
99 def healthcheck(self) -> Tuple[bool, str]:
100 """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
101
102 Returns (ok, message). Cheap to call before a long run.
103 """
104 cmd = (
105 f"conda activate {self.cfg.conda_env} && "
106 f"python -c \"import tensorflow as tf; "
107 f"print('TF', tf.__version__, 'GPUs', "
108 f"len(tf.config.list_physical_devices('GPU')))\n"
109)
110 try:
111 res = self._wsl_bash(cmd)
112 except FileNotFoundError:
113 return False, "wsl` not found ? is this running on Windows with WSL2
114 installed?"
115 except subprocess.TimeoutExpired:
116 return False, "WSL healthcheck timed out."
117 out = (res.stdout or "").strip()
118 if res.returncode != 0:
119 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
120 return True, out
121
122 # ----- inference
123
124 def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
125 """Run predict.py on a video. Returns the Windows path to the produced CSV, or
126 None.
127
128 ``output_win_dir`` is a Windows directory (under the repo's output/) that
129 WSL writes into via its /mnt view, so the Windows process can read the CSV
130 back with no copy.
131 """
132 if not self.cfg.weights_path:
133 logger.error(
134 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
135 "in config.json to the WSL path of your trained TrackNetV4 weights."
136)
137 return None
138
139 os.makedirs(output_win_dir, exist_ok=True)
140 base = os.path.basename(video_win_path)
141 stem, ext = os.path.splitext(base)
142
143 # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
144 if ext.lower() not in (".mp4", ".avi"):
145 logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
146 return None
147
148 wsl_out = to_wsl_mnt_path(output_win_dir)
149 expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
150
151 # Resolve the video path WSL will read.
152 if self.cfg.stage_in_wsl:

```



```

150 staged = self._stage_video(video_win_path, base)
151 if staged is None:
152 return None
153 wsl_video = staged
154 else:
155 wsl_video = to_wsl_mnt_path(video_win_path)
156
157 cmd = (
158 f"conda activate {self.cfg.conda_env} && "
159 f"cd {self.cfg.repo_dir}/src && "
160 f"python predict.py "
161 f"--video_path {shlex.quote(wsl_video)} "
162 f"--model_weights {shlex.quote(self.cfg.weights_path)} "
163 f"--output_dir {shlex.quote(wsl_out)} "
164 f"--queue_length {self.cfg.queue_length}"
165)
166 logger.info("Running TrackNet inference on %s ...", base)
167 try:
168 res = self._wsl_bash(cmd)
169 except subprocess.TimeoutExpired:
170 logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
171 return None
172
173 if res.returncode != 0:
174 logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
175 return None
176
177 if not os.path.exists(expected_csv_win):
178 logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
179 return None
180 logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
181 return expected_csv_win
182
183 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
184 """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
185
186 Returns the WSL path to the staged copy, or None on failure.
187 """
188 src_mnt = to_wsl_mnt_path(video_win_path)
189 dest = f"{self.cfg.wsl_stage_dir}/{base}"
190 cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
191 try:
192 res = self._wsl_bash(cmd)
193 except subprocess.TimeoutExpired:
194 logger.error("Staging video into WSL timed out.")
195 return None
196 if res.returncode != 0 or "OK" not in (res.stdout or ""):
197 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
198 return None
199 return dest
200
201 # ----- CSV -> windows
202
203 @staticmethod
204 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
205 """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
206 points: List[TrajectoryPoint] = []
207 with open(csv_win_path, newline="") as f:
208 reader = csv.DictReader(f)
209 for row in reader:

```

```

210 try:
211 points.append(TrajectoryPoint(
212 frame=int(row["Frame"]),
213 visible=int(row["Visibility"]) == 1,
214 x=float(row["X"]),
215 y=float(row["Y"]),
216))
217 except (KeyError, ValueError):
218 continue
219 points.sort(key=lambda p: p.frame)
220 return points
221
222 @staticmethod
223 def trajectory_to_action_windows(
224 points: List[TrajectoryPoint],
225 fps: float,
226 frame_width: float,
227 chunk_start: float = 0.0,
228 velocity_thresh: float = 0.02,
229 merge_gap: float = 2.0,
230 min_window_duration: float = 2.0,
231 chunk_index: Optional[int] = None,
232) -> List[Dict[str, Any]]:
233 """Convert a shuttle trajectory into candidate rally windows.
234
235 A frame is "active" when the shuttle is visible AND its inter-frame
236 displacement (normalised by frame width, per second) exceeds
237 ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
238 ``merge_gap`` seconds of each other are grouped into one window; short
239 windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
240 shape feeding the AI-handoff phase is identical.
241 """
242 if fps <= 0:
243 fps = 30.0
244 if frame_width <= 0:
245 frame_width = 1280.0
246
247 # Compute per-frame normalised velocity from the last *visible* point.
248 active = [] # (frame_idx, velocity)
249 prev: Optional[TrajectoryPoint] = None
250 for p in points:
251 if not p.visible:
252 prev = None # break the trajectory; absent shuttle = not in flight
253 continue
254 if prev is not None:
255 dframes = p.frame - prev.frame
256 if dframes > 0:
257 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
258 velocity = (dist / frame_width) * (fps / dframes)
259 if velocity > velocity_thresh:
260 active.append((p.frame, velocity))
261 prev = p
262
263 if not active:
264 return []
265
266 max_gap_frames = int(merge_gap * fps)
267
268 def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
269 vels = [v for _, v in group]
270 return {
271 "start": group[0][0] / fps + chunk_start,
272 "end": group[-1][0] / fps + chunk_start,
273 "active_frame_count": len(group),
274 "peak_velocity": max(vels),

```

```

275 "mean_velocity": sum(vels) / len(vels),
276 "track_id_count": 1, # single shuttle; kept for window-shape parity
277 "chunk_index": chunk_index,
278 }
279
280 windows = []
281 group = [active[0]]
282 for af in active[1:]:
283 if af[0] - group[-1][0] <= max_gap_frames:
284 group.append(af)
285 else:
286 windows.append(_finalize(group))
287 group = [af]
288 windows.append(_finalize(group))
289
290 return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
291

```

## 10. assistant (2026-06-06T15:26:52.840Z)

tracknet\_runner confirms: `parse\_trajectory\_csv` and `trajectory\_to\_action\_windows` are @staticmethods, the windowing brain, `to\_wsl\_mnt\_path`, `TrajectoryPoint(frame,visible,x,y)`, `TrackNetConfig`, fps<=0->30/frame\_width<=0->1280, weights\_path default '', timeout\_sec=0, visible==False resets prev, track\_id\_count=1. The candidate window dict shape matches §. Note the signature param is `min\_window\_duration` (map cites that correctly in 1B).

Now main.py, wasb\_infer, and the eval core.

## 11. user (2026-06-06T15:26:53.519Z)

```

1 import os
2 import sys
3 import argparse
4 import hashlib
5 import json
6 import uvicorn
7 from dotenv import load_dotenv
8
9 load_dotenv()
10 from fastapi import FastAPI, HTTPException
11 from fastapi.middleware.cors import CORSMiddleware
12 from pydantic import BaseModel
13 from typing import List, Dict, Any, Optional
14
15 from fastapi.staticfiles import StaticFiles
16 from fastapi.responses import HTMLResponse
17 from backend.validators.base import registry
18 from backend.database import Database
19 from backend.indexer.base import segmenter_registry, BasePlaySegmenter
20 from backend.stitcher.compiler import VideoCompiler
21
22 app = FastAPI(title="Sports Highlight Indexer API")
23
24 # Enable CORS for local web interface
25 app.add_middleware(
26 CORSMiddleware,
27 allow_origins=["*"],
28 allow_credentials=True,
29 allow_methods=["*"],
30 allow_headers=["*"],
31)
32
33 # Serves static frontend files directly
34 os.makedirs("backend/static", exist_ok=True)
35 app.mount("/static", StaticFiles(directory="backend/static"), name="static")

```

```

36
37 @app.get("/", response_class=HTMLResponse)
38 def serve_index():
39 index_path = "backend/static/index.html"
40 if os.path.exists(index_path):
41 with open(index_path, "r", encoding="utf-8") as f:
42 return f.read()
43 return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/static/</h3>"
44
45 # Global variables initialized at startup
46 db_instance: Optional[Database] = None
47 global_config: Optional[Dict[str, Any]] = None
48
49 def get_file_md5(filepath: str) -> str:
50 """Calculates file hash to uniquely identify videos and prevent duplicate
indexes."""
51 hasher = hashlib.md5()
52 with open(filepath, "rb") as f:
53 # Read in chunk sizes of 64kb
54 for chunk in iter(lambda: f.read(65536), b""):
55 hasher.update(chunk)
56 return hasher.hexdigest()
57
58 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
59 if os.path.exists(config_path):
60 with open(config_path, "r") as f:
61 return json.load(f)
62 return {}
63
64 # --- API Models ---
65 class ProcessRequest(BaseModel):
66 video_path: str
67 player_pool: Optional[List[str]] = None
68 max_frames: Optional[int] = None
69 segmenter: Optional[str] = None
70
71 class QueryRequest(BaseModel):
72 video_id: str
73 server: Optional[str] = None
74 receiver: Optional[str] = None
75 duration_min: Optional[float] = None
76 event_type: Optional[str] = None
77
78 class CompileRequest(BaseModel):
79 video_id: str
80 segment_ids: List[int]
81 output_filename: str
82
83 # --- API Endpoints ---
84 @app.get("/api/config")
85 def get_config():
86 return global_config
87
88 @app.post("/api/process")
89 def process_video(req: ProcessRequest):
90 if not os.path.exists(req.video_path):
91 raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
92
93 video_id = get_file_md5(req.video_path)
94
95 # 1. Run pluggable validators
96 print(f"[API] Running validations for {req.video_path}...")
97 val_results = registry.run_all(req.video_path, global_config)

```

```

98
99 # Check if any validator failed
100 failures = [r for r in val_results if not r.passed]
101 if failures:
102 # Save failed processing log
103 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
104 return {
105 "video_id": video_id,
106 "status": "failed",
107 "message": "Video failed validation checks.",
108 "results": [r.dict() for r in val_results]
109 }
110
111 # 2. Run segmenter & indexer
112 print(f"[API] Video passed checks. Segmenting and indexing...")
113 db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
114
115 seg_name = req.segmenter or global_config.get("indexing",
{ }).get("default_segmenter", "motion")
116 segmenter_cls = segmenter_registry.get(seg_name)
117 if not segmenter_cls:
118 available = list(segmenter_registry.list_available().keys())
119 raise HTTPException(
120 status_code=400,
121 detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
122)
123
124 print(f"[API] Using segmenter: {seg_name}")
125 segmenter = segmenter_cls(db_instance, global_config)
126 segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
127
128 return {
129 "video_id": video_id,
130 "status": "success",
131 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
132 "results": [r.dict() for r in val_results],
133 "play_segments": segments
134 }
135
136 @app.post("/api/query")
137 def query_play_segments(req: QueryRequest):
138 filters = {
139 "server": req.server,
140 "receiver": req.receiver,
141 "duration_min": req.duration_min,
142 "event_type": req.event_type
143 }
144 segments = db_instance.query_play_segments(req.video_id, filters)
145 return {"play_segments": segments}
146
147 @app.post("/api/compile")
148 def compile_highlights(req: CompileRequest):
149 video = db_instance.get_video(req.video_id)
150 if not video:
151 raise HTTPException(status_code=404, detail="Indexed video record not found.")
152
153 # Retrieve matching play segments from db
154 all_segments = db_instance.query_play_segments(req.video_id, { })
155 matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
156

```

```

157 if not matched_segments:
158 raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
159
160 # Extract start/end time pairs
161 time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
162
163 # Run compiler
164 output_dir = global_config.get("output", {}).get("output_dir", "output")
165 compiler = VideoCompiler(output_dir)
166 try:
167 out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
168 return {"status": "success", "filepath": out_path}
169 except Exception as e:
170 raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")
171
172 # --- CLI Debug Utility & Orchestration ---
173 def run_cli_diagnose_clip(video_path: str, timestamp: float):
174 """CLI debugging utility to examine segment properties around a timestamp."""
175 print("-" * 60)
176 print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
177 print("-" * 60)
178
179 cfg = load_config()
180
181 # 1. Validation check diagnostics
182 print("[DIAGNOSTIC] Running validation framework...")
183 results = registry.run_all(video_path, cfg)
184 for r in results:
185 status_symbol = "[PASS]" if r.passed else "[FAIL]"
186 print(f" {status_symbol} {r.validator_name}: {r.message}")
187 if r.details:
188 print(f" Details: {r.details}")
189
190 # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
191 print("-" * 60)
192 print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
193 import cv2
194 cap = cv2.VideoCapture(video_path)
195 if not cap.isOpened():
196 print("[FAIL] OpenCV could not open video.")
197 return
198
199 fps = cap.get(cv2.CAP_PROP_FPS)
200 total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
201
202 start_frame = max(0, int((timestamp - 3.0) * fps))
203 end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
204
205 # Measure frame-by-frame changes in sample region
206 cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
207 prev_gray = None
208 motions = []
209
210 for f in range(start_frame, end_frame):
211 ret, frame = cap.read()
212 if not ret:
213 break
214 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
215 gray = cv2.GaussianBlur(gray, (21, 21), 0)
216
217 if prev_gray is not None:
218 diff = cv2.absdiff(prev_gray, gray)

```

```

219 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
220 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
221 motions.append(motion_ratio)
222 prev_gray = gray
223
224 cap.release()
225
226 if motions:
227 avg_motion = sum(motions) / len(motions)
228 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)
229 print(f" Sample Frame Count: {len(motions)}")
230 print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
231 if avg_motion > threshold:
232 print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
233 else:
234 print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
235 else:
236 print(" [ERROR] No visual frames were extracted in the sample range.")
237
238 print("=" * 60)
239
240 def main():
241 parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
242 parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
243 parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
244 parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
245 parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
246 parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
247 parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
248 parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
249
250 available_segmenters = list(segmenter_registry.list_available().keys())
251 parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
252 parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
253 parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
254 parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
255
256 args = parser.parse_args()
257
258 if args.setup:
259 # Invoke setup diagnostics
260 import subprocess
261 print("[INFO] Invoking setup.py...")
262 subprocess.run([sys.executable, "setup.py"])
263 return
264
265 if args.trim_start is not None and args.trim_end is not None:
266 if not args.video_path:
267 print("[ERROR] Please provide --video-path to extract a segment.")
268 return

```

```

269
270 # Determine output path
271 out_filename = args.trim_output or "trimmed_segment.mp4"
272 if os.path.isabs(out_filename):
273 out_path = out_filename
274 out_dir = os.path.dirname(out_path)
275 out_name = os.path.basename(out_path)
276 else:
277 out_dir = args.output_dir
278 out_path = os.path.join(out_dir, out_filename)
279 out_name = out_filename
280
281 os.makedirs(out_dir, exist_ok=True)
282 print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
283
284 compiler = VideoCompiler(out_dir)
285 try:
286 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
287 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
288 except Exception as e:
289 print(f"[ERROR] Failed to compile trimmed segment: {e}")
290 return
291
292 if args.debug_clip is not None:
293 if not args.video_path:
294 print("[ERROR] Please provide --video-path to debug a specific clip.")
295 return
296 run_cli_diagnose_clip(args.video_path, args.debug_clip)
297 return
298
299 # Initialize Global Config and DB
300 global global_config, db_instance
301 global_config = load_config()
302 # Merge output_dir into config
303 if "output" not in global_config:
304 global_config["output"] = {}
305 global_config["output"]["output_dir"] = args.output_dir
306 os.makedirs(args.output_dir, exist_ok=True)
307
308 db_path = os.path.join(args.output_dir, global_config.get("output",
{}).get("db_name", "sports_indexer.db"))
309 db_instance = Database(db_path)
310
311 # CLI processing mode (no web UI needed)
312 if args.video_path and not args.server:
313 print(f"[CLI] Processing video file: {args.video_path}")
314 if not os.path.exists(args.video_path):
315 print(f"[FAIL] Video file not found: {args.video_path}")
316 return
317
318 video_id = get_file_md5(args.video_path)
319
320 # Validate
321 print("[CLI] Validating...")
322 val_results = registry.run_all(args.video_path, global_config)
323 failures = [r for r in val_results if not r.passed]
324
325 # Save video status
326 status = "failed" if failures else "processed"
327 db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])
328
329 print("\n" + "="*40)

```



```

330 print("VALIDATION SUMMARY")
331 print("="*40)
332 for r in val_results:
333 status_symbol = "[PASS]" if r.passed else "[FAIL]"
334 print(f"{status_symbol} {r.validator_name}: {r.message}")
335 print("="*40 + "\n")
336
337 if failures:
338 print("[FAIL] Video failed checks. Indexing halted.")
339 return
340
341 # Index
342 seg_name = args.segmenter or global_config.get("indexing",
343 {}).get("default_segmenter", "motion")
344 segmenter_cls = segmenter_registry.get(seg_name)
345 if not segmenter_cls:
346 print(f"[FAIL] Segmenter '{seg_name}' not found.")
347 return
348
349 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
350 segmenter = segmenter_cls(db_instance, global_config)
351 segments = segmenter.process_video(video_id, args.video_path,
352 max_frames=args.max_frames)
353 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
354 database: {db_path}")
355 return
356
357 # Start Local Server Mode
358 if args.server or not args.video_path:
359 print(f"[SERVER] Starting local FastAPI server on
360 http://localhost:{args.port}...")
361 uvicorn.run(app, host="127.0.0.1", port=args.port)
362
363 if __name__ == "__main__":
364 main()
365
366

```

## 12. assistant (2026-06-06T15:27:00.181Z)

main.py confirmed: `app`, `db\_instance`, `global\_config`, `get\_file\_md5` (64KB), `process\_video` @POST /api/process, `compile\_highlights` @POST /api/compile, `run\_cli\_diagnose\_clip` (lazy import cv2, ±3s, motion\_threshold), uvicorn host 127.0.0.1, CLI default\_segmenter fallback 'motion'. All accurate.

One note: map says `--server`/`process`/`debug-clip`/`trim` subcommands but it's argparse flags (`--setup`, `--debug-clip`, `--server`, `--trim-start`/`--trim-end`), not subcommands. The map's phrasing "argparse CLI (`server`/`process`/`debug-clip`/`trim`/`--setup`)" is a loose description ? acceptable but slightly imprecise. Let me continue verifying the heavy modules.

## 13. user (2026-06-06T15:27:00.897Z)

```

1 """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3 WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4 own building blocks (ImageDataset transform + detector + online tracker) to run on
5 any folder of extracted frames and emit a trajectory CSV in the exact shape the
6 indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8 Frame,Visibility,X,Y
9
10 It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11 imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12 (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14 Resumable across reboots

```

```

15 -----
16 The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17 the slow, expensive part; the CPU tracker pass that turns detections into a
18 trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19 resume mid-stream). We exploit that split:
20
21 * The detector pass writes its per-window raw outputs incrementally to
22 ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23 records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24 most ``--flush-every`` windows of GPU work instead of the whole pass.
25 * On restart we replay the cached windows, skip the DataLoader ahead to the first
26 un-cached window, and continue. Once every window is cached, the detector pass
27 is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28 the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29 not the GPU hour.
30
31 The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32 to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33 weights, sport, window size, or frame count change.
34
35 Usage (in WSL, from ~/models/WASB-SBDT/src):
36 python wasb_infer.py --frames_dir /path/frames --weights
37 ../pretrained_weights/wasb_badminton_best.pth.tar \
38 --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39 every 1000] [--fresh]
40
41 """
42 import os
43 import os.path as osp
44 import csv
45 import glob
46 import json
47 import argparse
48 from collections import defaultdict
49
50 CACHE_VERSION = 1
51 _DET_FILE = "det_raw.jsonl"
52 _MANIFEST_FILE = "manifest.json"
53
54 def _build_cfg(weights: str, sport: str):
55 """Compose the WASB eval config via Hydra, overriding model/weights/device."""
56 from hydra import compose, initialize
57 with initialize(config_path="configs", version_base=None):
58 cfg = compose(config_name="eval", overrides=[
59 f"dataset={sport}",
60 "model=wasb",
61 f"detector.model_path={weights}",
62 "runner.device=cuda",
63 "runner.gpus=[0]", # this box has a single GPU; default config assumes
64])
65 return cfg
66
67 def _frame_id(path: str) -> int:
68 """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
69 180)."""
70 stem = osp.splitext(osp.basename(path))[0]
71 digits = "".join(ch for ch in stem if ch.isdigit())
72 return int(digits) if digits else -1
73
74 # ----- #
75 # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
76 # ----- #

```

```

76 def _cache_paths(cache_dir: str):
77 return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
78
79
80 def default_cache_dir(out_csv: str) -> str:
81 """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
82 d = osp.dirname(osp.abspath(out_csv))
83 stem = osp.splitext(osp.basename(out_csv))[0]
84 return osp.join(d, stem + "_wasbcache")
85
86
87 def _atomic_write_text(path: str, text: str) -> None:
88 """Write then rename so a crash mid-write can't leave a half-written file."""
89 tmp = path + ".tmp"
90 with open(tmp, "w") as f:
91 f.write(text)
92 f.flush()
93 os.fsync(f.fileno())
94 os.replace(tmp, path)
95
96
97 def _det_plain(det) -> list:
98 """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
99 list."""
100 xy = det["xy"]
101 return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
102
103 def _dets_to_tracker(plain_list):
104 """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
105 array,
106 the tracker does vector math / np.linalg.norm on it)."""
107 import numpy as np
108 return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
109 for p in plain_list]
110
111 def write_manifest(cache_dir: str, manifest: dict) -> None:
112 _, mpath = _cache_paths(cache_dir)
113 _atomic_write_text(mpath, json.dumps(manifest, indent=2))
114
115
116 def read_manifest(cache_dir: str):
117 _, mpath = _cache_paths(cache_dir)
118 if not osp.exists(mpath):
119 return None
120 try:
121 with open(mpath) as f:
122 return json.load(f)
123 except (json.JSONDecodeError, OSError):
124 return None
125
126
127 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
128 frames_in: int, frames_total: int) -> bool:
129 """A cache is reusable only if the run that produced it matches this run's
130 inputs."""
131 if not manifest or manifest.get("version") != CACHE_VERSION:
132 return False
133 return (
134 manifest.get("frames_dir") == osp.abspath(frames_dir)
135 and manifest.get("weights") == weights
136 and manifest.get("sport") == sport
137 and manifest.get("frames_in") == frames_in
138 and manifest.get("frames_total") == frames_total

```

```

138)
139
140
141 def reset_cache(cache_dir: str) -> None:
142 """Drop any existing cache so the next run starts clean."""
143 det_jsonl, mpath = _cache_paths(cache_dir)
144 for p in (det_jsonl, mpath):
145 if osp.exists(p):
146 os.remove(p)
147
148
149 def load_and_clean_cache(cache_dir: str):
150 """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
151 rebuild the per-frame detection accumulation, and rewrite the file to exactly that
152 prefix so a trailing half-written line from a crash can't corrupt future appends.
153
154 Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
155 [x, y, score, scale] candidates (accumulated across every window the frame is in,
156 in window order ? identical to a non-resumed run).
157 """
158 det_jsonl, _ = _cache_paths(cache_dir)
159 det_by_fid = defaultdict(list)
160 valid = []
161 if osp.exists(det_jsonl):
162 with open(det_jsonl, "r") as f:
163 for line in f:
164 s = line.strip()
165 if not s:
166 continue
167 try:
168 obj = json.loads(s)
169 except json.JSONDecodeError:
170 break # truncated trailing line (crash mid-
flush)
171 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
172 break
173 valid.append(s)
174 for fid, plain in obj["f"]:
175 det_by_fid[int(fid)].extend([list(p) for p in plain])
176 # Guarantee a clean append boundary by rewriting only the good prefix.
177 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
178 return len(valid), det_by_fid
179
180
181 def _append_windows(fh, objs) -> None:
182 """Append window records as JSONL and durably flush (bounded loss on crash)."""
183 for o in objs:
184 fh.write(json.dumps(o, separators=(",", ":")) + "\n")
185 fh.flush()
186 os.fsync(fh.fileno())
187
188
189 def _atomic_write_trajectory(out_csv: str, rows) -> None:
190 os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
191 tmp = out_csv + ".tmp"
192 with open(tmp, "w", newline="") as f:
193 w = csv.writer(f)
194 w.writerow(["Frame", "Visibility", "X", "Y"])
195 w.writerows(rows)
196 f.flush()
197 os.fsync(f.fileno())
198 os.replace(tmp, out_csv)
199
200
201 # ----- #

```

```

202 # Inference
203 # ----- #
204 def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
205 limit: int = 0, batch_size: int = 8, num_workers: int = 4,
206 log_every_batches: int = 50, cache_dir: str = None,
207 flush_every: int = 1000, fresh: bool = False) -> str:
208 import time
209 import torch
210 from torch.utils.data import DataLoader
211 from detectors import build_detector
212 from trackers import build_tracker
213 from dataloaders import build_img_transforms, build_seq_transforms
214 from dataloaders.dataset_loader import ImageDataset
215 from utils import Center
216
217 cfg = _build_cfg(weights, sport)
218 frames_in = int(cfg["model"]["frames_in"])
219 input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
220 output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
221
222 frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
223 if limit:
224 frames = frames[:limit]
225 if len(frames) < frames_in:
226 raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
{frames_dir}")
227
228 # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
229 samples = []
230 for i in range(len(frames) - frames_in + 1):
231 window = frames[i:i + frames_in]
232 annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
233 samples.append({"frames": window, "annos": annos})
234 windows_total = len(samples)
235
236 # ---- cache setup / resume decision ----- #
237 if cache_dir is None:
238 cache_dir = default_cache_dir(out_csv)
239 os.makedirs(cache_dir, exist_ok=True)
240 det_jsonl, _ = _cache_paths(cache_dir)
241 manifest = None if fresh else read_manifest(cache_dir)
242 resume = manifest_compatible(
243 manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
244 frames_in=frames_in, frames_total=len(frames),
245)
246 if resume:
247 windows_done, det_by_fid = load_and_clean_cache(cache_dir)
248 print(f"[wasb_infer] resuming from cache: {windows_done}/{windows_total} windows
already detected",
249 flush=True)
250 else:
251 if manifest is not None:
252 print("[wasb_infer] cache incompatible with this run -> starting fresh",
flush=True)
253 reset_cache(cache_dir)
254 windows_done, det_by_fid = 0, defaultdict(list)
255
256 base_manifest = {
257 "version": CACHE_VERSION,
258 "frames_dir": osp.abspath(frames_dir),
259 "weights": weights,
260 "sport": sport,

```

```

261 "frames_in": frames_in,
262 "frames_total": len(frames),
263 "windows_total": windows_total,
264 "windows_done": windows_done,
265 }
266 write_manifest(cache_dir, base_manifest)
267
268 # ---- detector pass over the un-cached windows ----- #
269 remaining = samples[windows_done:]
270 if remaining:
271 _, transform_test = build_img_transforms(cfg)
272 try:
273 _, seq_transform_test = build_seq_transforms(cfg)
274 except Exception:
275 seq_transform_test = None
276 ds = ImageDataset(cfg, remaining, input_wh, output_wh,
277 transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
278 loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
279 detector = build_detector(cfg)
280
281 t0 = time.time()
282 done = windows_done
283 win_idx = windows_done
284 log_every = max(1, log_every_batches)
285 flush_n = max(1, flush_every)
286 pending = []
287 print(f"[wasb_infer] detecting on {len(remaining)} remaining windows ")
288 f"(total {windows_total}, batch={batch_size}, workers={num_workers})...",
flush=True)
289 det_f = open(det_jsonl, "a")
290 try:
291 with torch.no_grad():
292 for bi, batch in enumerate(loader):
293 imgs, _hms, trans = batch[0], batch[1], batch[2]
294 img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
295 batch_results, _ = detector.run_tensor(imgs, trans)
296 nb = imgs.shape[0]
297 for ib in range(nb):
298 frames_payload = []
299 for ie in sorted(batch_results[ib].keys()):
300 p = img_paths[ie][ib]
301 fid = _frame_id(p)
302 plain = [_det_plain(d) for d in batch_results[ib][ie]]
303 frames_payload.append([fid, plain])
304 det_by_fid[fid].extend(plain)
305 pending.append({"w": win_idx, "f": frames_payload})
306 win_idx += 1
307 done += nb
308 if len(pending) >= flush_n or done >= windows_total:
309 _append_windows(det_f, pending)
310 pending = []
311 base_manifest["windows_done"] = done
312 write_manifest(cache_dir, base_manifest)
313 if (bi + 1) % log_every == 0 or done >= windows_total:
314 el = time.time() - t0
315 new_done = done - windows_done
316 rate = new_done / el if el > 0 else 0.0
317 eta = (windows_total - done) / rate if rate > 0 else 0.0
318 print(f"[wasb_infer] {done}/{windows_total} "
319 f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
320 f"elapsed {el:.0f}s | ETA {eta:.0f}s", flush=True)

```

```

321 if pending:
322 _append_windows(det_f, pending)
323 base_manifest["windows_done"] = done
324 write_manifest(cache_dir, base_manifest)
325 finally:
326 det_f.close()
327 print(f"[wasb_infer] detection done in {time.time() - t0:.0f}s; running
tracker...", flush=True)
328 else:
329 print("[wasb_infer] detector cache already complete -> tracker-only re-run",
flush=True)
330
331 # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
332 tracker = build_tracker(cfg)
333 tracker.refresh()
334 rows = []
335 for fid in sorted(det_by_fid.keys()):
336 r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
337 rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
338
339 _atomic_write_trajectory(out_csv, rows)
340 visible = sum(1 for _, v, *_ in rows if v)
341 print(f"[wasb_infer] wrote {len(rows)} rows -> {out_csv}")
342 print(f"[wasb_infer] visible-shuttle frames: {visible}/{len(rows)}")
343 return out_csv
344
345
346 def extract_frames(video_path: str, frames_dir: str) -> str:
347 """Decode a video to PNG frames named by source frame index (00000000.png ...).
348
349 Idempotent: if the dir already holds the expected number of frames (matching the
350 container's reported frame count) we skip re-decoding ? re-extraction after a
351 reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
352 extracting with ffmpeg on the host and passing --frames_dir.
353 """
354 import cv2
355 os.makedirs(frames_dir, exist_ok=True)
356 cap = cv2.VideoCapture(video_path)
357 if not cap.isOpened():
358 raise SystemExit(f"cannot open video: {video_path}")
359 expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
360 existing = len(glob.glob(osp.join(frames_dir, "*.png")))
361 if expected > 0 and existing >= expected:
362 cap.release()
363 print(f"[wasb_infer] reusing {existing} already-extracted frames ->
{frames_dir}")
364 return frames_dir
365 i = 0
366 while True:
367 ok, frame = cap.read()
368 if not ok:
369 break
370 cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
371 i += 1
372 cap.release()
373 print(f"[wasb_infer] extracted {i} frames -> {frames_dir}")
374 return frames_dir
375
376
377 def main():
378 ap = argparse.ArgumentParser()
379 ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
380 ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
381 ap.add_argument("--frames_out_dir", help="where to extract frames when --video is

```

```

used")
382 ap.add_argument("--weights", required=True)
383 ap.add_argument("--out", required=True)
384 ap.add_argument("--sport", default="badminton")
385 ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
386 ap.add_argument("--batch-size", type=int, default=8)
387 ap.add_argument("--num-workers", type=int, default=4)
388 ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
389 ap.add_argument("--cache-dir", default=None,
390 help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
391 ap.add_argument("--flush-every", type=int, default=1000,
392 help="fsync the detector cache every N windows (bounds reboot
loss)")
393 ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
394 args = ap.parse_args()
395
396 frames_dir = args.frames_dir
397 if args.video:
398 frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
399 if not frames_dir:
400 raise SystemExit("provide --frames_dir or --video")
401 run(frames_dir, args.weights, args.out, args.sport, args.limit,
402 batch_size=args.batch_size, num_workers=args.num_workers,
403 log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
404 flush_every=args.flush_every, fresh=args.fresh)
405
406
407 if __name__ == "__main__":
408 main()
409

```

#### 14. assistant (2026-06-06T15:27:06.670Z)

```

wasb_infer confirmed: `run`, `_DET_FILE='det_raw.jsonl'`, `_MANIFEST_FILE='manifest.json'`,
`CACHE_VERSION=1`, `load_and_clean_cache` (contiguous prefix, w==line index, rewrites),
`manifest_compatible`, `_atomic_write_text`, `_atomic_write_trajectory`, `_dets_to_tracker` (xy
as np.array), runner.gpus=[0], extract_frames cv2-PNG. det_raw.jsonl shape `{"w":...,
"f":[[[fid,[x,y,score,scale],...]],...]}` matches $. manifest.json fields match $. All accurate.

```

Now the eval modules in parallel.

#### 15. user (2026-06-06T15:27:07.619Z)

```

1 """Score the pipeline's detected rallies against ground-truth annotations.
2
3 Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4 so scoring is fast, deterministic, and free. Two prediction "stages" can be
5 scored:
6
7 - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
8 the shuttle/person detector found, before AI confirmation.
9 - ``final`` ? confirmed play segments (``play_segments`` table). After the
10 AI handoff trimmed/validated boundaries.
11
12 Comparing the two tells you whether the weak link is *detection* (candidates
13 already miss rallies) or *segmentation* (candidates catch them but final is
14 wrong) ? see docs/EVALUATION.md.
15 """
16
17 import logging

```



```

18 from dataclasses import dataclass, field
19 from typing import List, Dict, Optional
20
21 from backend.database import Database
22 from backend.eval import metrics
23 from backend.eval.metrics import Interval, Score, MatchResult
24
25 logger = logging.getLogger(__name__)
26
27 STAGES = ("candidates", "final")
28
29
30 @dataclass
31 class StageReport:
32 stage: str
33 pred_intervals: List[Interval]
34 score: Score
35 match_result: MatchResult
36 pr_curve: List[Score] = field(default_factory=list)
37
38
39 @dataclass
40 class EvalReport:
41 video_id: str
42 iou_threshold: float
43 gt_intervals: List[Interval]
44 stages: Dict[str, StageReport] = field(default_factory=dict)
45
46
47 def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
48 """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
49 ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
50 ivs.sort(key=lambda iv: iv[0])
51 return ivs
52
53
54 def get_predictions(db: Database, video_id: str, stage: str,
55 detector: Optional[str] = None) -> List[Interval]:
56 """Fetch predicted rally intervals for one stage from the DB."""
57 if stage == "candidates":
58 rows = db.query_candidate_segments(video_id, detector=detector)
59 elif stage == "final":
60 rows = db.query_play_segments(video_id, {})
61 else:
62 raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
63 return _rows_to_intervals(rows)
64
65
66 def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
67 stages: List[str], iou_threshold: float = 0.5,
68 detector: Optional[str] = None,
69 with_pr_curve: bool = False) -> EvalReport:
70 """Score the requested stages for one video against ground truth."""
71 report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
72 gt_intervals=gt_intervals)
73 for stage in stages:
74 preds = get_predictions(db, video_id, stage, detector=detector)
75 mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76 s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77 curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78 report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79 score=s, match_result=mr, pr_curve=curve)
80 return report
81
82

```

```

83 # ----- reporting
84
85 def _fmt_ts(seconds: float) -> str:
86 seconds = max(0, int(seconds))
87 h, rem = divmod(seconds, 3600)
88 m, s = divmod(rem, 60)
89 return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92 def report_to_dict(report: EvalReport) -> dict:
93 """JSON-serialisable view of an EvalReport."""
94 out = {
95 "video_id": report.video_id,
96 "iou_threshold": report.iou_threshold,
97 "num_ground_truth": len(report.gt_intervals),
98 "stages": {},
99 }
100 for stage, sr in report.stages.items():
101 out["stages"][stage] = {
102 "score": sr.score.as_dict(),
103 "pr_curve": [s.as_dict() for s in sr.pr_curve],
104 }
105 return out
106
107
108 def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109 """Human-readable console report for one video."""
110 lines = [
111 "=" * 64,
112 f" Rally evaluation ? video '{report.video_id}'",
113 f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold: {report.iou_threshold}",
114 "=" * 64,
115 f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116 f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117 "-" * 64,
118]
119 for stage in report.stages:
120 s = report.stages[stage].score
121 lines.append(
122 f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4} "
123 f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124 f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125)
126 lines.append("=" * 64)
127
128 if show_failures:
129 for stage, sr in report.stages.items():
130 misses = [report.gt_intervals[i] for i in sr.match_result.unmatched_gt_indices]
131 fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132 lines.append(f"\n [{stage}] missed rallies (false negatives): {len(misses)}")
133 for iv in misses:
134 lines.append(f" - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135 lines.append(f" [{stage}] spurious detections (false positives): {len(fps)}")
136 for iv in fps:
137 lines.append(f" - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139 return "\n".join(lines)
140

```

```

1 """Pure temporal interval-matching metrics for rally evaluation.
2
3 No I/O, no DB, no global state ? every function here is deterministic and unit
4 tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5 start < end. We score a list of *predicted* rally intervals against a list of
6 *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7 """
8
9 from dataclasses import dataclass, field
10 from typing import List, Tuple, Optional
11
12 Interval = Tuple[float, float]
13
14
15 def interval_iou(a: Interval, b: Interval) -> float:
16 """Temporal intersection-over-union of two intervals.
17
18 Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19 overlap / union, both measured as durations on the timeline.
20 """
21 a_start, a_end = a
22 b_start, b_end = b
23 if a_end <= a_start or b_end <= b_start:
24 return 0.0
25 inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26 if inter <= 0.0:
27 return 0.0
28 union = (a_end - a_start) + (b_end - b_start) - inter
29 return inter / union if union > 0 else 0.0
30
31
32 @dataclass
33 class Match:
34 """One matched predicted?ground-truth pair."""
35 pred_index: int
36 gt_index: int
37 iou: float
38
39
40 @dataclass
41 class MatchResult:
42 """Outcome of matching predictions to ground truth at a given IoU threshold."""
43 matches: List[Match] = field(default_factory=list)
44 unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45 unmatched_gt_indices: List[int] = field(default_factory=list) # false negatives
46 (misses)
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49 iou_threshold: float = 0.5) -> MatchResult:
50 """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52 Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53 are claimed highest-IoU first, and each prediction/GT can be used once. This
54 keeps a single prediction from being credited for two ground-truth rallies
55 (and vice versa).
56 """
57 candidates = []
58 for pi, p in enumerate(preds):
59 for gi, g in enumerate(gts):
60 iou = interval_iou(p, g)
61 if iou >= iou_threshold:
62 candidates.append((iou, pi, gi))
63 # Highest IoU first; ties broken by indices for determinism.
64 candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

```

```

65
66 used_pred, used_gt = set(), set()
67 matches: List[Match] = []
68 for iou, pi, gi in candidates:
69 if pi in used_pred or gi in used_gt:
70 continue
71 used_pred.add(pi)
72 used_gt.add(gi)
73 matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
74
75 unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
76 unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
77 return MatchResult(matches=matches,
78 unmatched_pred_indices=unmatched_pred,
79 unmatched_gt_indices=unmatched_gt)
80
81
82 @dataclass
83 class Score:
84 """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
85 iou_threshold: float
86 num_pred: int
87 num_gt: int
88 true_positives: int
89 false_positives: int
90 false_negatives: int
91 precision: float
92 recall: float
93 f1: float
94 mean_iou: float # over matched pairs (0.0 if none)
95 mean_start_error: float # mean |pred_start - gt_start| over matches, seconds
96 mean_end_error: float # mean |pred_end - gt_end| over matches, seconds
97
98 def as_dict(self) -> dict:
99 return self.__dict__.copy()
100
101
102 def score(preds: List[Interval], gts: List[Interval],
103 iou_threshold: float = 0.5,
104 match_result: Optional[MatchResult] = None) -> Score:
105 """Compute precision/recall/F1, mean matched IoU, and boundary errors.
106
107 Pass a precomputed `match_result` to avoid re-matching (e.g. when also
108 extracting the failure lists); otherwise matching is done here.
109 """
110 mr = match_result if match_result is not None else match_intervals(preds, gts,
111 iou_threshold)
112
113 tp = len(mr.matches)
114 fp = len(mr.unmatched_pred_indices)
115 fn = len(mr.unmatched_gt_indices)
116
117 precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
118 recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
119 f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
120 else 0.0
121
122 if mr.matches:
123 mean_iou = sum(m.iou for m in mr.matches) / tp
124 mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
125 mr.matches) / tp
126 mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
127 mr.matches) / tp
128 else:
129 mean_iou = mean_start_err = mean_end_err = 0.0

```

```

126
127 return Score(
128 iou_threshold=iou_threshold,
129 num_pred=len(preds), num_gt=len(gts),
130 true_positives=tp, false_positives=fp, false_negatives=fn,
131 precision=precision, recall=recall, f1=f1,
132 mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
133)
134
135
136 def pr_curve(preds: List[Interval], gts: List[Interval],
137 thresholds: Optional[List[float]] = None) -> List[Score]:
138 """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
139
140 Lets you see strict-vs-lenient behaviour rather than a single operating point.
141 """
142 if thresholds is None:
143 thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
144 return [score(preds, gts, iou_threshold=t) for t in thresholds]
145

```

## 17. user (2026-06-06T15:27:08.423Z)

```

1 """Batch evaluation across a collector export manifest.
2
3 Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4 sports-data-collector's `curate export`, (optionally) process each video through
5 a segmenter so detections land in the indexer DB, then score every video's
6 detected rallies against its ground-truth CSV and aggregate the results.
7
8 This module holds the *pure* pieces (path resolution, stage selection, metric
9 aggregation) so they can be unit-tested without touching video files, the GPU,
10 or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11 """
12
13 import os
14 from typing import Dict, List, Optional
15
16
17 # Segmenters that only ever populate the `final` play_segments table (no raw
18 # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19 _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22 def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23 """Resolve a manifest `local_path` to an absolute path.
24
25 The collector stores paths like ``./data/videos\\shuttle1set_1.mp4`` relative
26 to its own repo root and with Windows separators. Normalise separators, strip
27 a leading ``./``, and resolve relative paths against `videos_root` (the
28 collector root). Returns None for a missing/empty path.
29 """
30 if not local_path:
31 return None
32 p = local_path.replace("\\", "/")
33 if p.startswith("./"):
34 p = p[2:]
35 if os.path.isabs(p):
36 return os.path.normpath(p)
37 return os.path.normpath(os.path.join(videos_root, p))
38
39
40 def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
41 """Pick which prediction stages to score.
42

```

```

43 ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
44 but only ``final`` for segmenters that don't emit candidates. An explicit
45 request ('candidates' | 'final' | 'both') overrides the heuristic.
46 """
47 if requested == "both":
48 return ["candidates", "final"]
49 if requested in ("candidates", "final"):
50 return [requested]
51 # auto
52 if segmenter in _FINAL_ONLY_SEGMENTERS:
53 return ["final"]
54 return ["candidates", "final"]
55
56
57 def _safe_div(num: float, den: float) -> float:
58 return num / den if den else 0.0
59
60
61 def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
62 """Aggregate per-video stage scores into corpus-level metrics.
63
64 `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
65 ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
66 compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
67 a TP-weighted mean IoU, plus the macro mean of per-video F1.
68 """
69 out: Dict[str, dict] = {}
70 for stage in stages:
71 tp = fp = fn = 0
72 iou_weighted = 0.0
73 f1s: List[float] = []
74 n_videos = 0
75 for rep in per_video:
76 sr = rep.get("stages", {}).get(stage)
77 if not sr:
78 continue
79 s = sr["score"]
80 tp += s["true_positives"]
81 fp += s["false_positives"]
82 fn += s["false_negatives"]
83 iou_weighted += s["mean_iou"] * s["true_positives"]
84 f1s.append(s["f1"])
85 n_videos += 1
86 precision = _safe_div(tp, tp + fp)
87 recall = _safe_div(tp, tp + fn)
88 out[stage] = {
89 "videos": n_videos,
90 "tp": tp, "fp": fp, "fn": fn,
91 "micro_precision": precision,
92 "micro_recall": recall,
93 "micro_f1": _safe_div(2 * precision * recall, precision + recall),
94 "macro_f1": _safe_div(sum(f1s), len(f1s)),
95 "mean_iou": _safe_div(iou_weighted, tp),
96 }
97 return out
98
99
100 def format_aggregate(agg: Dict[str, dict]) -> str:
101 """Human-readable corpus-level summary table."""
102 lines = [
103 "=" * 78,
104 " CORPUS AGGREGATE",
105 "=" * 78,
106 f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
107 f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",

```

```

108 "-" * 78,
109]
110 for stage, a in agg.items():
111 lines.append(
112 f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
113 f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
114 f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
115)
116 lines.append("=" * 78)
117 return "\n".join(lines)
118

```

## 18. assistant (2026-06-06T15:27:15.793Z)

Eval core confirmed. Notable verifications:

- harness: `STAGES=('candidates','final')`, `get\_predictions`  
(candidates->`query\_candidate\_segments(detector=)` , final->`query\_play\_segments({})` ALL segs, detector arg ignored for final), `evaluate`, `EvalReport`, `report\_to\_dict`. ?
- metrics: `interval\_iou`, `match\_intervals`, `score`, `Score`, `pr\_curve`, `Match`, `MatchResult`. The map's MatchResult field names say `unmatched\_pred(=FP)` / `unmatched\_gt(=FN)`  
? actual field names are `unmatched\_pred\_indices` / `unmatched\_gt\_indices`. The \$ eval primitives section abbreviates them. Minor.
- batch: `aggregate`, `default\_stages\_for`, `\_FINAL\_ONLY\_SEGMENTERS={motion,gemini}`, `resolve\_video\_path`. ?

Now the regression, calibration, annotations, classifier, training modules.

## 19. user (2026-06-06T15:27:16.742Z)

```

1 """Regression runner ? verify the production tool against golden-set ground truth.
2
3 Implements the explicit ask: *"given a video, get a golden set created and verify
4 its precision and recall against the production tool."* This is deliberately a thin
5 wrapper over machinery that already exists:
6
7 ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8 predictions ? the production pipeline's stored DB rows [backend.eval.harness]
9 comparison ? interval matching + P/R/F1 [backend.eval.metrics]
10 baseline ? a per-video snapshot of "known good" numbers [this module]
11
12 A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13 baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14 later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15 independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16 """
17 import os
18 import json
19 import logging
20 from dataclasses import dataclass, asdict
21 from typing import Dict, List, Optional
22
23 from backend.database import Database
24 from backend.eval import metrics
25 from backend.eval.harness import get_predictions
26 from backend.annotations import annotation_registry
27
28 logger = logging.getLogger(__name__)
29
30 DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
31 DEFAULT_TOLERANCE = 0.05
32
33
34 @dataclass
35 class RegressionResult:
36 video_id: str

```

```

37 stage: str
38 iou_threshold: float
39 precision: float
40 recall: float
41 f1: float
42 # Baseline values compared against (None when no baseline existed yet).
43 baseline_precision: Optional[float]
44 baseline_recall: Optional[float]
45 tolerance: float
46 regressed: bool
47 reasons: List[str]
48
49 def as_dict(self) -> dict:
50 return asdict(self)
51
52
53 # ----- baseline store
54
55 def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
56 """Load the per-video baseline snapshot file (returns {} if absent)."""
57 if not os.path.isfile(path):
58 return {}
59 with open(path) as f:
60 return json.load(f)
61
62
63 def _baseline_key(video_id: str, stage: str) -> str:
64 return f"{video_id}::{stage}"
65
66
67 def save_baseline(video_id: str, stage: str, precision: float, recall: float,
68 f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
69 """Snapshot a video/stage's current numbers as the new 'known good' baseline.
70
71 Use this to (re)baseline after a *legitimate* production improvement, so old
72 baselines don't raise false regressions. Stored as plain JSON for easy diffing.
73 """
74 baselines = load_baselines(path)
75 baselines[_baseline_key(video_id, stage)] = {
76 "video_id": video_id, "stage": stage,
77 "precision": precision, "recall": recall, "f1": f1,
78 }
79 os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
80 with open(path, "w") as f:
81 json.dump(baselines, f, indent=2, sort_keys=True)
82 logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
83 _baseline_key(video_id, stage), precision, recall)
84
85 # ----- the runner
86
87 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
88 stage: str = "final", iou_threshold: float = 0.5,
89 detector: Optional[str] = None,
90 tolerance: float = DEFAULT_TOLERANCE,
91 baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
92 """Score stored predictions for one video against ground truth and compare to
93 baseline.
94
95 `ground_truth` is a list of (start, end) intervals ? typically obtained from an
96 AnnotationProvider (see `regress_with_provider`). A regression is flagged when
97 precision OR recall falls more than `tolerance` below the snapshotted baseline.
98 If no baseline exists yet, `regressed` is False (nothing to compare to) and the
99 caller can choose to snapshot the current numbers via `save_baseline`.
100 """

```



```

100 preds = get_predictions(db, video_id, stage, detector=detector)
101 s = metrics.score(preds, ground_truth, iou_threshold)
102
103 baselines = load_baselines(baseline_path)
104 b = baselines.get(_baseline_key(video_id, stage))
105
106 reasons: List[str] = []
107 regressed = False
108 base_p = base_r = None
109 if b is not None:
110 base_p, base_r = b["precision"], b["recall"]
111 if s.precision < base_p - tolerance:
112 regressed = True
113 reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
114 if s.recall < base_r - tolerance:
115 regressed = True
116 reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
117 else:
118 reasons.append("no baseline recorded for this video/stage ? nothing to compare")
119
120 return RegressionResult(
121 video_id=video_id, stage=stage, iou_threshold=iou_threshold,
122 precision=s.precision, recall=s.recall, f1=s.f1,
123 baseline_precision=base_p, baseline_recall=base_r,
124 tolerance=tolerance, regressed=regressed, reasons=reasons,
125)
126
127
128 def regress_with_provider(db: Database, video_id: str, video_ref: str,
129 tier: str = "file", stage: str = "final",
130 iou_threshold: float = 0.5,
131 detector: Optional[str] = None,
132 tolerance: float = DEFAULT_TOLERANCE,
133 baseline_path: str = DEFAULT_BASELINE_PATH,
134 guideline: Optional[str] = None,
135 sport: str = "badminton",
136 provider_kwargs: Optional[dict] = None) -> RegressionResult:
137 """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
138
139 `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
140 once those providers land). `video_ref` is whatever that provider expects (a CSV
141 path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
142 predictions in the DB (the file MD5).
143 """
144 provider = annotation_registry.get(tier, **(provider_kwargs or {}))
145 gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
146 return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
147 detector=detector, tolerance=tolerance, baseline_path=baseline_path)
148
149
150 def format_result(r: RegressionResult) -> str:
151 verdict = "REGRESSED" if r.regressed else "ok"
152 head = (f"[{verdict}] {r.video_id} ({r.stage}) "
153 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
154 if r.baseline_precision is not None:
155 head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
156 return head + " ".join(f"\n - {why}" for why in r.reasons)
157

```

```

1 """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3 This is the measurement layer for per-video / per-corpus calibration. It answers
4 two questions honestly:
5
6 1. ***"Did precision improve, by how much?"** ? `improvement_report` scores a
7 baseline config vs a tuned config on the same ground truth and reports the
8 before/after metrics and the % delta.
9
10 2. ***"Is that gain real or overfit?"** ? calibration must be measured on data the
11 tuning never saw, otherwise you're just memorising. Two protocols:
12 - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13 train folds, score on the held-out fold. The **generalization_gap**
14 (train ? test) is the overfit alarm.
15 - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16 pick the config on all *other* videos, score on the held-out video. Reports
17 per-video held-out scores + mean±std. This is the number to trust.
18
19 Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20 List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
21 harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
22 """
23 from __future__ import annotations
24
25 import statistics as _st
26 from typing import Callable, Dict, List, Sequence, Tuple, Any
27
28 from backend.eval.metrics import score, Score, Interval
29
30 Config = Any # opaque to this module (e.g. a thresholds
dict)
31 PredictFn = Callable[[Config], List[Interval]]
32
33
34 def pct_improvement(before: float, after: float) -> float:
35 """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
36 if before <= 0:
37 return float("inf") if after > 0 else 0.0
38 return (after - before) / before * 100.0
39
40
41 def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
42 """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43 return score(preds, gts, iou_threshold)
44
45
46 def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47 """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
48 if n <= 0:
49 return []
50 k = max(2, min(k, n)) # need >=2 folds; cap at n
51 folds = [list(range(i, n, k)) for i in range(k)] # round-robin ? balanced,
deterministic
52 splits = []
53 for f in range(k):
54 test = folds[f]
55 train = [i for i in range(n) if i not in set(test)]
56 if test and train:
57 splits.append((train, test))
58 return splits
59
60
61 def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
62 metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:

```

```

63 """Pick the config whose predictions maximise `metric` against `gts`."""
64 best_cfg, best_val = None, -1.0
65 for cfg in configs:
66 val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
67 if val > best_val:
68 best_cfg, best_val = cfg, val
69 return best_cfg, best_val
70
71
72 def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
73 k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
74 """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).
75
76 Per fold: choose the best config on the train rallies, score it on the held-out
77 rallies. A large `generalization_gap` (train >> test) means the thresholds are
78 overfitting that video's quirks.
79
80 Default metric is **recall** on purpose: predictions are global (whole-video
81 windows), so scoring them against a *fold subset* of GT would unfairly count the
82 other rallies' windows as false positives ? precision/F1 are not well-defined
83 per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
84 is the honest within-video generalization signal. For precision/F1, use
85 `leave_one_video_out` (each video scored whole) or `improvement_report` on a
86 held-out video.
87 """
88 splits = kfold_indices(len(gts), k)
89 if not splits:
90 return {"error": "not enough rallies for cross-validation", "n": len(gts)}
91 train_scores, test_scores, picked = [], [], []
92 for train_idx, test_idx in splits:
93 gts_tr = [gts[i] for i in train_idx]
94 gts_te = [gts[i] for i in test_idx]
95 cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
96 te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
97 train_scores.append(tr)
98 test_scores.append(te)
99 picked.append(cfg)
100 mean_test = _st.mean(test_scores)
101 mean_train = _st.mean(train_scores)
102 return {
103 "metric": metric,
104 "folds": len(splits),
105 "mean_test": mean_test,
106 "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
107 "mean_train": mean_train,
108 "generalization_gap": mean_train - mean_test, # >~0.1 = likely overfitting
109 "picked_configs": picked,
110 }
111
112
113 def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
114 metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
Any]:
115 """Leave-one-video-out CV ? the honest metric as the corpus grows.
116
117 `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
118 For each held-out video: pick the config that's best *pooled across the other
119 videos*, then score it on the held-out one. The mean held-out score is the
120 "precision on unseen videos" number; per-video spread shows stability.
121 """
122 if len(videos) < 2:
123 return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
124 per_video = []

```

```

125 for i, held in enumerate(videos):
126 others = [v for j, v in enumerate(videos) if j != i]
127
128 def pooled_score(cfg: Config) -> float:
129 vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
metric)
130 for v in others]
131 return _st.mean(vals)
132
133 best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
t: t[1])
134 held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
iou_threshold), metric)
135 per_video.append({"video": held.get("name", f"video_{i}"),
136 "held_out": held_score, "train_pooled": best_train, "config":
best_cfg})
137 held_scores = [p["held_out"] for p in per_video]
138 train_scores = [p["train_pooled"] for p in per_video]
139 return {
140 "metric": metric,
141 "mean_held_out": _st.mean(held_scores),
142 "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
143 "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
144 "per_video": per_video,
145 }
146
147
148 def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
149 gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
150 """The "+X%" precision headline: baseline vs tuned config on the same GT.
151
152 NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
153 or read the cross-validated `mean_test` from `cross_validate` instead.
154 """
155 b = score(predict_fn(baseline_config), gts, iou_threshold)
156 t = score(predict_fn(tuned_config), gts, iou_threshold)
157 return {
158 "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
159 "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
160 "precision_delta_pts": t.precision - b.precision,
161 "precision_delta_pct": pct_improvement(b.precision, t.precision),
162 "f1_delta_pct": pct_improvement(b.f1, t.f1),
163 }
164

```

## 21. assistant (2026-06-06T15:27:22.595Z)

Both confirmed. regression.py imports `annotation\_registry` from `backend.annotations` (NOT backend.eval.annotations) ?; gate fires on precision OR recall drop, F1 reported not gated ?; first run silently passes (no baseline) ?; `DEFAULT\_TOLERANCE=0.05`, `DEFAULT\_BASELINE\_PATH=output/regression\_baseline.json`, `regress`, `regress\_with\_provider`. ?

calibration.py confirmed: `cross\_validate` (defaults `metric='recall'`, `leave\_one\_video\_out`, `improvement\_report` (optimistic), `select\_best`, `kfold\_indices`, `generalization\_gap >~0.1`. ?

Now the remaining eval modules, database, and DB query methods.

## 22. user (2026-06-06T15:27:23.523Z)

```

1 """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".

```

```

2
3 Wires the already-tested pieces together:
4 WASB trajectory CSV (Frame,Visibility,X,Y)
5 -> TrackNetRunner.trajectory_to_action_windows(cfg) [predict_fn]
6 -> backend.eval.calibration (improvement_report / cross_validate)
7
8 Run (after a full-video WASB pass exists):
9 python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10 --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12 Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13 same GT is an **optimistic** upper bound. The trustworthy number is the
14 cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15 `leave_one_video_out` for precision/F1 generalization).
16 """
17 from __future__ import annotations
18
19 import csv
20 import argparse
21 from typing import List, Tuple, Callable
22
23 from backend.detectors.tracknet_runner import TrackNetRunner
24 from backend.eval.calibration import (
25 select_best, improvement_report, cross_validate, evaluate,
26)
27
28 Interval = Tuple[float, float]
29 Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
30
31 BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
32
33
34 def load_golden_gts(csv_path: str) -> List[Interval]:
35 """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36 gts: List[Interval] = []
37 with open(csv_path, newline="", encoding="utf-8") as f:
38 reader = csv.DictReader(f)
39 reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
40 [])]
41 for row in reader:
42 row = {(k or "").strip().lower(): v for k, v in row.items()}
43 try:
44 s = float((row.get("start_time") or "").strip())
45 e = float((row.get("end_time") or "").strip())
46 except (TypeError, ValueError):
47 continue
48 if e > s:
49 gts.append((s, e))
50 return sorted(gts)
51
52 def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53 """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
54 points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
55
56 def predict_fn(cfg: Config) -> List[Interval]:
57 vt, mg, mwd = cfg
58 wins = TrackNetRunner.trajectory_to_action_windows(
59 points, fps=fps, frame_width=frame_width,
60 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
61)
62 return [(w["start"], w["end"]) for w in wins]

```

```

63
64 return predict_fn
65
66
67 def default_grid() -> List[Config]:
68 return [(vt, mg, mwd)
69 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
70 for mg in (1.0, 2.0, 3.0)
71 for mwd in (1.0, 2.0, 3.0)]
72
73
74 def run(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
75 iou: float = 0.5) -> dict:
76 gts = load_golden_gts(labels_csv)
77 if not gts:
78 raise SystemExit(f"no usable golden rallies in {labels_csv}")
79 predict_fn = make_predict_fn(trajjectory_csv, fps, frame_width)
80 grid = default_grid()
81
82 base = evaluate(predict_fn(BASELINE), gts, iou)
83 best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
84 fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC
(fit on full GT)
85 cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-
out recall
86
87 print("=== WASB rally-segmentation calibration ===")
88 print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
89 print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
90 f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
91 print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
92 print(f" precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
93 f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
94 print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
95 f"({fit['f1_delta_pct']:+.1f}%)")
96 print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
97 f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
98 f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
99 print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
100 return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
101
102
103 def main():
104 ap = argparse.ArgumentParser()
105 ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
106 ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
107 ap.add_argument("--fps", type=float, default=59.94)
108 ap.add_argument("--frame-width", type=float, default=1920.0)
109 ap.add_argument("--iou", type=float, default=0.5)
110 args = ap.parse_args()
111 run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
112
113
114 if __name__ == "__main__":
115 main()
116

```

```

1 """Export tuned WASB rally segments for manual eyeball verification.
2
3 After calibration picks a tuned threshold config, this turns a cached WASB
4 trajectory into the actual rally [start,end] segments that config produces, so the
5 user can answer "are the rallies actually better?" ? by scrubbing the video to each
6 segment, by diffing tuned-vs-golden quantitatively, or by cutting clips.
7
8 Reuses (no new windowing/metrics/cutting logic):
9 - calibrate_wasb.make_predict_fn -> trajectory -> segments for a given config
10 - eval.metrics.match_intervals/score -> tuned-vs-golden IoU comparison
11 - tools.rally_slicer.slice_rallies -> optional clip cutting (the exported CSV is
12 written in the golden/slicer schema, so it feeds straight back in)
13
14 Run (uses the tuned config from the first calibration by default):
15 python -m backend.eval.export_wasb_segments --trajectory
~/clips/sample_long_traj.csv \
16 --labels "Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-
width 1920
17 # add --video <mp4> --slice to also cut one padded clip per tuned rally
18 """
19 from __future__ import annotations
20
21 import csv
22 import os
23 import os.path as osp
24 import argparse
25 from typing import List, Tuple, Dict, Optional
26
27 from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
28 from backend.eval.metrics import match_intervals, score
29
30 Interval = Tuple[float, float]
31 Config = Tuple[float, float, float]
32
33 # The tuned config from the first per-video calibration (velocity, merge_gap, min_dur);
34 # F1 0.588 (baseline) -> 0.746. Override on the CLI to export a different operating
point.
35 TUNED: Config = (0.05, 1.0, 1.0)
36
37 SEG_FIELDS = ["rally_number", "start_time", "end_time", "duration", "start_mmss",
"end_mmss"]
38 COMP_FIELDS = ["type", "ref", "start_time", "end_time", "mmss", "status",
39 "iou", "pred_start", "pred_end", "start_err_s", "end_err_s"]
40
41
42 def seconds_to_mmss(t: float) -> str:
43 """Human-scrubbable timestamp, e.g. 106.031 -> '1:46.031'."""
44 t = max(0.0, float(t))
45 m = int(t // 60)
46 return f"{m}:{t - m * 60:06.3f}"
47
48
49 def segments_to_rows(segments: List[Interval]) -> List[Dict[str, str]]:
50 """Number segments 1..N in time order; emit the golden/slicer CSV schema."""
51 rows = []
52 for i, (s, e) in enumerate(segments, start=1):
53 rows.append({
54 "rally_number": str(i),
55 "start_time": f"{s:.3f}",
56 "end_time": f"{e:.3f}",
57 "duration": f"{e - s:.3f}",
58 "start_mmss": seconds_to_mmss(s),
59 "end_mmss": seconds_to_mmss(e),
60 })
61 return rows

```

```

62
63
64 def _ensure_parent(path: str) -> None:
65 os.makedirs(osp.dirname(osp.abspath(path)) or ".", exist_ok=True)
66
67
68 def write_segments_csv(path: str, segments: List[Interval]) -> str:
69 """Write segments in the golden schema (rally_number/start_time/end_time + mm:ss).
70
71 The schema is what tools.rally_slicer.load_rally_labels consumes, so the file
72 doubles as input to clip cutting.
73 """
74 _ensure_parent(path)
75 with open(path, "w", newline="", encoding="utf-8") as f:
76 w = csv.DictWriter(f, fieldnames=SEG_FIELDS)
77 w.writeheader()
78 w.writerows(segments_to_rows(segments))
79 return path
80
81
82 def build_comparison(preds: List[Interval], gts: List[Interval],
83 iou_threshold: float = 0.5) -> dict:
84 """Golden-centric comparison: per golden rally, did a tuned segment match (IoU),
85 plus the boundary errors and any extra (false-positive) tuned segments."""
86 mr = match_intervals(preds, gts, iou_threshold)
87 sc = score(preds, gts, iou_threshold, match_result=mr)
88 by_gt = {m.gt_index: m for m in mr.matches}
89
90 rows: List[Dict[str, str]] = []
91 for gi, (gs, ge) in enumerate(gts, start=0):
92 m = by_gt.get(gi)
93 if m is not None:
94 ps, pe = preds[m.pred_index]
95 rows.append({
96 "type": "golden", "ref": str(gi + 1),
97 "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
98 "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
99 "status": "matched", "iou": f"{m.iou:.3f}",
100 "pred_start": f"{ps:.3f}", "pred_end": f"{pe:.3f}",
101 "start_err_s": f"{ps - gs:+.3f}", "end_err_s": f"{pe - ge:+.3f}",
102 })
103 else:
104 rows.append({
105 "type": "golden", "ref": str(gi + 1),
106 "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
107 "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
108 "status": "MISS", "iou": "0.000",
109 "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
110 })
111 for pi in mr.unmatched_pred_indices:
112 # extra tuned segments = false
113 positives
114 ps, pe = preds[pi]
115 rows.append({
116 "type": "pred_extra", "ref": str(pi + 1),
117 "start_time": f"{ps:.3f}", "end_time": f"{pe:.3f}",
118 "mmss": f"{seconds_to_mmss(ps)}-{seconds_to_mmss(pe)}",
119 "status": "false_positive", "iou": "",
120 "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
121 })
122 return {"score": sc, "rows": rows}
123
124 def write_comparison_csv(path: str, comparison: dict) -> str:
125 _ensure_parent(path)
126 with open(path, "w", newline="", encoding="utf-8") as f:

```



```

126 w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
127 w.writeheader()
128 w.writerows(comparison["rows"])
129 return path
130
131
132 def _with_suffix(path: str, suffix: str) -> str:
133 root, ext = osp.splitext(path)
134 return root + suffix + ext
135
136
137 def run(trajecory_csv: str, fps: float, frame_width: float, cfg: Config = TUNED,
138 out_csv: str = "output/wasb_tuned_segments.csv", labels_csv: Optional[str] =
None,
139 iou: float = 0.5, baseline: bool = False,
140 comparison_out: Optional[str] = None) -> dict:
141 predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
142 tuned_segs = predict_fn(cfg)
143 write_segments_csv(out_csv, tuned_segs)
144 total = sum(e - s for s, e in tuned_segs)
145 print("=== WASB tuned-segment export ===")
146 print(f"tuned cfg {cfg}: {len(tuned_segs)} rally segments, {total:.1f}s of play ->
{out_csv}")
147 result: dict = {"tuned_cfg": cfg, "num_tuned": len(tuned_segs), "out_csv": out_csv}
148
149 if baseline:
150 base_segs = predict_fn(BASELINE)
151 base_out = _with_suffix(out_csv, "_baseline")
152 write_segments_csv(base_out, base_segs)
153 print(f"baseline cfg {BASELINE}: {len(base_segs)} segments -> {base_out}")
154 result["num_baseline"] = len(base_segs)
155 result["baseline_csv"] = base_out
156
157 if labels_csv:
158 gts = load_golden_gts(labels_csv)
159 if not gts:
160 raise SystemExit(f"no usable golden rallies in {labels_csv}")
161 comp = build_comparison(tuned_segs, gts, iou)
162 sc = comp["score"]
163 comp_out = comparison_out or _with_suffix(out_csv, "_vs_golden")
164 write_comparison_csv(comp_out, comp)
165 print(f"\n--- tuned vs golden ({len(gts)} rallies, IoU>={iou}) ---")
166 print(f"matched (TP) {sc.true_positives} | misses (FN) {sc.false_negatives} | "
167 f"extra (FP) {sc.false_positives}")
168 print(f"precision {sc.precision:.3f} | recall {sc.recall:.3f} | f1 {sc.f1:.3f} |
"
169 f"meanIoU {sc.mean_iou:.3f}")
170 print(f"mean boundary error: start {sc.mean_start_error:.2f}s | end
{sc.mean_end_error:.2f}s")
171 print(f"per-rally detail -> {comp_out}")
172 result["comparison"] = sc.as_dict()
173 result["comparison_csv"] = comp_out
174 return result
175
176
177 def _maybe_slice(video: str, segments_csv: str, clips_dir: str) -> None:
178 """Cut one padded clip per exported rally (reuses the rally slicer)."""
179 from backend.tools.rally_slicer import slice_rallies
180 written = slice_rallies(video, segments_csv, ["all"], clips_dir)
181 print(f"\ncut {len(written)} clip(s) -> {clips_dir}")
182
183
184 def main() -> None:
185 ap = argparse.ArgumentParser(description="Export tuned WASB rally segments for
eyeballing.")

```

```

186 ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
187 ap.add_argument("--fps", type=float, default=59.94)
188 ap.add_argument("--frame-width", type=float, default=1920.0)
189 ap.add_argument("--velocity-thresh", type=float, default=TUNED[0])
190 ap.add_argument("--merge-gap", type=float, default=TUNED[1])
191 ap.add_argument("--min-duration", type=float, default=TUNED[2])
192 ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden
comparison")
193 ap.add_argument("--iou", type=float, default=0.5)
194 ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
195 ap.add_argument("--comparison-out", default=None)
196 ap.add_argument("--baseline", action="store_true", help="also export baseline-config
segments")
197 ap.add_argument("--video", default=None, help="source video (with --slice) to cut
clips from")
198 ap.add_argument("--slice", action="store_true", help="cut one padded clip per tuned
rally")
199 ap.add_argument("--clips-dir", default="output/wasb_tuned_clips")
200 args = ap.parse_args()
201
202 cfg = (args.velocity_thresh, args.merge_gap, args.min_duration)
203 res = run(args.trajectory, args.fps, args.frame_width, cfg=cfg, out_csv=args.out,
204 labels_csv=args.labels, iou=args.iou, baseline=args.baseline,
205 comparison_out=args.comparison_out)
206 if args.slice:
207 if not args.video:
208 raise SystemExit("--slice requires --video")
209 _maybe_slice(args.video, res["out_csv"], args.clips_dir)
210
211
212 if __name__ == "__main__":
213 main()
214

```

#### 24. user (2026-06-06T15:27:24.645Z)

```

1 """Build a training set for the offline, API-free learned rally segmenter.
2
3 Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4 a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5 produces candidate rally windows with features; the collector's ground-truth
6 rally intervals tell us which windows are real. A lightweight classifier learns
7 that mapping, replacing the paid Gemini refinement with a local model.
8
9 This module is the *data layer* ? pure and unit-tested:
10
11 - `label_candidates` ? generic: label windows positive/negative by whether they
12 match a ground-truth rally (same IoU matching the evaluator uses, so training
13 labels are consistent with how we score).
14 - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15 persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16 - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18 No model training or I/O here; that lives in the trainer/segmenter once the
19 substrate is chosen from the Phase 3 candidate-recall result.
20 """
21
22 from typing import Callable, Dict, List, Tuple
23
24 from backend.eval.metrics import Interval, match_intervals
25
26 # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27 # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28 YOLO_FEATURE_NAMES = [

```

```

29 "duration", "peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count",
30]
31
32
33 def extract_yolo_features(row: Dict) -> List[float]:
34 """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36 `stats` is already JSON-deserialized by the DB layer. Missing fields default
37 to 0.0 so a sparse/old row still yields a well-formed vector.
38 """
39 stats = row.get("stats") or {}
40 duration = float(row["end_time"]) - float(row["start_time"])
41 return [
42 duration,
43 float(stats.get("peak_velocity", 0.0)),
44 float(stats.get("mean_velocity", 0.0)),
45 float(stats.get("active_frame_count", 0)),
46 float(stats.get("track_id_count", 0)),
47]
48
49
50 def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51 iou_threshold: float = 0.5) -> List[int]:
52 """Label each candidate window 1 (real rally) or 0 (spurious).
53
54 A window is positive iff it is matched one-to-one to a ground-truth rally at
55 `iou_threshold` ? the *same* matching the evaluator uses for true positives,
56 so the classifier is trained on exactly what we score. Substrate-agnostic:
57 works for YOLO or WASB-derived windows alike.
58 """
59 mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
60 positive = {m.pred_index for m in mr.matches}
61 return [1 if i in positive else 0 for i in range(len(candidate_intervals))]
62
63
64 def build_training_set(
65 candidate_rows: List[Dict],
66 gt_intervals: List[Interval],
67 iou_threshold: float = 0.5,
68 feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
69) -> Tuple[List[List[float]], List[int], List[Interval]]:
70 """Turn one video's candidate rows + ground truth into (X, y, intervals).
71
72 `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
73 features without changing the labeling logic. Returns the feature matrix `X`,
74 binary labels `y`, and the candidate intervals (handy for downstream merging).
75 """
76 intervals: List[Interval] = [
77 (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
78]
79 y = label_candidates(intervals, gt_intervals, iou_threshold)
80 X = [feature_fn(r) for r in candidate_rows]
81 return X, y, intervals
82

```

## 25. assistant (2026-06-06T15:27:34.075Z)

Several confirmations and one notable discrepancy:

- calibrate\_wasb: `BASELINE=(0.02,2.0,2.0)`, `default\_grid` (5×3×3=45 cfigs ?),  
`make\_predict\_fn`, `load\_golden\_gts` (silently skips bad rows ?), CLI defaults fps=59.94/frame-width=1920 ?.
- export\_wasb\_segments: `TUNED=(0.05,1.0,1.0)`, `SEG\_FIELDS`, `COMP\_FIELDS`, `build\_comparison`,  
`--slice` requires `--video` ?.
- training: `YOLO\_FEATURE\_NAMES` (5-d ?), `extract\_yolo\_features`, `label\_candidates`,

```
`build_training_set`.
```

**\*\*DISCREPANCY in training.py\*\*:** The map says `extract\_yolo\_features` "? assumes `row['stats']` already dict; raw JSON str -> all-zero". Looking at the actual code: `stats = row.get("stats")` or `{}` then `stats.get(...)` . If `stats` were a raw JSON string, `stats.get` would raise `AttributeError (str has no .get)`, NOT yield all-zero. So the "all-zero" claim is wrong ? it would crash. But the docstring confirms "stats is already JSON-deserialized by the DB layer." The gotcha is mildly misleading. I'll note and soften it.

Now let me verify the annotations, classifier, database, providers, sports.

## 26. user (2026-06-06T15:27:35.602Z)

```
1 """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3 The offline learned segmenter (ADR-004) must run locally with no external API and
4 no heavy ML stack ? the project already depends on numpy, and sklearn is not
5 installed. This implements just enough: standardized features, balanced-class
6 gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7 model is a small, human-readable, version-controllable file.
8
9 It deliberately stays simple: the rally-window features are few and roughly
10 linearly separable (high player motion + sustained activity => real rally), so a
11 well-regularized linear model is a sensible, inspectable baseline. Swap in a
12 richer model later behind the same fit/predict_proba/save/load interface.
13 """
14
15 import json
16 from typing import List, Optional
17
18 import numpy as np
19
20
21 class LogisticRegression:
22 def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23 self.lr = lr
24 self.epochs = epochs
25 self.l2 = l2
26 self.w: Optional[np.ndarray] = None
27 self.b: float = 0.0
28 self.mean: Optional[np.ndarray] = None
29 self.std: Optional[np.ndarray] = None
30 self.feature_names: Optional[List[str]] = None
31
32 @staticmethod
33 def _sigmoid(z: np.ndarray) -> np.ndarray:
34 # Clip for numerical stability (avoid overflow in exp).
35 return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37 def _standardize(self, X: np.ndarray) -> np.ndarray:
38 return (X - self.mean) / self.std
39
40 def fit(self, X, y, feature_names: Optional[List[str]] = None,
41 class_weight: str = "balanced") -> "LogisticRegression":
42 """Fit with optional balanced class weighting (rally windows are imbalanced
43 ? many spurious candidates per real rally)."""
44 X = np.asarray(X, dtype=float)
45 y = np.asarray(y, dtype=float)
46 if X.ndim != 2 or X.shape[0] == 0:
47 raise ValueError("X must be a non-empty 2D array")
48 self.feature_names = feature_names
49
50 self.mean = X.mean(axis=0)
51 self.std = X.std(axis=0)
52 self.std[self.std == 0] = 1.0 # guard constant features
```

```

53 Xs = self._standardize(X)
54 n, d = Xs.shape
55
56 if class_weight == "balanced":
57 n_pos = max(1, int(y.sum()))
58 n_neg = max(1, int((1 - y).sum()))
59 w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60 else:
61 w_pos = w_neg = 1.0
62 sw = np.where(y == 1, w_pos, w_neg)
63
64 self.w = np.zeros(d)
65 self.b = 0.0
66 for _ in range(self.epochs):
67 p = self._sigmoid(Xs @ self.w + self.b)
68 err = (p - y) * sw
69 grad_w = Xs.T @ err / n + self.l2 * self.w
70 grad_b = float(err.mean())
71 self.w -= self.lr * grad_w
72 self.b -= self.lr * grad_b
73 return self
74
75 def predict_proba(self, X) -> np.ndarray:
76 if self.w is None:
77 raise RuntimeError("model is not fitted")
78 Xs = self._standardize(np.asarray(X, dtype=float))
79 return self._sigmoid(Xs @ self.w + self.b)
80
81 def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82 return (self.predict_proba(X) >= threshold).astype(int)
83
84 # --- persistence (small JSON, no pickle/joblib) -----
85
86 def to_dict(self) -> dict:
87 return {
88 "model": "logreg",
89 "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90 "w": self.w.tolist() if self.w is not None else None,
91 "b": self.b,
92 "mean": self.mean.tolist() if self.mean is not None else None,
93 "std": self.std.tolist() if self.std is not None else None,
94 "feature_names": self.feature_names,
95 }
96
97 @classmethod
98 def from_dict(cls, d: dict) -> "LogisticRegression":
99 m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100 m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101 m.b = float(d.get("b", 0.0))
102 m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103 m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104 m.feature_names = d.get("feature_names")
105 return m
106
107 def save(self, path: str) -> None:
108 with open(path, "w") as f:
109 json.dump(self.to_dict(), f, indent=2)
110
111 @classmethod
112 def load(cls, path: str) -> "LogisticRegression":
113 with open(path) as f:
114 return cls.from_dict(json.load(f))
115

```

## 27. user (2026-06-06T15:27:37.517Z)

```
1 """Load hand-annotated ground-truth rally timestamps from CSV.
2
3 Annotation file format (one file per video):
4
5 start,end
6 00:01:12,00:01:39
7 102.5,131.0
8
9 - A header row `start,end` is optional (auto-detected and skipped).
10 - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11 `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12 - Blank lines and `#` comment lines are ignored.
13 """
14
15 import csv
16 import logging
17 from typing import List, Tuple
18
19 logger = logging.getLogger(__name__)
20
21 Interval = Tuple[float, float]
22
23
24 def parse_timestamp(value: str) -> float:
25 """Parse a timestamp string to float seconds.
26
27 Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28 Raises ValueError on anything unparseable so bad annotations fail loudly.
29 """
30 value = value.strip()
31 if not value:
32 raise ValueError("empty timestamp")
33 if ":" in value:
34 parts = value.split(":")
35 if len(parts) > 3:
36 raise ValueError(f"too many ':' segments in timestamp '{value}'")
37 total = 0.0
38 for part in parts:
39 # most-significant first (H, M, S)
40 total = total * 60 + float(part)
41 return total
42 return float(value)
43
44 def load_annotations(csv_path: str) -> List[Interval]:
45 """Load and validate ground-truth rally intervals from a CSV file.
46
47 Returns intervals sorted by start time. Raises ValueError on malformed rows
48 (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49 than silently skewing scores.
50 """
51 intervals: List[Interval] = []
52 with open(csv_path, newline="") as f:
53 reader = csv.reader(f)
54 for lineno, row in enumerate(reader, start=1):
55 if not row:
56 continue
57 first = row[0].strip()
58 if not first or first.startswith("#"):
59 continue
60 # Skip an optional header row.
61 if first.lower() in ("start", "start_time"):
62 continue
63 if len(row) < 2:
```

```

64 raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65 try:
66 start = parse_timestamp(row[0])
67 end = parse_timestamp(row[1])
68 except ValueError as e:
69 raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70 if start >= end:
71 raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end}")
72 intervals.append((start, end))
73
74 intervals.sort(key=lambda iv: iv[0])
75 logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76 return intervals
77
78
79 def write_scaffold(csv_path: str) -> None:
80 """Write an empty annotation file with just the header, for hand-filling."""
81 with open(csv_path, "w", newline="") as f:
82 writer = csv.writer(f)
83 writer.writerow(["start", "end"])
84

```

## 28. user (2026-06-06T15:27:37.741Z)

```

1 import sqlite3
2 import json
3 import os
4 import logging
5 from typing import List, Dict, Any, Optional
6
7 logger = logging.getLogger(__name__)
8
9 class Database:
10 def __init__(self, db_path: str):
11 self.db_path = db_path
12 self._init_db()
13
14 def _get_connection(self):
15 conn = sqlite3.connect(self.db_path)
16 conn.row_factory = sqlite3.Row
17 # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
18 conn.execute("PRAGMA foreign_keys = ON")
19 return conn
20
21 def _init_db(self):
22 """Initializes tables for videos, play_segments, and events."""
23 with self._get_connection() as conn:
24 cursor = conn.cursor()
25
26 # Videos Table
27 cursor.execute("""
28 CREATE TABLE IF NOT EXISTS videos (
29 id TEXT PRIMARY KEY,
30 filepath TEXT NOT NULL,
31 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32 status TEXT NOT NULL,
33 validation_results TEXT
34)
35 """)
36
37 # Play Segments Table (Extensible metadata JSON)
38 cursor.execute("""
39 CREATE TABLE IF NOT EXISTS play_segments (
40 id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

41 video_id TEXT NOT NULL,
42 start_time REAL NOT NULL,
43 end_time REAL NOT NULL,
44 duration REAL NOT NULL,
45 server TEXT,
46 receiver TEXT,
47 metadata TEXT,
48 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49)
50 """)
51
52 # Events Table
53 cursor.execute("""
54 CREATE TABLE IF NOT EXISTS events (
55 id INTEGER PRIMARY KEY AUTOINCREMENT,
56 segment_id INTEGER NOT NULL,
57 timestamp REAL NOT NULL,
58 type TEXT NOT NULL,
59 player TEXT,
60 details TEXT,
61 FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
62)
63 """)
64
65 # Versioned migrations live here. PRAGMA user_version is the schema
66 # contract ? bump it for every change and add an ordered `if version < N`
67 # block below. Tests assert the expected version, so accidental drift fails
68 CI.
69
70 version = cursor.execute("PRAGMA user_version").fetchone()[0]
71
72 if version < 1:
73 # v1: raw candidate detections (pre-confirmation), detector-agnostic.
74 # Common fields are columns; detector-specific metrics go in `stats`
75 # so a new segmenter reuses this table by setting its own
76 `detector`/`stats`.
77 cursor.execute("""
78 CREATE TABLE IF NOT EXISTS candidate_segments (
79 id INTEGER PRIMARY KEY AUTOINCREMENT,
80 video_id TEXT NOT NULL,
81 detector TEXT NOT NULL,
82 chunk_index INTEGER,
83 start_time REAL NOT NULL,
84 end_time REAL NOT NULL,
85 duration REAL NOT NULL,
86 confidence REAL,
87 stats TEXT,
88 detection_params TEXT,
89 confirmed_segment_id INTEGER,
90 human_label TEXT,
91 notes TEXT,
92 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
93 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
94 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
95)
96 ON DELETE SET NULL
97 """)
98 cursor.execute("""
99 CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
100 ON candidate_segments(video_id, detector)
101 """)
102 cursor.execute("PRAGMA user_version = 1")
103 # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
104 user_version = 2

```



```

100
101 conn.commit()
102
103 def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
104 try:
105 with self._get_connection() as conn:
106 conn.cursor().execute(
107 "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
108 (video_id, filepath, status, json.dumps(validation_results))
109)
110 conn.commit()
111 return True
112 except Exception as e:
113 logger.error(f"Failed to add video: {e}")
114 return False
115
116 def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
117 try:
118 with self._get_connection() as conn:
119 cursor = conn.cursor()
120 cursor.execute(
121 "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
122 (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
123)
124 conn.commit()
125 return cursor.lastrowid
126 except Exception as e:
127 logger.error(f"Failed to add play segment: {e}")
128 return None
129
130 def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
131 try:
132 with self._get_connection() as conn:
133 conn.cursor().execute(
134 "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
135 (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
136)
137 conn.commit()
138 return True
139 except Exception as e:
140 logger.error(f"Failed to add event: {e}")
141 return False
142
143 def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
144 chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
145 stats: Optional[Dict[str, Any]] = None,
146 detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
147 """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
148 detector-specific dicts stored as JSON. Returns the new candidate id, or
149 None."""
150 try:
151 with self._get_connection() as conn:
152 cursor = conn.cursor()

```

```

152 cursor.execute(
153 """INSERT INTO candidate_segments
154 (video_id, detector, chunk_index, start_time, end_time, duration,
155 confidence, stats, detection_params)
156 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
157 (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
158 confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
159)
160 conn.commit()
161 return cursor.lastrowid
162 except Exception as e:
163 logger.error(f"Failed to add candidate segment: {e}")
164 return None
165
166 def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
167 """Record that a candidate detection was confirmed as a given play_segment."""
168 try:
169 with self._get_connection() as conn:
170 conn.cursor().execute(
171 "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
172 (segment_id, candidate_id)
173)
174 conn.commit()
175 return True
176 except Exception as e:
177 logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
178 return False
179
180 def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
181 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
182 """Fetch candidate detections for the annotation / fine-tuning workflow.
183 `stats` and `detection_params` are deserialized from JSON."""
184 query = "SELECT * FROM candidate_segments WHERE video_id = ?"
185 params: List[Any] = [video_id]
186
187 if detector:
188 query += " AND detector = ?"
189 params.append(detector)
190 if only_unlabeled:
191 query += " AND human_label IS NULL"
192
193 query += " ORDER BY start_time"
194
195 candidates = []
196 with self._get_connection() as conn:
197 rows = conn.cursor().execute(query, params).fetchall()
198 for row in rows:
199 d = dict(row)
200 d["stats"] = json.loads(d["stats"] or "{}")
201 d["detection_params"] = json.loads(d["detection_params"] or "{}")
202 candidates.append(d)
203 return candidates
204
205 def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
206 with self._get_connection() as conn:
207 row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
208 if row:
209 d = dict(row)
210 d["validation_results"] = json.loads(d["validation_results"] or "[]")
211 return d

```

```

212 return None
213
214 def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
215 """
216 Dynamically queries play segments based on filters:
217 - duration_min: float
218 - server: string
219 - receiver: string
220 - event_type: string (e.g. smash, foul)
221 """
222 query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
223 params = [video_id]
224
225 if filters.get("duration_min"):
226 query += " AND r.duration >= ?"
227 params.append(filters["duration_min"])
228
229 if filters.get("server"):
230 query += " AND r.server = ?"
231 params.append(filters["server"])
232
233 if filters.get("receiver"):
234 query += " AND r.receiver = ?"
235 params.append(filters["receiver"])
236
237 if filters.get("event_type"):
238 # Check if there is an event of a specific type inside this segment
239 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
240 params.append(filters["event_type"])
241
242 segments_list = []
243 with self._get_connection() as conn:
244 cursor = conn.cursor()
245 rows = cursor.execute(query, params).fetchall()
246
247 for row in rows:
248 r_dict = dict(row)
249 r_id = r_dict["id"]
250 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
251
252 # Fetch associated events
253 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
254 r_dict["events"] = []
255 for er in event_rows:
256 e_dict = dict(er)
257 e_dict["details"] = json.loads(e_dict["details"] or "{}")
258 r_dict["events"].append(e_dict)
259
260 segments_list.append(r_dict)
261
262 return segments_list
263
264 def clear_video_data(self, video_id: str):
265 with self._get_connection() as conn:
266 conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
267 conn.commit()
268

```

**29. assistant (2026-06-06T15:27:46.126Z)**

All three confirmed:

- classifier.py: `LogisticRegression` with

```
`fit/predict_proba/predict/to_dict/from_dict/save/load`, `class_weight='balanced'` default,
epochs=1000 default. ? The map says "feature ORDER at predict must match fit (names are metadata
only)" ? confirmed: `_standardize` uses positional mean/std, feature_names stored but unused in
math. ?
- annotations.py: `load_annotations` (RAISES on bad rows/start>=end), `parse_timestamp`,
`write_scaffold`. ?
- database.py: `Database`, 4 tables (videos/play_segments/events/candidate_segments), `PRAGMA
user_version` migration ladder (=1), all listed methods present, `PRAGMA foreign_keys=ON` per
conn, `add_video` INSERT OR REPLACE, events column is `type` ?, duration stored.
```

Note: map says write helpers "swallow exceptions (return False/None)" ? confirmed. `\_init\_db` is the migration ladder ? map's ramp path cites `Database.\_init\_db` which is correct.

Now annotations providers, sports, providers, validators, stitcher, utils, tools, and the hybrid segmenters.

### 30. user (2026-06-06T15:27:47.036Z)

```
1 """Annotation provider registry ? pluggable sources of ground-truth rally labels.
2
3 An `AnnotationProvider` turns a video reference into a list of `(start, end)` rally
4 intervals ? the same shape that `backend.eval.annotations.load_annotations`
5 produces and `backend.eval.metrics` consumes. That symmetry is the whole point:
6 golden-set ground truth can come from a CSV file, a human annotation vendor, or an
7 automated oracle behind one interface, so the regression flow
8 (`backend.eval.regression`) never needs to know where labels came from.
9
10 Mirrors the existing registry pattern in `backend/providers/base.py` and
11 `backend/indexer/base.py`. See `docs/GOLDEN_SET_VENDORING.md` (and ADR-006) for the
12 design rationale.
13 """
14 from abc import ABC, abstractmethod
15 from typing import Dict, List, Optional, Tuple, Type
16
17 # An (start, end) interval in seconds ? identical to backend.eval.metrics.Interval.
18 Interval = Tuple[float, float]
19
20 # Job lifecycle statuses returned by poll().
21 QUEUED = "queued"
22 RUNNING = "running"
23 DONE = "done"
24 FAILED = "failed"
25 STATUSES = (QUEUED, RUNNING, DONE, FAILED)
26
27
28 class AnnotationProvider(ABC):
29 """Abstract source of ground-truth rally intervals for a video.
30
31 The three-method contract (submit ? poll ? fetch) accommodates both
32 synchronous providers (a file already on disk) and asynchronous ones (a human
33 vendor that takes days). Synchronous providers can use the `annotate()`
34 convenience, which runs the full cycle in one call.
35 """
36
37 @property
38 @abstractmethod
39 def name(self) -> str:
40 """Unique registry key for this provider (e.g. 'file', 'vendor', 'auto')."""
41
42 @abstractmethod
43 def submit(self, video_ref: str, guideline: Optional[str] = None,
44 sport: str = "badminton") -> str:
45 """Submit a video for annotation; return an opaque job_id.
46
47 `video_ref` is provider-defined (a file path, a video_id/MD5, a URL).
```

```

48 `guideline` is the annotation spec (what counts as a rally start/end).
49 """
50
51 @abstractmethod
52 def poll(self, job_id: str) -> str:
53 """Return one of STATUSES for the given job."""
54
55 @abstractmethod
56 def fetch(self, job_id: str) -> List[Interval]:
57 """Return the completed annotations as sorted `(start, end)` intervals.
58
59 Should raise if the job is not DONE / the labels are unavailable, rather
60 than returning an empty list (which would silently skew downstream scores).
61 """
62
63 def annotate(self, video_ref: str, guideline: Optional[str] = None,
64 sport: str = "badminton") -> List[Interval]:
65 """Convenience: submit + fetch for a provider that completes synchronously.
66
67 Async providers (e.g. a human vendor with a multi-day SLA) should be driven
68 via submit/poll/fetch explicitly instead ? calling this on one will raise
69 once the job isn't immediately DONE.
70 """
71 job_id = self.submit(video_ref, guideline=guideline, sport=sport)
72 status = self.poll(job_id)
73 if status != DONE:
74 raise RuntimeError(
75 f"{self.name}.annotate(): job '{job_id}' is '{status}', not DONE. "
76 f"Use submit/poll/fetch for asynchronous providers."
77)
78 return self.fetch(job_id)
79
80
81 class AnnotationProviderRegistry:
82 """Registry of annotation providers, keyed by each provider's `name`."""
83
84 def __init__(self):
85 self._providers: Dict[str, Type[AnnotationProvider]] = {}
86
87 def register(self, provider_cls: Type[AnnotationProvider]) ->
88 Type[AnnotationProvider]:
89 """Register a provider class under its `name`. Returns the class (decorator-
90 friendly)."""
91 try:
92 key = provider_cls().name
93 except Exception:
94 # Providers needing constructor args fall back to the class name.
95 key = getattr(provider_cls, "__name__", "unknown")
96 self._providers[key] = provider_cls
97 return provider_cls
98
99 def get(self, name: str, **kwargs) -> AnnotationProvider:
100 """Instantiate the named provider, passing through any constructor kwargs."""
101 cls = self._providers.get(name)
102 if cls is None:
103 raise KeyError(
104 f"Unknown annotation provider '{name}'. Available:
105 {self.list_available()}"
106)
107 return cls(**kwargs)
108
109 def list_available(self) -> List[str]:
110 return list(self._providers.keys())

```

```
110 annotation_registry = AnnotationProviderRegistry()
111
```

### 31. user (2026-06-06T15:27:50.995Z)

```
1 """Pluggable sources of ground-truth rally annotations.
2
3 `AnnotationProvider`s yield `(start, end)` intervals ? the same shape the eval
4 harness scores ? so golden-set labels can come from a file, a human vendor, or an
5 automated oracle behind one interface. See `docs/GOLDEN_SET_VENDORING.md`.
6 """
7 from .base import annotation_registry, AnnotationProvider
8 from . import file_provider # noqa: F401 (side-effect: registers FileProvider)
9 from . import automated_provider # noqa: F401 (side-effect: registers
AutomatedProvider)
10
11 __all__ = ["annotation_registry", "AnnotationProvider"]
12
```

### 32. user (2026-06-06T15:27:51.179Z)

```
1 """AutomatedProvider ? the fast-tier "oracle" annotation source (GS-4, interface stub).
2
3 The fast/CI tier of the golden-set flow needs an *automated* labeler that turns a
4 video into rally `(start, end)` intervals quickly and cheaply, as a smoke alarm
5 between authoritative human-vendor runs (the slow tier).
6
7 This slice ships the **interface only**, deliberately model-agnostic: an
8 `AutomatedProvider` that delegates labeling to an injected `labeler` callable, plus
9 a deterministic fake for tests/CI. No concrete model is wired yet ? that's a
10 tracked backlog item (docs/BACKLOG.md), to be chosen alongside the production LLM
11 step so the lineages differ (see the oracle rule below).
12
13 ?? **Oracle rule.** The labeler MUST be a *different model lineage* than whatever
14 production uses. If production becomes the offline ActionFormer segmenter and the
15 oracle is also ActionFormer, they share blind spots and the regression test is
16 blind ? green while both are wrong. `lineage` records the oracle's model family so
17 callers can guard against it matching production (`warn_if_same_lineage`).
18 """
19 import logging
20 from typing import Callable, Dict, List, Optional
21
22 from backend.annotations.base import (
23 AnnotationProvider,
24 annotation_registry,
25 Interval,
26 DONE,
27 FAILED,
28)
29
30 logger = logging.getLogger(__name__)
31
32 # A labeler turns (video_ref, guideline, sport) into rally intervals.
33 Labeler = Callable[[str, Optional[str], str], List[Interval]]
34
35
36 def _not_configured(video_ref: str, guideline: Optional[str], sport: str) ->
List[Interval]:
37 """Default labeler: refuses to run until a real model is injected.
38
39 Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
40 while making accidental use of an unconfigured oracle fail loudly rather than
41 silently returning no rallies.
42 """
43 raise NotImplementedError(
```

```

44 "AutomatedProvider has no labeler configured. Inject one via "
45 "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>')."
46 "Choosing the concrete oracle model is a tracked item ? see docs/BACKLOG.md
(GS-4).\"
47)
48
49
50 def warn_if_same_lineage(oracle_lineage: Optional[str], production_lineage:
Optional[str]) -> bool:
51 """Warn (and return True) if the oracle shares production's model lineage.
52
53 Encodes the oracle rule in code, not just docs: a same-lineage oracle can't be
54 trusted to catch production's failures. Callers wiring CI should pass both.
55 """
56 if oracle_lineage and production_lineage and oracle_lineage == production_lineage:
57 logger.warning(
58 "Oracle lineage '%s' matches production's ? the regression test may be "
59 "blind to shared failure modes. Use a different model lineage.",
60 oracle_lineage,
61)
62 return True
63 return False
64
65
66 class AutomatedProvider(AnnotationProvider):
67 def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
68 """
69 Args:
70 labeler: callable producing rally intervals for a video. Defaults to a
71 stub that raises until configured.
72 lineage: the oracle's model family (e.g. 'gemini', 'qwen-vl') ? recorded
73 so callers can enforce the oracle rule via warn_if_same_lineage.
74 """
75 self._labeler = labeler or _not_configured
76 self.lineage = lineage
77 # In-process synchronous job store: job_id -> labeled intervals.
78 self._jobs: Dict[str, List[Interval]] = {}
79
80 @property
81 def name(self) -> str:
82 return "auto"
83
84 def submit(self, video_ref: str, guideline: Optional[str] = None,
85 sport: str = "badminton") -> str:
86 # Synchronous stub: run the labeler now and stash the result. A real async
87 # oracle (e.g. a queued API job) would kick off here and resolve in poll().
88 intervals = self._labeler(video_ref, guideline, sport)
89 intervals = sorted((float(s), float(e)) for s, e in intervals)
90 job_id = video_ref
91 self._jobs[job_id] = intervals
92 return job_id
93
94 def poll(self, job_id: str) -> str:
95 return DONE if job_id in self._jobs else FAILED
96
97 def fetch(self, job_id: str) -> List[Interval]:
98 if job_id not in self._jobs:
99 raise KeyError(f"AutomatedProvider: unknown job '{job_id}' (submit it
first)")
100 return self._jobs[job_id]
101
102
103 annotation_registry.register(AutomatedProvider)
104

```

### 33. user (2026-06-06T15:27:51.349Z)

```
1 """FileProvider ? ground-truth rally labels from pre-made CSV files.
2
3 This formalizes today's manual / ShuttleSet annotation flow behind the
4 `AnnotationProvider` interface: the "annotation job" is simply a CSV that already
5 exists on disk (the same `start,end` format `run_eval.py --scaffold` writes). The
6 job_id IS the resolved CSV path, so submit/poll/fetch all operate on that path.
7
8 It's the synchronous, zero-cost provider used as the default golden-set source and
9 as the stand-in in tests/CI before a human-vendor or automated provider is wired in.
10 """
11 import os
12 from typing import List, Optional
13
14 from backend.annotations.base import (
15 AnnotationProvider,
16 annotation_registry,
17 Interval,
18 DONE,
19 FAILED,
20)
21 from backend.eval.annotations import load_annotations
22
23
24 class FileProvider(AnnotationProvider):
25 def __init__(self, csv_dir: Optional[str] = None, ext: str = ".csv"):
26 """
27 Args:
28 csv_dir: if set, a bare `video_ref` (not itself a path) is resolved to
29 `
```



```

64
65
66 annotation_registry.register(FileProvider)
67

```

#### 34. assistant (2026-06-06T15:27:58.838Z)

Annotations subsystem confirmed:

```

- base.py: `AnnotationProvider` ABC (`submit/poll/fetch/annotate`), `Interval`, `STATUSES`,
`annotation_registry`. ?
- __init__.py: `noqa F401` imports load-bearing (file_provider, automated_provider). ?
- automated_provider.py: `AutomatedProvider` ('auto'), injectable `labeler`,
`warn_if_same_lineage` oracle rule, `_not_configured` (raises NotImplementedError). ? Registered
at module bottom via `annotation_registry.register(AutomatedProvider)` ?.
- file_provider.py: `FileProvider` ('file'), job_id is CSV path, -> `load_annotations`. ?

```

Note: the map's "add a tier" registration says "subclass, register at module bottom + import in `\_\_init\_\_.py`". The registry's `register()` takes the **class** (not name), keying off `provider\_cls().name` ? this matches base.py. ? Also the map's \$annotation contract says `fetch -> List[Interval]` sorted, "contract says RAISE (not [])" ? confirmed in the docstring. ?

Now sports, providers, and the hybrid segmenters.

#### 35. user (2026-06-06T15:27:59.697Z)

```

1 from typing import Dict, Type
2 from backend.sports.base import SportProfile
3 from backend.sports.badminton import BadmintonSportProfile
4
5 class SportRegistry:
6 def __init__(self):
7 self._registry: Dict[str, Type[SportProfile]] = {}
8 # Automatically register Badminton
9 self.register(BadmintonSportProfile)
10
11 def register(self, profile_cls: Type[SportProfile]):
12 inst = profile_cls()
13 self._registry[inst.name] = profile_cls
14
15 def get(self, name: str) -> SportProfile:
16 if name not in self._registry:
17 raise KeyError(f"Sport '{name}' is not registered in the SportRegistry.
Available: {list(self._registry.keys())}")
18 return self._registry[name]()
19
20 def list_available(self) -> Dict[str, str]:
21 return {name: cls.__doc__ or f"{name.capitalize()} sport profile" for name, cls
in self._registry.items()}
22
23 sport_registry = SportRegistry()
24

```

#### 36. user (2026-06-06T15:28:00.614Z)

```

1 from abc import ABC, abstractmethod
2
3 class SportProfile(ABC):
4 @property
5 @abstractmethod
6 def name(self) -> str:
7 """The identifier of the sport (e.g., 'badminton')."""
8 pass
9
10 @abstractmethod

```

```

11 def get_prompt(self, chunk_duration: float = 0.0) -> str:
12 """Full system prompt for the AI model, including output schema."""
13 pass
14
15 @abstractmethod
16 def get_relevance_config(self) -> dict:
17 """Sport-specific visual validation params (court colors, thresholds, etc.)."""
18 pass
19

```

### 37. user (2026-06-06T15:28:01.226Z)

```

1 from typing import Dict, Type
2 from backend.providers.base import AIVideoProvider
3 from backend.providers.gemini import GeminiProvider
4
5 class ProviderRegistry:
6 def __init__(self):
7 self._registry: Dict[str, Type[AIVideoProvider]] = {}
8 # Register Gemini provider
9 self.register("gemini", GeminiProvider)
10
11 def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12 self._registry[name] = provider_cls
13
14 def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15 if name not in self._registry:
16 raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17 return self._registry[name](api_key=api_key, model_name=model_name)
18
19 def list_available(self) -> Dict[str, str]:
20 return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22 provider_registry = ProviderRegistry()
23

```

### 38. user (2026-06-06T15:28:01.479Z)

```

1 from abc import ABC, abstractmethod
2 from typing import Tuple
3
4 class AIVideoProvider(ABC):
5 def __init__(self, api_key: str, model_name: str):
6 self.api_key = api_key
7 self.model_name = model_name
8
9 @abstractmethod
10 def analyze_video(self, video_path: str, prompt: str,
11 chunk_duration: float = 0.0,
12 label: str = "") -> Tuple[list, dict]:
13 """Sends a video file to the AI model along with a prompt, and returns a tuple
of
14 (segments_list, metrics_dict).
15
16 Args:
17 video_path: Path to the video file to analyze.
18 prompt: Prompt instructions.
19 chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
20 label: Diagnostic label for logging.
21
22 Returns:
23 Tuple containing:
24 - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":

```

```

...]])
25 - A dictionary of performance metrics (e.g. {"upload_time": ...,
"process_time": ..., "gen_time": ...})
26 """
27 pass
28

```

### 39. assistant (2026-06-06T15:28:08.269Z)

Confirmed:

- sports/\_\_init\_\_.py: `sport\_registry`, `SportRegistry`, auto-registers Badminton. ? Map says "add a sport: subclass SportProfile, sport\_registry.register(MyProfile)" ?. "get()" returns a fresh instance each call" ? confirmed (`return self.\_registry[name]()`) . ?
- sports/base.py: `SportProfile` ABC with `name`, `get\_prompt`, `get\_relevance\_config`. ?
- providers/\_\_init\_\_.py: `provider\_registry`, `ProviderRegistry`, auto-registers 'gemini'. ? Map says "add a provider: register('name', MyCls)" ? confirmed, key passed explicitly ? (note: `register(name, cls)` and also `get(name, api\_key, model\_name)`).
- providers/base.py: `AIVideoProvider` ABC, `analyze\_video(path, prompt, chunk\_duration=0, label='')` -> (list, dict)`. ?

Note: the map's \$provider contract says metrics dict `{upload\_time,process\_time,gen\_time}` ? confirmed in docstring. ?

Now let me verify the hybrid segmenters, gemini, motion, validators, stitcher, utils, tools, badminton, and the run\_\* entrypoints.

### 40. user (2026-06-06T15:28:09.192Z)

```

1 """WASB-based hybrid play segmenter.
2
3 Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4 (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5 (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6 badminton weights and runs today, so this is the live shuttle-motion substrate.
7 Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9 For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10 windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11 are what get tuned against golden rally labels.
12 """
13 import os
14 import time
15 import logging
16 import concurrent.futures
17 from typing import List, Dict, Any
18
19 import cv2
20
21 from backend.indexer.base import BasePlaySegmenter, segmenter_registry
22 from backend.utils.video import get_video_duration, extract_video_chunk
23 from backend.sports import sport_registry
24 from backend.providers import provider_registry
25 from backend.detectors.wasb_runner import WasbRunner, WasbConfig
26
27 logger = logging.getLogger(__name__)
28
29
30 def _fmt_ts(seconds: float) -> str:
31 seconds = max(0, int(seconds))
32 h, rem = divmod(seconds, 3600)
33 m, s = divmod(rem, 60)
34 return f"{h:02d}:{m:02d}:{s:02d}"
35
36
37 class WasbHybridSegmenter(BasePlaySegmenter):

```

```

38 @property
39 def name(self) -> str:
40 return "wasb_hybrid"
41
42 @property
43 def description(self) -> str:
44 return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
45 "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
46
47 @staticmethod
48 def _probe_fps_width(video_path: str) -> tuple:
49 cap = cv2.VideoCapture(video_path)
50 fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
51 width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
52 cap.release()
53 return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
54
55 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
56 max_frames: int = None) -> List[Dict[str, Any]]:
57 sport_name = self.config.get("sport", "badminton")
58 try:
59 sport_profile = sport_registry.get(sport_name)
60 except KeyError as e:
61 logger.error(str(e))
62 return []
63
64 idx_cfg = self.config.get("indexing", {})
65 provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
66 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
67
68 api_key_env_var = f"{provider_name.upper()}_API_KEY"
69 api_key = os.environ.get(api_key_env_var)
70 if not api_key:
71 logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
72 return []
73 try:
74 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
75 except KeyError as e:
76 logger.error(str(e))
77 return []
78
79 video_duration = get_video_duration(video_path)
80 if video_duration <= 0:
81 logger.error("Invalid video duration.")
82 return []
83 fps, frame_width = self._probe_fps_width(video_path)
84
85 wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
86 runner = WasbRunner(wasb_cfg)
87
88 w = idx_cfg.get("wasb", {})
89 velocity_thresh = w.get("velocity_threshold", 0.02)
90 merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
91 min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
92 handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
93 ai_max_workers = idx_cfg.get("ai_max_workers", 5)
94 log_candidates = idx_cfg.get("log_yolo_candidates", True)
95 store_candidates = idx_cfg.get("store_yolo_candidates", True)

```

```

96
97 detection_params = {
98 "detector": "wasb_hybrid",
99 "velocity_thresh": velocity_thresh,
100 "merge_gap": merge_gap,
101 "min_action_window_duration": min_window_duration,
102 "weights_path": wasb_cfg.weights_path,
103 }
104
105 ok, msg = runner.healthcheck()
106 if not ok:
107 logger.error("WASB WSL environment not ready: %s", msg)
108 return []
109 logger.info("WASB WSL env OK: %s", msg)
110
111 # --- Phase 2: shuttle trajectory -> candidate rally windows ---
112 t_start = time.time()
113 out_dir = os.path.join("output", "wasb")
114 csv_path = runner.run_predict(video_path, out_dir)
115 if not csv_path:
116 logger.error("WASB produced no trajectory; aborting.")
117 return []
118
119 points = runner.parse_trajectory_csv(csv_path)
120 logger.info("Parsed %d trajectory points (%d visible).",
121 len(points), sum(1 for p in points if p.visible))
122
123 all_candidate_windows = runner.trajectory_to_action_windows(
124 points, fps=fps, frame_width=frame_width, chunk_start=0.0,
125 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
126 min_window_duration=min_window_duration, chunk_index=0,
127)
128 logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
129 time.time() - t_start, len(all_candidate_windows))
130
131 if log_candidates:
132 for cw in all_candidate_windows:
133 logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
134 f"({cw['end'] - cw['start']:.1f}s) | "
135 f"peak_v={cw['peak_velocity']:.3f} "
136 f"mean_v={cw['mean_velocity']:.3f}")
137
138 candidate_ids = [None] * len(all_candidate_windows)
139 if store_candidates:
140 for i, cw in enumerate(all_candidate_windows):
141 stats = {
142 "peak_velocity": cw["peak_velocity"], "mean_velocity":
143 cw["mean_velocity"],
144 "active_frame_count": cw["active_frame_count"], "track_id_count":
145 cw["track_id_count"],
146 }
147 candidate_ids[i] = self.db.add_candidate_segment(
148 video_id, detector="wasb_hybrid", start_time=cw["start"],
149 end_time=cw["end"],
150 chunk_index=cw["chunk_index"], confidence=min(1.0,
151 cw["peak_velocity"]),
152 stats=stats, detection_params=detection_params,
153)
154
155 if not all_candidate_windows:
156 return []
157
158 # --- Phase 3: AI provider handoff (shared pattern) ---
159 handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")

```

```

155 os.makedirs(handoff_dir, exist_ok=True)
156 logger.info("Phase 3: %s validation on %d candidate windows...",
157 provider_name, len(all_candidate_windows))
158
159 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
160 w_start = max(0, window["start"] - handoff_padding)
161 w_end = min(video_duration, window["end"] + handoff_padding)
162 w_dur = w_end - w_start
163 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
164 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
165 return []
166 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
167 segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
168 label=f"Handoff {wi+1}")
169 valid = []
170 for seg in segments:
171 try:
172 seg["start_time"] = float(seg["start_time"]) + w_start
173 seg["end_time"] = float(seg["end_time"]) + w_start
174 seg["_candidate_id"] = candidate_ids[wi]
175 valid.append(seg)
176 except (ValueError, TypeError, KeyError) as e:
177 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
178 try:
179 os.remove(w_path)
180 except OSError:
181 pass
182 return valid
183
184 ai_segments: List[dict] = []
185 with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
186 futures = [executor.submit(process_candidate_window, wi, cw)
187 for wi, cw in enumerate(all_candidate_windows)]
188 for future in futures:
189 ai_segments.extend(future.result())
190 try:
191 os.rmdir(handoff_dir)
192 except OSError:
193 pass
194 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
195
196 # --- Phase 4: DB population (shared pattern) ---
197 segments_data = []
198 for segment in ai_segments:
199 try:
200 start = float(segment["start_time"])
201 end = float(segment["end_time"])
202 dur = end - start
203 shot_count = segment.get("shuttle_exchange_count")
204 try:
205 shot_count = int(shot_count)
206 except (ValueError, TypeError):
207 shot_count = max(3, int(dur / 0.8))
208 metadata = {"shot_count": shot_count,
209 "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
210 "detector": f"wasb-hybrid-{model_name}"}
211 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
212 if segment_id:
213 candidate_id = segment.get("_candidate_id")

```

```

214 if candidate_id is not None:
215 self.db.link_candidate_to_segment(candidate_id, segment_id)
216 segments_data.append({"id": segment_id, "start_time": start,
"end_time": end,
217 "duration": dur, "metadata": metadata})
218 except Exception as e:
219 logger.warning("Skipping invalid segment struct: %s", e)
220
221 return segments_data
222
223
224 segmenter_registry.register(WasbHybridSegmenter)
225

```

#### 41. user (2026-06-06T15:28:10.187Z)

```

1 """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2 (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4 Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
5 service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
6 detection now uses torchvision's permissively-licensed (BSD) detectors and the
7 association/tracking that YOLO.track() used to bundle in is provided by
8 supervision's MIT-licensed ByteTrack.
9
10 The registry key stays "yolo_hybrid" for back-compat with existing configs and the
11 DB `detector` tags, even though no YOLO is involved anymore.
12 """
13 import os
14 import time
15 import logging
16 import cv2
17 import torch
18 import concurrent.futures
19 from typing import List, Dict, Any, Tuple
20
21 from backend.indexer.base import BasePlaySegmenter, segmenter_registry
22 from backend.utils.video import get_video_duration, extract_video_chunk
23 from backend.utils.geometry import get_velocity, get_velocity_normalised
24 from backend.sports import sport_registry
25 from backend.providers import provider_registry
26
27 logger = logging.getLogger(__name__)
28
29 # torchvision detection models are trained on COCO where the "person" category is
30 # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
31 # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
32 _PERSON_CLASS_ID = 1
33
34 # Permissively-licensed torchvision detectors selectable via config
35 # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
36 _DETECTOR_FACTORIES = {
37 "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
38 "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
39 "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
40 }
41 _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
42
43
44 def _fmt_ts(seconds: float) -> str:
45 """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
46 seconds = max(0, int(seconds))
47 h, rem = divmod(seconds, 3600)
48 m, s = divmod(rem, 60)
49 return f"{h:02d}:{m:02d}:{s:02d}"

```

```

50
51
52 class HybridYoloSegmenter(BasePlaySegmenter):
53 def __init__(self, db, config):
54 super().__init__(db, config)
55 self.model = None
56 self.device = None
57
58 @property
59 def name(self) -> str:
60 return "yolo_hybrid"
61
62 @property
63 def description(self) -> str:
64 return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
65 "candidate filtering, then an AI provider for high-precision semantic
66 boundaries.")
67
68 def _lazy_load_model(self):
69 if self.model is not None:
70 return
71
72 # Imported lazily so the module imports without torchvision present and so
73 # the test suite (which pre-sets self.model) never pulls in model weights.
74 from torchvision.models import detection as tv_detection
75
76 idx_cfg = self.config.get("indexing", {})
77 use_gpu = idx_cfg.get("use_gpu", True)
78 self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
79 if use_gpu and not torch.cuda.is_available():
80 logger.warning("use_gpu is True but CUDA is not available. Falling back to
81 CPU.")
82
83 model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
84 if model_name not in _DETECTOR_FACTORIES:
85 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
86 {_DEFAULT_DETECTOR}.")
87 model_name = _DEFAULT_DETECTOR
88
89 builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
90 logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
91 # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
92 self.model = builder(weights="DEFAULT")
93 self.model.eval()
94 self.model.to(self.device)
95
96 def _make_tracker(self, fps: float):
97 """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
98
99 Imported lazily so tests can mock this method without supervision installed.
100 """
101 import supervision as sv
102 frame_rate = max(1, int(round(fps)))
103 return sv.ByteTrack(frame_rate=frame_rate)
104
105 def _detect_and_track(self, frame, tracker, score_thresh: float) -> List[Tuple[int,
106 Tuple[float, float, float, float]]]:
107 """Detect persons in a frame and assign persistent track IDs.
108
109 Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
110 This replaces the old ultralytics ``model.track(...)``, call, which bundled
111 detection AND association: torchvision does the detection, supervision's
112 ByteTrack does the association.
113 """
114 import numpy as np

```



```

111 import supervision as sv
112
113 # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
114 rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
115 tensor = torch.from_numpy(rgb).permute(2, 0,
116 1).float().div(255.0).to(self.device)
117 with torch.no_grad():
118 outputs = self.model([tensor])
119
120 out = outputs[0]
121 boxes = out["boxes"].detach().cpu().numpy()
122 labels = out["labels"].detach().cpu().numpy()
123 scores = out["scores"].detach().cpu().numpy()
124
125 # Keep only confident person detections.
126 mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
127 if not mask.any():
128 return []
129
130 # ByteTrack requires confidence to be populated (it filters on it internally).
131 dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
132 class_id=labels[mask])
133 tracked = tracker.update_with_detections(dets)
134
135 result: List[Tuple[int, Tuple[float, float, float, float]]] = []
136 for i in range(len(tracked)):
137 tid = tracked.tracker_id[i]
138 if tid is None:
139 continue
140 x1, y1, x2, y2 = tracked.xyxy[i]
141 result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
142 return result
143
144 def _extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
145 velocity_thresh: float, merge_gap: float,
146 frame_skip: int = 3,
147 tracker_config: str = "bytetrack.yaml",
148 chunk_index: int = None) -> List[Dict[str,
149 Any]]:
150 """
151 Runs person detection + tracking on a video chunk and extracts high-motion
152 action windows. Returns a list of dicts in the full video's timeframe, each
153 with:
154 start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
155 track_id_count, chunk_index.
156
157 Args:
158 frame_skip: Process every Nth frame (1 = every frame). Higher values
159 dramatically reduce inference cost with minimal quality loss.
160 tracker_config: Retained for back-compat (was the YOLO tracker yaml); the
161 tracker is now supervision ByteTrack and this is unused.
162 chunk_index: Index of the source chunk (recorded on each window for
163 traceability).
164 """
165 cap = cv2.VideoCapture(chunk_path)
166 if not cap.isOpened():
167 logger.error(f"Could not open {chunk_path}")
168 return []
169
170 fps = cap.get(cv2.CAP_PROP_FPS)
171 if fps <= 0:
172 fps = 30.0
173
174 frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
175 if frame_width <= 0:

```

```

171 frame_width = 1280.0 # Sensible fallback
172
173 # Effective sampling rate after frame-skipping
174 effective_fps = fps / max(frame_skip, 1)
175
176 # A fresh tracker per chunk ? track IDs only need to be consistent within a
177 # chunk (windows are computed per chunk, then merged across chunks by time).
178 tracker = self._make_tracker(effective_fps)
179 score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
180
181 # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
182 # so windows can be enriched with motion stats for logging + fine-tuning.
183 active_frames = []
184 frame_idx = 0
185 track_history = {}
186
187 while True:
188 ret, frame = cap.read()
189 if not ret:
190 break
191
192 # Skip frames for performance ? at 30fps, every 3rd frame still
193 # gives ~10 samples/second, plenty for human-scale velocity.
194 if frame_idx % max(frame_skip, 1) != 0:
195 frame_idx += 1
196 continue
197
198 # Detect persons + assign persistent track IDs.
199 detections = self._detect_and_track(frame, tracker, score_thresh)
200
201 frame_peak_velocity = 0.0
202 frame_active_ids = set()
203 current_ids = set()
204
205 for track_id, bbox in detections:
206 current_ids.add(track_id)
207
208 if track_id in track_history:
209 prev_bbox = track_history[track_id]
210 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
211
212 if velocity > velocity_thresh:
213 frame_peak_velocity = max(frame_peak_velocity, velocity)
214 frame_active_ids.add(track_id)
215
216 track_history[track_id] = bbox
217
218 # Prune stale IDs that are no longer visible ? prevents false
219 # velocity spikes when the tracker reassigns an old ID.
220 track_history = {k: v for k, v in track_history.items() if k in current_ids}
221
222 if frame_active_ids:
223 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
224
225 frame_idx += 1
226
227 cap.release()
228
229 # Group active frames into continuous windows
230 if not active_frames:
231 return []
232
233 max_gap_frames = int(merge_gap * fps)

```

```

234 def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
235 velocities = [v for _, v, _ in group]
236 track_ids = set().union(*[ids for _, _, ids in group])
237 return {
238 "start": group[0][0] / fps + chunk_start,
239 "end": group[-1][0] / fps + chunk_start,
240 "active_frame_count": len(group),
241 "peak_velocity": max(velocities),
242 "mean_velocity": sum(velocities) / len(velocities),
243 "track_id_count": len(track_ids),
244 "chunk_index": chunk_index,
245 }
246
247 windows = []
248 group = [active_frames[0]]
249 for af in active_frames[1:]:
250 if af[0] - group[-1][0] <= max_gap_frames:
251 group.append(af)
252 else:
253 windows.append(_finalize(group))
254 group = [af]
255 windows.append(_finalize(group))
256
257 # Filter out extremely short action windows
258 min_window_duration = self.config.get("indexing",
259 {}).get("min_action_window_duration", 2.0)
259 windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
260
261 return windows
262
263 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
264 None, max_frames: int = None) -> List[Dict[str, Any]]:
265 self._lazy_load_model()
266
267 # Get sport profile config
268 sport_name = self.config.get("sport", "badminton")
269 try:
270 sport_profile = sport_registry.get(sport_name)
271 except KeyError as e:
272 logger.error(str(e))
273 return []
274
275 # Get AI provider and model config
276 idx_cfg = self.config.get("indexing", {})
277 provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
278 "gemini"))
279 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
280 "gemini-2.5-flash"))
281
282 # Get appropriate API key based on the provider
283 api_key_env_var = f"{provider_name.upper()}_API_KEY"
284 api_key = os.environ.get(api_key_env_var)
285 if not api_key:
286 logger.error(
287 f"{api_key_env_var} environment variable is not set! "
288 f"To run the {provider_name} segmenter, set your API key. "
289 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
290)
291 return []
292
293 try:
294 provider = provider_registry.get(provider_name, api_key=api_key,
295 model_name=model_name)
296 except KeyError as e:
297 logger.error(str(e))

```

```

294 return []
295
296 logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
297
298 video_duration = get_video_duration(video_path)
299 if video_duration <= 0:
300 logger.error("Invalid video duration.")
301 return []
302
303 chunk_duration = idx_cfg.get("chunk_duration", 300)
304 chunk_overlap = idx_cfg.get("chunk_overlap", 30)
305 velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
306 merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
307 frame_skip = idx_cfg.get("yolo_frame_skip", 3)
308 tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
309 handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
310 ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
311 log_candidates = idx_cfg.get("log_yolo_candidates", True)
312 store_candidates = idx_cfg.get("store_yolo_candidates", True)
313
314 # Snapshot of the params that produced these detections ? stored alongside each
315 # candidate so a fine-tuning run can reproduce / correlate against them.
316 detection_params = {
317 "velocity_thresh": velocity_thresh,
318 "merge_gap": merge_gap,
319 "frame_skip": frame_skip,
320 "tracker": tracker_config,
321 "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
322 "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
323 "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
324 }
325
326 chunks_dir = os.path.join("output", "chunks_yolo")
327 os.makedirs(chunks_dir, exist_ok=True)
328
329 step = chunk_duration - chunk_overlap
330 chunk_starts = []
331 t = 0.0
332 while t < video_duration:
333 chunk_starts.append(t)
334 t += step
335
336 num_chunks = len(chunk_starts)
337 logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
338
339 all_candidate_windows = []
340
341 t_start = time.time()
342 prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
343
344 if num_chunks > 1 and prefetch_workers > 0:
345 # --- PIPELINED PROCESSING ---
346 # GPU inference must stay sequential (single GPU), but ffmpeg chunk
347 # extraction is pure I/O. We pre-extract upcoming chunks in background
348 # threads so extraction of chunk N+1 overlaps with inference on chunk N.
349 logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
350
351 extract_executor =

```

```

concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
352
353 def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
354 chunk_end = min(cs + chunk_duration, video_duration)
355 actual_dur = chunk_end - cs
356 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
357 success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
358 return (ci, cs, chunk_path, success)
359
360 # Submit all extractions upfront ? they'll queue and run as workers free up
361 extract_futures = [
362 extract_executor.submit(_extract_chunk, ci, cs)
363 for ci, cs in enumerate(chunk_starts)
364]
365
366 # Process results in order: wait for extraction, then run inference on GPU
367 for future in extract_futures:
368 ci, chunk_start, chunk_path, success = future.result()
369 chunk_end = min(chunk_start + chunk_duration, video_duration)
370 logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
371
372 if not success:
373 logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
374 continue
375
376 windows = self._extract_action_windows_from_chunk(
377 chunk_path, chunk_start, velocity_thresh, merge_gap,
378 frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
379)
380 logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
381 all_candidate_windows.extend(windows)
382
383 try:
384 os.remove(chunk_path)
385 except Exception:
386 pass
387
388 extract_executor.shutdown(wait=False)
389 else:
390 # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
391 for ci, chunk_start in enumerate(chunk_starts):
392 chunk_end = min(chunk_start + chunk_duration, video_duration)
393 actual_dur = chunk_end - chunk_start
394 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
395
396 logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
397 success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
398
399 if not success:
400 continue
401
402 windows = self._extract_action_windows_from_chunk(
403 chunk_path, chunk_start, velocity_thresh, merge_gap,
404 frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
405)
406 logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
407
408 all_candidate_windows.extend(windows)
409
410 try:
411 os.remove(chunk_path)
412 except Exception:

```

```

411 pass
412
413 # Deduplicate overlapping candidate windows across chunks (chunks overlap by
414 # chunk_overlap, so the same rally can surface in two adjacent chunks).
415 def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
416 fa, fb = a["active_frame_count"], b["active_frame_count"]
417 total = fa + fb or 1
418 return {
419 "start": a["start"],
420 "end": max(a["end"], b["end"]),
421 "active_frame_count": fa + fb,
422 "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
423 # Frame-weighted mean so longer sub-windows dominate proportionally.
424 "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
425 "track_id_count": max(a["track_id_count"], b["track_id_count"]),
426 "chunk_index": a["chunk_index"],
427 }
428
429 if all_candidate_windows:
430 all_candidate_windows.sort(key=lambda w: w["start"])
431 merged_windows = [all_candidate_windows[0]]
432 for current in all_candidate_windows[1:]:
433 last = merged_windows[-1]
434 if current["start"] <= last["end"]: # Overlap
435 merged_windows[-1] = _merge_stats(last, current)
436 else:
437 merged_windows.append(current)
438 all_candidate_windows = merged_windows
439
440 logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
441 logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
442
443 # Per-rally console log (final, full-video timestamps) for video cross-
verification.
444 if log_candidates:
445 for w in all_candidate_windows:
446 logger.info(
447 f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
448 f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
449 f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
450 f"tracks={w['track_id_count']}"
451)
452
453 # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
454 # so confirmed AI segments can be linked back in Phase 4.
455 candidate_ids = [None] * len(all_candidate_windows)
456 if store_candidates:
457 for i, w in enumerate(all_candidate_windows):
458 stats = {
459 "peak_velocity": w["peak_velocity"],
460 "mean_velocity": w["mean_velocity"],
461 "active_frame_count": w["active_frame_count"],
462 "track_id_count": w["track_id_count"],
463 }
464 # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
465 confidence = min(1.0, w["peak_velocity"])
466 candidate_ids[i] = self.db.add_candidate_segment(
467 video_id, detector="yolo_hybrid",
468 start_time=w["start"], end_time=w["end"],
469 chunk_index=w["chunk_index"], confidence=confidence,
470 stats=stats, detection_params=detection_params,

```

```

471)
472
473 try:
474 os.rmdir(chunks_dir)
475 except Exception:
476 pass
477
478 # Phase 3: AI Provider Handoff
479 gemini_segments = []
480 gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
481 os.makedirs(gemini_dir, exist_ok=True)
482
483 logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
484
485 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
486 w_start = max(0, window["start"] - handoff_padding)
487 w_end = min(video_duration, window["end"] + handoff_padding)
488 w_dur = w_end - w_start
489
490 w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
491 # Re-encode short handoff clips to avoid keyframe-boundary drift
492 success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
493 if not success:
494 return []
495
496 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
497 segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
498
499 # Coordinate mapping ? shift local timestamps back to full-video time
500 valid_segments = []
501 for segment in segments:
502 try:
503 segment["start_time"] = float(segment["start_time"]) + w_start
504 segment["end_time"] = float(segment["end_time"]) + w_start
505 # Stamp the source candidate so Phase 4 can link
506 segment["_candidate_id"] = candidate_ids[wi]
507 valid_segments.append(segment)
508 except (ValueError, TypeError, KeyError) as e:
509 logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")
510
511 try:
512 os.remove(w_path)
513 except OSError:
514 pass
515
516 return valid_segments
517
518 with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
519 futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
520 # Iterate in submission order to preserve temporal ordering
521 for future in futures:
522 gemini_segments.extend(future.result())
523
524 try:
525 os.rmdir(gemini_dir)
526 except OSError:
527 pass
528

```

```

529 logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
530
531 # Phase 4: DB Population
532 segments_data = []
533 for segment in gemini_segments:
534 try:
535 start = float(segment["start_time"])
536 end = float(segment["end_time"])
537 dur = end - start
538
539 shot_count = segment.get("shuttle_exchange_count")
540 if shot_count is None: shot_count = max(3, int(dur / 0.8))
541 else:
542 try: shot_count = int(shot_count)
543 except (ValueError, TypeError): shot_count = max(3, int(dur / 0.8))
544
545 metadata = {
546 "shot_count": shot_count,
547 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
548 "detector": f"yolo-hybrid-{model_name}"
549 }
550
551 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
552 if segment_id:
553 # Link back to the candidate that produced this segment. A candidate
554 # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
555 # the last one ? the link table records "this candidate was
confirmed".
556 candidate_id = segment.get("_candidate_id")
557 if candidate_id is not None:
558 self.db.link_candidate_to_segment(candidate_id, segment_id)
559 segments_data.append({
560 "id": segment_id,
561 "start_time": start,
562 "end_time": end,
563 "duration": dur,
564 "metadata": metadata
565 })
566 except Exception as e:
567 logger.warning(f"Skipping invalid segment struct: {e}")
568
569 return segments_data
570
571 # Register the segmenter
572 segmenter_registry.register(HybridYoloSegmenter)
573

```

## 42. assistant (2026-06-06T15:28:18.956Z)

Hybrid segmenters confirmed:

```

- wasb_hybrid: `WasbHybridSegmenter` (name 'wasb_hybrid'), `velocity_threshold(0.02)`, shared
keys `dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`, whole-
video single pass (chunk_index=0), reuses YOLO-named keys
`log_yolo_candidates/store_yolo_candidates`, hardcoded `output/wasb`, needs
`{PROVIDER}_API_KEY`, `_probe_fps_width` fallback (30.0, 1280.0). Registered at bottom. ?
- yolo_hybrid: `HybridYoloSegmenter` (name 'yolo_hybrid'), torchvision + ByteTrack,
`_PERSON_CLASS_ID=1` (ultralytics used 0), `_lazy_load_model`,
`_extract_action_windows_from_chunk`, `import torch` at top, velocity at effective_fps coupled
to frame_skip, fresh tracker per chunk. ? `detection_params` has
`frame_skip,tracker,detector_model,detector_score_threshold`. ?

```



One detail: the map says yolo velocity threshold config key ? map's \$detection\_params lists `frame\_skip` for yolo correctly. The actual yolo velocity config key is `yolo\_velocity\_threshold` (default 0.15), which the map doesn't explicitly contradict. Good.

Now the wasb-hybrid \$detection\_params: map says it includes `weights\_path` ? confirmed. The wasb detection\_params does NOT include `queue\_length` (that's tracknet only). Let me verify tracknet\_hybrid and the remaining files.

#### 43. user (2026-06-06T15:28:19.855Z)

```
1 """TrackNet-based hybrid play segmenter.
2
3 Same three-phase shape as HybridYoloSegmenter, but Phase 2 (candidate
4 generation) uses a TrackNetV4 *shuttle* trajectory instead of YOLO *person*
5 motion. TrackNet runs in WSL2 (see backend/detectors/tracknet_runner.py and
6 docs/DECISIONS.md ADR-001); the shuttle trajectory is converted into candidate
7 rally windows, which are then handed to the AI provider for semantic boundary
8 confirmation (Phase 3) and written to the DB (Phase 4).
9
10 Rationale: a tiny, fast, motion-blurred shuttle is exactly what general
11 detectors miss; a purpose-built trajectory model localises it far more
12 precisely, giving cleaner candidate rallies than person-velocity heuristics.
13 """
14
15 import os
16 import time
17 import logging
18 import concurrent.futures
19 from typing import List, Dict, Any
20
21 import cv2
22
23 from backend.indexer.base import BasePlaySegmenter, segmenter_registry
24 from backend.utils.video import get_video_duration, extract_video_chunk
25 from backend.sports import sport_registry
26 from backend.providers import provider_registry
27 from backend.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
28
29 logger = logging.getLogger(__name__)
30
31
32 def _fmt_ts(seconds: float) -> str:
33 seconds = max(0, int(seconds))
34 h, rem = divmod(seconds, 3600)
35 m, s = divmod(rem, 60)
36 return f"{h:02d}:{m:02d}:{s:02d}"
37
38
39 class TrackNetHybridSegmenter(BasePlaySegmenter):
40 @property
41 def name(self) -> str:
42 return "tracknet_hybrid"
43
44 @property
45 def description(self) -> str:
46 return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
47 "candidate rallies + AI provider for high-precision semantic
boundaries.")
48
49 @staticmethod
50 def _probe_fps_width(video_path: str) -> tuple:
51 """Return (fps, frame_width) for a video, with sensible fallbacks."""
52 cap = cv2.VideoCapture(video_path)
53 fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
```

```

54 width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
55 cap.release()
56 return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
57
58 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
59 max_frames: int = None) -> List[Dict[str, Any]]:
60 # --- Sport profile + AI provider wiring (identical to the other segmenters) ---
61 sport_name = self.config.get("sport", "badminton")
62 try:
63 sport_profile = sport_registry.get(sport_name)
64 except KeyError as e:
65 logger.error(str(e))
66 return []
67
68 idx_cfg = self.config.get("indexing", {})
69 provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
70 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
71
72 api_key_env_var = f"{provider_name.upper()}_API_KEY"
73 api_key = os.environ.get(api_key_env_var)
74 if not api_key:
75 logger.error(
76 f"{api_key_env_var} environment variable is not set! "
77 f"To run the {provider_name} handoff, set your API key. "
78 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
79)
80 return []
81 try:
82 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
83 except KeyError as e:
84 logger.error(str(e))
85 return []
86
87 video_duration = get_video_duration(video_path)
88 if video_duration <= 0:
89 logger.error("Invalid video duration.")
90 return []
91 fps, frame_width = self._probe_fps_width(video_path)
92
93 # --- TrackNet runner config + thresholds ---
94 tn_cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
95 runner = TrackNetRunner(tn_cfg)
96
97 tn = idx_cfg.get("tracknet", {})
98 velocity_thresh = tn.get("velocity_threshold", 0.02) # normalised shuttle
displacement/sec
99 merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
100 min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
101 handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
102 ai_max_workers = idx_cfg.get("ai_max_workers", 5)
103 log_candidates = idx_cfg.get("log_yolo_candidates", True)
104 store_candidates = idx_cfg.get("store_yolo_candidates", True)
105
106 detection_params = {
107 "detector": "tracknet_hybrid",
108 "velocity_thresh": velocity_thresh,
109 "merge_gap": merge_gap,
110 "min_action_window_duration": min_window_duration,
111 "weights_path": tn_cfg.weights_path,
112 "queue_length": tn_cfg.queue_length,
113 }

```

```

114
115 # --- Phase 1: env healthcheck (fail fast with a clear message) ---
116 ok, msg = runner.healthcheck()
117 if not ok:
118 logger.error("TrackNet WSL environment not ready: %s", msg)
119 return []
120 logger.info("TrackNet WSL env OK: %s", msg)
121
122 # --- Phase 2: shuttle trajectory -> candidate rally windows ---
123 # TrackNet processes the whole video in one pass (it carries its own
124 # frame buffering), so unlike the YOLO segmenter we don't pre-chunk here.
125 t_start = time.time()
126 tn_out_dir = os.path.join("output", "tracknet")
127 csv_path = runner.run_predict(video_path, tn_out_dir)
128 if not csv_path:
129 logger.error("TrackNet produced no trajectory; aborting.")
130 return []
131
132 points = runner.parse_trajectory_csv(csv_path)
133 logger.info("Parsed %d trajectory points (%d visible).",
134 len(points), sum(1 for p in points if p.visible))
135
136 all_candidate_windows = runner.trajectory_to_action_windows(
137 points, fps=fps, frame_width=frame_width, chunk_start=0.0,
138 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
139 min_window_duration=min_window_duration, chunk_index=0,
140)
141 logger.info("Phase 2 (TrackNet) completed in %.1fs. Candidate windows: %d",
142 time.time() - t_start, len(all_candidate_windows))
143
144 if log_candidates:
145 for w in all_candidate_windows:
146 logger.info(
147 f"[TrackNet rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
148 f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
149 f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f}"
150)
151
152 # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
153 candidate_ids = [None] * len(all_candidate_windows)
154 if store_candidates:
155 for i, w in enumerate(all_candidate_windows):
156 stats = {
157 "peak_velocity": w["peak_velocity"],
158 "mean_velocity": w["mean_velocity"],
159 "active_frame_count": w["active_frame_count"],
160 "track_id_count": w["track_id_count"],
161 }
162 confidence = min(1.0, w["peak_velocity"])
163 candidate_ids[i] = self.db.add_candidate_segment(
164 video_id, detector="tracknet_hybrid",
165 start_time=w["start"], end_time=w["end"],
166 chunk_index=w["chunk_index"], confidence=confidence,
167 stats=stats, detection_params=detection_params,
168)
169
170 if not all_candidate_windows:
171 return []
172
173 # --- Phase 3: AI provider handoff (identical pattern to HybridYoloSegmenter)
174
175 handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
176 os.makedirs(handoff_dir, exist_ok=True)
177 logger.info("Phase 3: %s validation on %d candidate windows...",

```

```

177 provider_name, len(all_candidate_windows))
178
179 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
180 w_start = max(0, window["start"] - handoff_padding)
181 w_end = min(video_duration, window["end"] + handoff_padding)
182 w_dur = w_end - w_start
183 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
184 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
185 return []
186 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
187 segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
188 label=f"Handoff {wi+1}")
189 valid = []
190 for seg in segments:
191 try:
192 seg["start_time"] = float(seg["start_time"]) + w_start
193 seg["end_time"] = float(seg["end_time"]) + w_start
194 seg["_candidate_id"] = candidate_ids[wi]
195 valid.append(seg)
196 except (ValueError, TypeError, KeyError) as e:
197 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
198 try:
199 os.remove(w_path)
200 except OSError:
201 pass
202 return valid
203
204 ai_segments: List[dict] = []
205 with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
206 futures = [executor.submit(process_candidate_window, wi, w)
207 for wi, w in enumerate(all_candidate_windows)]
208 for future in futures:
209 ai_segments.extend(future.result())
210
211 try:
212 os.rmdir(handoff_dir)
213 except OSError:
214 pass
215 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
216
217 # --- Phase 4: DB population (identical pattern to HybridYoloSegmenter) ---
218 segments_data = []
219 for segment in ai_segments:
220 try:
221 start = float(segment["start_time"])
222 end = float(segment["end_time"])
223 dur = end - start
224 shot_count = segment.get("shuttle_exchange_count")
225 if shot_count is None:
226 shot_count = max(3, int(dur / 0.8))
227 else:
228 try:
229 shot_count = int(shot_count)
230 except (ValueError, TypeError):
231 shot_count = max(3, int(dur / 0.8))
232 metadata = {
233 "shot_count": shot_count,
234 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
235 "detector": f"tracknet-hybrid-{model_name}",
236 }

```

```

237 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
238 if segment_id:
239 candidate_id = segment.get("_candidate_id")
240 if candidate_id is not None:
241 self.db.link_candidate_to_segment(candidate_id, segment_id)
242 segments_data.append({
243 "id": segment_id, "start_time": start, "end_time": end,
244 "duration": dur, "metadata": metadata,
245 })
246 except Exception as e:
247 logger.warning("Skipping invalid segment struct: %s", e)
248
249 return segments_data
250
251
252 # Register the segmenter
253 segmenter_registry.register(TrackNetHybridSegmenter)
254

```

#### 44. user (2026-06-06T15:28:20.416Z)

```

1 import cv2
2 import time
3 import numpy as np
4 import random
5 from typing import List, Dict, Any, Tuple
6 from backend.database import Database
7 from backend.indexer.base import BasePlaySegmenter, segmenter_registry
8
9 class MotionPlaySegmenter(BasePlaySegmenter):
10 @property
11 def name(self) -> str:
12 return "motion"
13
14 @property
15 def description(self) -> str:
16 return "Traditional CV pixel-motion heuristics tracking frame differences."
17
18 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
19 """
20 Processes a video to detect play segments using motion heuristics,
21 populates SQLite with detected segments, and indexes detailed events.
22 """
23 if not player_pool:
24 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
25
26 cap = cv2.VideoCapture(video_path)
27 if not cap.isOpened():
28 print(f"[ERROR] Segmenter could not open video: {video_path}")
29 return []
30
31 fps = cap.get(cv2.CAP_PROP_FPS)
32 total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
33 duration = total_frames / fps if fps > 0 else 0
34
35 # Sub-sample settings (5 FPS for analysis)
36 analysis_fps = 5.0
37 frame_interval = max(1, int(fps / analysis_fps))
38
39 idx_cfg = self.config.get("indexing", {})
40 motion_threshold = idx_cfg.get("motion_threshold", 0.08)
41 min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
42 dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

```

```

43
44 prev_gray = None
45 motion_signals = []
46 timestamps = []
47
48 frame_idx = 0
49 processed_count = 0
50 t_start = time.time()
51 # Emit a progress line roughly every 5% of the video so long runs are
52 # observable instead of silent (a full 1080p match is ~tens of minutes).
53 progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
54 next_progress = progress_step
55 print(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
56 f"(~{duration/60:.1f} min), analyzing every {frame_interval}th frame",
flush=True)
57 while True:
58 # Skip decoding for intermediate frames using cap.grab() for high
performance
59 for _ in range(frame_interval - 1):
60 if not cap.grab():
61 break
62 frame_idx += 1
63
64 ret, frame = cap.read()
65 if not ret:
66 break
67
68 # Process frame
69 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
70 # Apply Gaussian Blur to smooth noise
71 gray = cv2.GaussianBlur(gray, (21, 21), 0)
72
73 t = frame_idx / fps
74 timestamps.append(t)
75
76 if prev_gray is not None:
77 # Frame absolute difference (highly optimized motion tracking)
78 diff = cv2.absdiff(prev_gray, gray)
79 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
80
81 # Dilate to fill gaps
82 thresh = cv2.dilate(thresh, None, iterations=2)
83
84 # Motion ratio
85 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
86 motion_signals.append(motion_ratio)
87 else:
88 motion_signals.append(0.0)
89
90 prev_gray = gray
91 frame_idx += 1
92 processed_count += 1
93
94 if progress_step and frame_idx >= next_progress:
95 pct = 100.0 * frame_idx / total_frames
96 elapsed = time.time() - t_start
97 eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
98 print(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames, "
99 f"{processed_count} analyzed, {elapsed:.0f}s elapsed, ~{eta:.0f}s
left", flush=True)
100 next_progress += progress_step
101
102 if max_frames is not None and processed_count >= max_frames:
103 print(f"[DEBUG] Reached max_frames limit of {max_frames}. Halting

```

```

indexer frame parsing.")
104 break
105
106 if frame_idx >= total_frames:
107 break
108
109 cap.release()
110
111 # Smooth signal using a moving average
112 window_size = 5
113 smoothed = []
114 for i in range(len(motion_signals)):
115 start = max(0, i - window_size // 2)
116 end = min(len(motion_signals), i + window_size // 2 + 1)
117 smoothed.append(sum(motion_signals[start:end]) / (end - start))
118
119 # Detect active play boundaries
120 in_segment = False
121 segment_start = None
122 raw_segments: List[Tuple[float, float]] = []
123
124 for i, t in enumerate(timestamps):
125 signal = smoothed[i]
126 if signal > motion_threshold:
127 if not in_segment:
128 in_segment = True
129 segment_start = t
130 else:
131 if in_segment:
132 in_segment = False
133 raw_segments.append((segment_start, t))
134
135 # Handle video ending while in active segment
136 if in_segment:
137 raw_segments.append((segment_start, timestamps[-1]))
138
139 # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
140 merged_segments: List[Tuple[float, float]] = []
141 if raw_segments:
142 curr_start, curr_end = raw_segments[0]
143 for next_start, next_end in raw_segments[1:]:
144 if next_start - curr_end < dead_time_merge_gap:
145 # Merge
146 curr_end = next_end
147 else:
148 merged_segments.append((curr_start, curr_end))
149 curr_start, curr_end = next_start, next_end
150 merged_segments.append((curr_start, curr_end))
151
152 # Post-Processing: 2. Filter segments shorter than min_segment_duration
153 final_segments: List[Tuple[float, float]] = [
154 r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
155]
156
157 # Populate SQLite DB with segments and metadata events
158 segments_data = []
159 for start, end in final_segments:
160 # Assign Server and Receiver randomly from pool
161 server, receiver = random.sample(player_pool, 2)
162
163 # Formulate dynamic rich tags
164 dur = end - start
165 metadata = {
166 "shot_count": random.randint(4, 25),
167 "intensity": "high" if dur > 12 else "medium",

```

```

168 "avg_speed_kmh": round(random.uniform(40, 85), 1)
169 }
170
171 segment_id = self.db.add_play_segment(video_id, start, end, server,
172 receiver, metadata)
173
174 if segment_id:
175 # Add sub-segment events
176 # A: Serve Event (always at start)
177 serve_foul = random.random() < 0.15 # 15% chance of serve foul
178 self.db.add_event(
179 segment_id=segment_id,
180 timestamp=start + 0.5,
181 event_type="serve",
182 player=server,
183 details={"status": "foul" if serve_foul else "valid", "type":
184 "low_serve" if random.random() > 0.5 else "high_serve"}
185)
186
187 # B: Smash Events (1 to 3 random smashes per segment)
188 smashes_count = random.randint(0, 3)
189 for _ in range(smashes_count):
190 smash_time = random.uniform(start + 1.5, end - 1.0)
191 smasher = random.choice([server, receiver])
192 self.db.add_event(
193 segment_id=segment_id,
194 timestamp=round(smash_time, 2),
195 event_type="smash",
196 player=smasher,
197 details={"speed_kmh": round(random.uniform(120, 250), 1)}
198)
199
200 # C: Segment Termination Event (foul/winner)
201 foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
202 is_foul = random.random() > 0.4
203 error_player = random.choice([server, receiver])
204
205 self.db.add_event(
206 segment_id=segment_id,
207 timestamp=end - 0.2,
208 event_type="foul" if is_foul else "winner",
209 player=error_player,
210 details={"type": random.choice(foul_types)} if is_foul else
211 {"shot_type": "drop" if random.random() > 0.5 else "smash"}
212)
213
214 # Store segment representation to return to caller
215 segments_data.append({
216 "id": segment_id,
217 "start_time": start,
218 "end_time": end,
219 "duration": dur,
220 "server": server,
221 "receiver": receiver,
222 "metadata": metadata
223 })
224
225 return segments_data
226
227 # Register the segmenter
228 segmenter_registry.register(MotionPlaySegmenter)
229

```



Both confirmed:

- tracknet\_hybrid: `TrackNetHybridSegmenter` (name 'tracknet\_hybrid'), copy-paste twin of wasb, `detection\_params` adds `queue\_length` ?, `\_probe\_fps\_width` fallback. ?
- motion: `MotionPlaySegmenter` (name 'motion'), pixel absdiff, metadata + serve/smash/foul/winner events are RANDOM (random.sample/randint/uniform/choice), uses `db.add\_event` + server/receiver, uses `print()`, honors `max\_frames`. ? metadata adds `intensity/avg\_speed\_kmh` (random) ?.

Now the remaining: gemini segmenter, validators, stitcher, utils, tools, badminton, providers/gemini, and the run\_\* / config / setup entrypoints. Let me do targeted grep checks for symbols plus read the smaller files.

#### 46. user (2026-06-06T15:28:27.591Z)

```
1 import os
2 import subprocess
3 import tempfile
4 from typing import List, Tuple, Union
5
6 class VideoCompiler:
7 def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
8 self.output_dir = output_dir
9 self.end_padding = end_padding
10 self.begin_padding = begin_padding
11 os.makedirs(output_dir, exist_ok=True)
12
13 @staticmethod
14 def _parse_time(val: Union[int, float, str]) -> float:
15 """Normalizes a timestamp value to float seconds.
16 Handles: float/int (pass-through), plain numeric strings ("12.4"),
17 and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
18 This mirrors the parse_time logic used in gemini.py to ensure
19 the stitcher can always consume whatever format the indexer produced."""
20 if isinstance(val, (int, float)):
21 return float(val)
22 if isinstance(val, str):
23 val = val.strip()
24 if ":" in val:
25 # Handle MM:SS, HH:MM:SS, etc.
26 parts = val.split(":")
27 parts.reverse()
28 total = 0.0
29 for i, p in enumerate(parts):
30 try:
31 total += float(p) * (60 ** i)
32 except ValueError:
33 pass
34 return total
35 try:
36 return float(val)
37 except ValueError:
38 print(f"[COMPILER WARNING] Could not parse timestamp value '{val}' as a
number. Defaulting to 0.0.")
39 return 0.0
40 print(f"[COMPILER WARNING] Unexpected timestamp type {type(val).__name__} for
value '{val}'. Defaulting to 0.0.")
41 return 0.0
42
43 def _get_video_duration(self, video_path: str) -> float:
44 """Probes the video file to get its total duration in seconds.
45 Returns 0.0 if it cannot be determined."""
46 try:
47 result = subprocess.run(
48 [
```

```

49 "ffprobe", "-v", "error",
50 "-show_entries", "format=duration",
51 "-of", "default=noprint_wrappers=1:nokey=1",
52 video_path
53],
54 capture_output=True, text=True, check=True
55)
56 return float(result.stdout.strip())
57 except Exception as e:
58 print(f"[COMPILER WARNING] Could not determine video duration via ffprobe:
{e}")
59 return 0.0
60
61 def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
62 """
63 Extracts selected segments from a raw video and stitches them together
64 into a seamless highlights file, applying end_padding to prevent abrupt endings.
65 """
66 if not segments:
67 raise ValueError("No highlight segments provided to compile.")
68
69 if not os.path.exists(raw_video_path):
70 raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
71
72 output_path = os.path.join(self.output_dir, output_filename)
73
74 # Probe video duration so we can detect out-of-range segments
75 video_duration = self._get_video_duration(raw_video_path)
76 if video_duration > 0:
77 print(f"[COMPILER] Source video duration: {video_duration:.2f}s")
78 else:
79 print(f"[COMPILER WARNING] Video duration unknown. Cannot validate segment
boundaries against video length.")
80
81 # We will create temporary clips and stitch them using FFmpeg concat demuxer
82 temp_clips = []
83 skipped_segments = []
84
85 # Create a temp directory within output_dir
86 with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
87 print(f"[COMPILER] Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
88 for idx, (raw_start, raw_end) in enumerate(segments):
89 # Normalize timestamps to float seconds ? handles float, int,
90 # plain numeric strings, and colon-separated formats like "MM:SS"
91 start = self._parse_time(raw_start)
92 end = self._parse_time(raw_end)
93 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
94
95 # Validate the segment times
96 if start < 0:
97 print(f"[COMPILER WARNING] {segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
98 start = 0.0
99
100 if start >= end:
101 print(f"[COMPILER ERROR] {segment_label}: start_time ({start:.2f}s)
>= end_time ({end:.2f}s). Skipping invalid segment.")
102 skipped_segments.append((idx, start, end, "start >= end"))
103 continue
104
105 # Check if segment extends beyond video duration

```

```

106 if video_duration > 0:
107 if start >= video_duration:
108 print(f"[COMPILER ERROR] {segment_label}: start_time
({start:.2f}s) is beyond the video duration ({video_duration:.2f}s). "
109 f"This segment cannot be extracted ? the video ran out.
Skipping.")
110 skipped_segments.append((idx, start, end, "start beyond video
end"))
111 continue
112
113 if end > video_duration:
114 original_end = end
115 end = video_duration
116 print(f"[COMPILER WARNING] {segment_label}: end_time
({original_end:.2f}s) exceeds video duration ({video_duration:.2f}s). "
117 f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")
118
119 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
120
121 # Precise sub-second trim using fast re-encoding to preserve stream
headers
122 padded_start = max(0.0, start - self.begin_padding)
123 padded_end = end + self.end_padding
124
125 # Clamp padded_end to video duration if known
126 if video_duration > 0 and padded_end > video_duration:
127 padded_end = video_duration
128 print(f"[COMPILER INFO] {segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
129 f"Clamping to {video_duration:.2f}s (end padding partially
applied).")
130
131 trim_duration = padded_end - padded_start
132 print(f"[COMPILER] Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
133
134 cmd = [
135 "ffmpeg", "-y",
136 "-ss", str(round(padded_start, 3)),
137 "-to", str(round(padded_end, 3)),
138 "-i", raw_video_path,
139 "-c:v", "libx264",
140 "-preset", "superfast",
141 "-crf", "22",
142 "-c:a", "aac",
143 "-b:a", "128k",
144 clip_path
145]
146
147 try:
148 result = subprocess.run(cmd, capture_output=True, check=True)
149 # Verify the output clip was actually created and has content
150 if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
151 temp_clips.append(clip_path)
152 else:
153 print(f"[COMPILER ERROR] {segment_label}: FFmpeg exited
successfully but output clip is missing or empty. Skipping.")
154 skipped_segments.append((idx, start, end, "empty output clip"))
155 except subprocess.CalledProcessError as e:
156 stderr_msg = e.stderr.decode(errors='replace').strip()
157 print(f"[COMPILER ERROR] {segment_label}: FFmpeg trim failed.")
158 print(f" Command: {' '.join(cmd)}")
159 print(f" FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding

```

```

160 skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
161 continue
162
163 # Report summary of skipped segments
164 if skipped_segments:
165 print(f"\n[COMPILER SUMMARY] {len(skipped_segments)} out of
{len(segments)} segments were SKIPPED:")
166 for seg_idx, seg_start, seg_end, reason in skipped_segments:
167 print(f" - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
168 print()
169
170 if not temp_clips:
171 raise RuntimeError(
172 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
173 f"All {len(segments)} segments were skipped. "
174 f"Check the errors above for details. "
175 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
176)
177
178 print(f"[COMPILER] Successfully trimmed {len(temp_clips)}/{len(segments)}
clips. Preparing to stitch...")
179
180 # Create a list file for FFmpeg concat demuxer
181 list_file_path = os.path.join(tmp_dir, "clips.txt")
182 with open(list_file_path, "w") as f:
183 for clip in temp_clips:
184 # Escape single quotes in path
185 safe_path = clip.replace("'", "'\\'")
186 f.write(f"file '{safe_path}'\n")
187
188 print("[COMPILER] Stitching clips together...")
189 # Stitch them together without re-encoding (since they were encoded with
identical settings)
190 concat_cmd = [
191 "ffmpeg", "-y",
192 "-f", "concat",
193 "-safe", "0",
194 "-i", list_file_path,
195 "-c", "copy",
196 output_path
197]
198
199 try:
200 result = subprocess.run(concat_cmd, capture_output=True, check=True)
201
202 # Final verification
203 if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
204 raise RuntimeError(
205 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
206 f"but output file is missing or empty: {output_path}"
207)
208
209 output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
210 print(f"[SUCCESS] Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
211 return output_path
212 except subprocess.CalledProcessError as e:
213 stderr_msg = e.stderr.decode(errors='replace').strip()
214 raise RuntimeError(
215 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
216 f"Command: {' '.join(concat_cmd)}\n"
217 f"FFmpeg stderr: {stderr_msg[-500:]}")

```

218 )  
219

47. user (2026-06-06T15:28:28.067Z)

```

1 from abc import ABC, abstractmethod
2 from pydantic import BaseModel
3 from typing import List, Dict, Any, Optional
4
5 class ValidationResult(BaseModel):
6 validator_name: str
7 passed: bool
8 message: str
9 details: Dict[str, Any] = {}
10
11 class VideoValidator(ABC):
12 @property
13 @abstractmethod
14 def name(self) -> str:
15 """Unique identifier for this validator."""
16 pass
17
18 @property
19 @abstractmethod
20 def description(self) -> str:
21 """Human-readable explanation of what this validator checks."""
22 pass
23
24 @abstractmethod
25 def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any]) -> ValidationResult:
26 """
27 Executes validation.
28 - video_path: Local path to the MP4 file.
29 - config: Dict parsed from config.json.
30 - context: Dynamic context dict shared across validators (stores shared values
31 like ffprobe metadata).
32 """
33 pass
34
35 class ValidationRegistry:
36 def __init__(self):
37 self._validators: List[VideoValidator] = []
38
39 def register(self, validator: VideoValidator):
40 """Registers a new validator."""
41 self._validators.append(validator)
42
43 def run_all(self, video_path: str, config: Dict[str, Any]) -> List[ValidationResult]:
44 """Runs all registered validators in sequence, accumulating results."""
45 results = []
46 context = {} # Shared context dictionary (e.g. for caching ffprobe outputs)
47 for val in self._validators:
48 try:
49 res = val.validate(video_path, config, context)
50 results.append(res)
51 except Exception as e:
52 results.append(ValidationResult(
53 validator_name=val.name,
54 passed=False,
55 message=f"Validator threw an unexpected error: {str(e)}")
56))
57 return results

```

```
58 # Shared global registry
59 registry = ValidationRegistry()
60
```

#### 48. user (2026-06-06T15:28:29.018Z)

```
1 import math
2 from typing import Tuple
3
4 def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5 """Calculate the center (x, y) of a bounding box.
6 bbox format: (x1, y1, x2, y2)
7 """
8 x1, y1, x2, y2 = bbox
9 return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11 def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12 """Calculate the Euclidean distance between two points (x, y)."""
13 return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15 def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
16 float, float], fps: float) -> float:
17 """Calculate velocity (pixels per second) of a bounding box moving between two
18 consecutive frames.
19 """
20 c1 = get_center(bbox1)
21 c2 = get_center(bbox2)
22 dist = get_distance(c1, c2)
23 return dist * fps
24
25 def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
26 Tuple[float, float, float, float],
27 fps: float, frame_width: float) -> float:
28 """Calculate velocity as a fraction of frame width per second.
29
30 This makes the threshold resolution-independent:
31 - 0.15 means the center moved 15% of the frame width in one second.
32 - Works identically on 720p, 1080p, and 4K footage.
33
34 Args:
35 bbox1: Previous bounding box (x1, y1, x2, y2).
36 bbox2: Current bounding box (x1, y1, x2, y2).
37 fps: Effective frames per second (after frame-skipping).
38 frame_width: Width of the video frame in pixels.
39 """
40 raw_velocity = get_velocity(bbox1, bbox2, fps)
41 if frame_width <= 0:
42 return raw_velocity # Fallback to raw pixels if width unknown
43 return raw_velocity / frame_width
44
45 def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
46 float, float]) -> float:
47 """Calculate the Intersection over Union (IoU) of two bounding boxes.
48 bbox format: (x1, y1, x2, y2)
49 """
50 xA = max(boxA[0], boxB[0])
51 yA = max(boxA[1], boxB[1])
52 xB = min(boxA[2], boxB[2])
53 yB = min(boxA[3], boxB[3])
54
55 interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
56 if interArea == 0:
57 return 0.0
58
59 boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
```

```

56 boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
57
58 iou = interArea / float(boxAArea + boxBArea - interArea)
59 return iou
60

```

#### 49. user (2026-06-06T15:28:29.096Z)

```

1 import subprocess
2 import os
3 import shutil
4 import logging
5
6 logger = logging.getLogger(__name__)
7
8 def get_video_duration(video_path: str) -> float:
9 """Returns the exact duration of a video file in seconds using ffprobe.
10 Returns 0.0 on failure."""
11 try:
12 duration_output = subprocess.check_output([
13 "ffprobe", "-v", "error", "-show_entries", "format=duration",
14 "-of", "default=noprint_wrappers=1:nokey=1", video_path
15]).decode().strip()
16 return float(duration_output)
17 except Exception as e:
18 logger.error(f"Could not detect video duration for {video_path}: {e}")
19 return 0.0
20
21 def extract_video_chunk(video_path: str, start_time: float, duration: float,
22 output_path: str, reencode: bool = False) -> bool:
23 """Extracts a chunk from a video file.
24
25 Args:
26 reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27 If False, uses stream-copy (fast but cuts at nearest keyframe).
28 Use True for short clips where keyframe drift matters.
29
30 """
31 cmd = [
32 "ffmpeg", "-y",
33 "-ss", str(start_time),
34 "-t", str(duration),
35 "-i", video_path,
36]
37 if reencode:
38 cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
39 else:
40 cmd += ["-c", "copy"]
41 cmd.append(output_path)
42 try:
43 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
44 stderr=subprocess.DEVNULL)
45 return True
46 except Exception as e:
47 logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s: {e}")
48 return False
49

```

#### 50. assistant (2026-06-06T15:28:37.057Z)

All confirmed:

- compiler.py: `VideoCompiler.compile\_highlights(raw, segments, out\_name)`, per-clip re-encode (libx264/superfast/crf22), `-f concat -c copy`, skipped tuple `(idx,start,end,reason)`, if duration probe fails (0.0) no boundary validation, `padded\_start` uses `begin\_padding`, temp clips in output\_dir, raises ValueError/FileNotFoundError/RuntimeError, uses print(). ? Note

```
`begin_padding=0.5`, `end_padding=1.0` defaults match.
- validators/base.py: `VideoValidator` ABC, `ValidationResult` (pydantic), `ValidationRegistry`,
`registry`, shared context dict, exceptions wrapped as passed=False (never abort). ?
- geometry.py: `get_velocity_normalised`, `calculate_iou` (+1 px convention),
`get_center/get_distance/get_velocity`. ?
- video.py: `get_video_duration`, `extract_video_chunk(...,reencode=False)`, uses `-t DURATION`,
copy mode keyframe drift, reencode=True available. ?
```

Now the validators (format, resolution\_fps, scene\_cut, pts\_continuity, relevance), gemini segmenter, badminton, providers/gemini, rally\_slicer, and run\_\* entrypoints. Let me grep for the key validator symbols and gotchas first, then read the entrypoints and remaining files.

## 51. user (2026-06-06T15:28:37.980Z)

```
1 import subprocess
2 import json
3 import os
4 from backend.validators.base import VideoValidator, ValidationResult
5 from typing import Dict, Any
6
7 class FormatValidator(VideoValidator):
8 @property
9 def name(self) -> str:
10 return "format_check"
11
12 @property
13 def description(self) -> str:
14 return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16 def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17 if not os.path.exists(video_path):
18 return ValidationResult(
19 validator_name=self.name,
20 passed=False,
21 message=f"File not found: {video_path}"
22)
23
24 cmd = [
25 "ffprobe",
26 "-v", "error",
27 "-select_streams", "v:0",
28 "-show_format",
29 "-show_streams",
30 "-of", "json",
31 video_path
32]
33
34 try:
35 result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36 info = json.loads(result.stdout)
37
38 if not info.get("streams") or not info.get("format"):
39 return ValidationResult(
40 validator_name=self.name,
41 passed=False,
42 message="Failed to retrieve valid video streams from the file."
43)
44
45 # Store the metadata in context for subsequent validators to utilize
46 context["ffprobe_meta"] = info
47
48 stream = info["streams"][0]
49 fmt = info["format"]
```



```

50
51 details = {
52 "format_name": fmt.get("format_name"),
53 "duration": float(fmt.get("duration", 0)),
54 "size_bytes": int(fmt.get("size", 0)),
55 "codec_name": stream.get("codec_name"),
56 }
57
58 # Check if container name includes mp4
59 if "mp4" not in fmt.get("format_name", "").lower():
60 return ValidationResult(
61 validator_name=self.name,
62 passed=False,
63 message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
64 details=details
65)
66
67 return ValidationResult(
68 validator_name=self.name,
69 passed=True,
70 message="Video format is valid MP4 container.",
71 details=details
72)
73
74 except subprocess.CalledProcessError as e:
75 return ValidationResult(
76 validator_name=self.name,
77 passed=False,
78 message="Video stream is corrupted or unreadable by ffprobe.",
79 details={"error": e.stderr}
80)
81 except Exception as e:
82 return ValidationResult(
83 validator_name=self.name,
84 passed=False,
85 message=f"Unexpected error running ffprobe: {str(e)}"
86)
87

```

## 52. user (2026-06-06T15:28:38.926Z)

```

1 from backend.validators.base import VideoValidator, ValidationResult
2 from typing import Dict, Any
3
4 class ResolutionFPSValidator(VideoValidator):
5 @property
6 def name(self) -> str:
7 return "resolution_fps_check"
8
9 @property
10 def description(self) -> str:
11 return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13 def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14 meta = context.get("ffprobe_meta")
15 if not meta or not meta.get("streams"):
16 return ValidationResult(
17 validator_name=self.name,
18 passed=False,
19 message="Video metadata is missing. Run format_check validator first."
20)
21

```

```

22 stream = meta["streams"][0]
23 width = int(stream.get("width", 0))
24 height = int(stream.get("height", 0))
25
26 # Parse frame rate (e.g. "30/1" or "30000/1001")
27 fps_str = stream.get("r_frame_rate", "0/1")
28 try:
29 num, den = map(int, fps_str.split("/"))
30 fps = num / den if den != 0 else 0
31 except ValueError:
32 fps = 0.0
33
34 details = {
35 "width": width,
36 "height": height,
37 "fps": round(fps, 2)
38 }
39
40 # Check bounds from config
41 val_cfg = config.get("validation", {})
42 min_w = val_cfg.get("min_resolution_width", 1280)
43 min_h = val_cfg.get("min_resolution_height", 720)
44 min_fps = val_cfg.get("min_fps", 29.9)
45
46 if width < min_w or height < min_h:
47 return ValidationResult(
48 validator_name=self.name,
49 passed=False,
50 message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51 details=details
52)
53
54 if fps < min_fps:
55 return ValidationResult(
56 validator_name=self.name,
57 passed=False,
58 message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
59 details=details
60)
61
62 return ValidationResult(
63 validator_name=self.name,
64 passed=True,
65 message="Video resolution and frame rate meet minimum standards.",
66 details=details
67)
68

```

### 53. user (2026-06-06T15:28:39.515Z)

```

1 import cv2
2 import numpy as np
3 from backend.validators.base import VideoValidator, ValidationResult
4 from typing import Dict, Any
5
6 class SceneCutDensityValidator(VideoValidator):
7 @property
8 def name(self) -> str:
9 return "scene_cut_check"
10
11 @property
12 def description(self) -> str:
13 return "Analyzes visual frames to detect abrupt scene cuts, which indicate an

```

```

edited or compiled video."
14
15 def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16 cap = cv2.VideoCapture(video_path)
17 if not cap.isOpened():
18 return ValidationResult(
19 validator_name=self.name,
20 passed=False,
21 message="Failed to open video file using OpenCV."
22)
23
24 fps = cap.get(cv2.CAP_PROP_FPS)
25 total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
26 duration = total_frames / fps if fps > 0 else 0
27
28 if duration <= 0:
29 cap.release()
30 return ValidationResult(
31 validator_name=self.name,
32 passed=False,
33 message="Invalid video duration."
34)
35
36 # Sample at 2 frames per second to keep it fast
37 sample_interval = max(1, int(fps / 2))
38
39 cuts_detected = 0
40 prev_hist = None
41
42 frame_idx = 0
43 sampled_count = 0
44
45 # Adjust histogram comparison thresholds
46 # Using correlation: 1.0 is perfect match, < 0.6 indicates a sudden massive
scene change
47 cut_threshold = 0.6
48
49 while True:
50 cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
51 ret, frame = cap.read()
52 if not ret:
53 break
54
55 # Calculate color histogram
56 hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
57 hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
58 cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)
59
60 if prev_hist is not None:
61 # Compare correlation
62 score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)
63 if score < cut_threshold:
64 cuts_detected += 1
65
66 prev_hist = hist
67 sampled_count += 1
68 frame_idx += sample_interval
69
70 # Cap check at 100 frames max (e.g. first 50 seconds or evenly spread) to
keep validation instant
71 if frame_idx >= total_frames or sampled_count > 100:
72 break
73
74 cap.release()

```

```

75
76 # Compute cuts per minute
77 sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
78 cuts_per_minute = (cuts_detected / sampled_duration) * 60 if sampled_duration >
0 else 0
79
80 details = {
81 "cuts_detected": cuts_detected,
82 "sampled_frames": sampled_count,
83 "cuts_per_minute": round(cuts_per_minute, 3)
84 }
85
86 val_cfg = config.get("validation", {})
87 max_cuts_rate = val_cfg.get("max_scene_cuts_per_minute", 0.5)
88
89 if cuts_per_minute > max_cuts_rate:
90 return ValidationResult(
91 validator_name=self.name,
92 passed=False,
93 message=f"Video splicing compromise: Detected {cuts_detected} cuts
({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of
{max_cuts_rate} cuts/min.",
94 details=details
95)
96
97 return ValidationResult(
98 validator_name=self.name,
99 passed=True,
100 message="Scene continuity check passed. No abnormal visual cuts detected.",
101 details=details
102)
103

```

#### 54. user (2026-06-06T15:28:39.910Z)

```

1 import subprocess
2 from backend.validators.base import VideoValidator, ValidationResult
3 from typing import Dict, Any
4
5 class PTSContinuityValidator(VideoValidator):
6 @property
7 def name(self) -> str:
8 return "pts_continuity_check"
9
10 @property
11 def description(self) -> str:
12 return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
timeline gaps, jumps, or editing splices."
13
14 def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15 # Command to extract packet PTS and flags (extremely fast, works on H.264 &
H.265)
16 cmd = [
17 "ffprobe",
18 "-v", "error",
19 "-select_streams", "v:0",
20 "-show_entries", "packet=pts_time,flags",
21 "-of", "csv=p=0",
22 video_path
23]
24
25 try:
26 result = subprocess.run(cmd, capture_output=True, text=True, check=True)
27 output = result.stdout.strip()

```

```

28
29 if not output:
30 return ValidationResult(
31 validator_name=self.name,
32 passed=False,
33 message="No packet timestamps could be extracted from the video
stream."
34)
35
36 timestamps = []
37 for line in output.split("\n"):
38 line = line.strip()
39 if line:
40 parts = line.split(",")
41 if len(parts) == 2:
42 pts_str, flags = parts
43 if "K" in flags: # Check if keyframe
44 try:
45 timestamps.append(float(pts_str))
46 except ValueError:
47 continue
48
49 if len(timestamps) < 2:
50 return ValidationResult(
51 validator_name=self.name,
52 passed=True,
53 message="Video is too short to perform a PTS continuity check.",
54 details={"keyframes_count": len(timestamps)}
55)
56
57 # Analyze gaps
58 max_gap = 0.0
59 backward_jumps = 0
60 gaps = []
61
62 for i in range(1, len(timestamps)):
63 gap = timestamps[i] - timestamps[i-1]
64 gaps.append(gap)
65 if gap < 0:
66 backward_jumps += 1
67 if gap > max_gap:
68 max_gap = gap
69
70 # An continuous GoPro recording usually has keyframe intervals (GOP size) of
1.0 to 4.0 seconds.
71 # If there's a keyframe gap larger than 8.0 seconds or a negative gap
(backward jump), it's highly likely spliced.
72 details = {
73 "keyframes_count": len(timestamps),
74 "max_keyframe_gap_seconds": round(max_gap, 2),
75 "backward_jumps_detected": backward_jumps,
76 "first_keyframe": timestamps[0],
77 "last_keyframe": timestamps[-1]
78 }
79
80 if backward_jumps > 0:
81 return ValidationResult(
82 validator_name=self.name,
83 passed=False,
84 message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
85 details=details
86)
87
88 # Threshold for sudden gap (e.g. 10.0 seconds)

```

```

89 if max_gap > 10.0:
90 return ValidationResult(
91 validator_name=self.name,
92 passed=False,
93 message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
94 details=details
95)
96
97 return ValidationResult(
98 validator_name=self.name,
99 passed=True,
100 message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
101 details=details
102)
103
104 except subprocess.CalledProcessError as e:
105 return ValidationResult(
106 validator_name=self.name,
107 passed=False,
108 message="Failed to execute ffprobe PTS analysis.",
109 details={"error": e.stderr}
110)
111 except Exception as e:
112 return ValidationResult(
113 validator_name=self.name,
114 passed=False,
115 message=f"Unexpected error in PTS validation: {str(e)}"
116)
117

```

## 55. user (2026-06-06T15:28:40.885Z)

```

1 import cv2
2 import numpy as np
3 from backend.validators.base import VideoValidator, ValidationResult
4 from typing import Dict, Any
5
6 class RelevanceValidator(VideoValidator):
7 @property
8 def name(self) -> str:
9 return "relevance_check"
10
11 @property
12 def description(self) -> str:
13 return "Checks if the video content is relevant to badminton by scanning for
court color palettes and geometric features."
14
15 def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16 cap = cv2.VideoCapture(video_path)
17 if not cap.isOpened():
18 return ValidationResult(
19 validator_name=self.name,
20 passed=False,
21 message="Failed to open video file for relevance validation."
22)
23
24 total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
25 if total_frames <= 0:
26 cap.release()
27 return ValidationResult(
28 validator_name=self.name,
29 passed=False,

```

```

30 message="Invalid frame count."
31)
32
33 # Sample 10 frames across the video
34 num_samples = 10
35 step = max(1, total_frames // num_samples)
36
37 green_ratios = []
38
39 # Get sport profile config
40 sport_name = config.get("sport", "badminton")
41 from backend.sports import sport_registry
42 try:
43 sport_profile = sport_registry.get(sport_name)
44 sport_rel_cfg = sport_profile.get_relevance_config()
45 except KeyError:
46 sport_rel_cfg = {}
47
48 val_cfg = config.get("validation", {})
49 rel_cfg = val_cfg.get("relevance", {})
50
51 # Read bounds from sport profile with fallback to user config/defaults
52 hsv_lower = np.array(rel_cfg.get("court_green_lower",
sport_rel_cfg.get("court_green_lower", [35, 40, 40])))
53 hsv_upper = np.array(rel_cfg.get("court_green_upper",
sport_rel_cfg.get("court_green_upper", [85, 255, 255])))
54 min_court_ratio = rel_cfg.get("court_pixels_min_ratio",
sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
55
56 for i in range(num_samples):
57 frame_idx = i * step
58 cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
59 ret, frame = cap.read()
60 if not ret:
61 continue
62
63 # Downscale frame for speed
64 h, w = frame.shape[:2]
65 small_frame = cv2.resize(frame, (320, 240))
66
67 # Convert to HSV color space
68 hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)
69
70 # Mask court green pixels
71 mask = cv2.inRange(hsv, hsv_lower, hsv_upper)
72 green_pixels = cv2.countNonZero(mask)
73 total_pixels = mask.shape[0] * mask.shape[1]
74
75 ratio = green_pixels / total_pixels
76 green_ratios.append(ratio)
77
78 cap.release()
79
80 if not green_ratios:
81 return ValidationResult(
82 validator_name=self.name,
83 passed=False,
84 message="Failed to extract frames for relevance checking."
85)
86
87 avg_green_ratio = sum(green_ratios) / len(green_ratios)
88
89 details = {
90 "avg_green_court_ratio": round(avg_green_ratio, 4),
91 "max_green_court_ratio": round(max(green_ratios), 4),

```

```

92 "min_green_court_ratio": round(min(green_ratios), 4),
93 "green_threshold": min_court_ratio
94 }
95
96 # If a certain amount of court color is visible, consider it relevant
97 if avg_green_ratio < min_court_ratio:
98 return ValidationResult(
99 validator_name=self.name,
100 passed=False,
101 message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
102 details=details
103)
104
105 return ValidationResult(
106 validator_name=self.name,
107 passed=True,
108 message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
109 details=details
110)
111

```

## 56. user (2026-06-06T15:28:40.952Z)

```

1 from backend.validators.base import registry, VideoValidator, ValidationResult
2 from backend.validators.format import FormatValidator
3 from backend.validators.resolution_fps import ResolutionFPSValidator
4 from backend.validators.pts_continuity import PTSContinuityValidator
5 from backend.validators.scene_cut import SceneCutDensityValidator
6 from backend.validators.blur import BlurValidator
7 from backend.validators.relevance import RelevanceValidator
8
9 # Register all default validators in sequence (dependencies first)
10 registry.register(FormatValidator())
11 registry.register(ResolutionFPSValidator())
12 registry.register(PTSContinuityValidator())
13 registry.register(SceneCutDensityValidator())
14 registry.register(BlurValidator())
15 registry.register(RelevanceValidator())
16
17 __all__ = ["registry", "VideoValidator", "ValidationResult"]
18

```

## 57. assistant (2026-06-06T15:28:53.059Z)

Validators confirmed, with an important ordering discrepancy. Let me note:

- format.py: `FormatValidator` (`format\_check`), PRODUCER of `context['ffprobe\_meta']`, container check substring `mp4` (lowercased). ?
- resolution\_fps.py: `ResolutionFPSValidator` (`resolution\_fps\_check`), CONSUMES ffprobe\_meta, hard-fails if format\_check didn't run, `min\_fps` default 29.9. ?
- scene\_cut.py: `SceneCutDensityValidator` (`scene\_cut\_check`), `cut\_threshold=0.6` hardcoded, capped at 100 sampled frames (samples at 2fps, not "first ~100 frames" ? it samples up to 100 frames at 2fps spacing, so covers ~first 50s). The map says "only first ~100 frames analyzed" ? slightly imprecise (it's 100 \*sampled\* frames at 2fps = ~50s of video), but the substance (capped at 100 samples) holds. Minor.
- pts\_continuity.py: 10.0s gap hardcoded (`if max\_gap > 10.0`), comment says 8.0s ("larger than 8.0 seconds") ? drift confirmed. ?
- relevance.py: HSV court-green, lazy `import backend.sports`, unknown sport -> empty `sport\_rel\_cfg = {}` (falls back to defaults, does NOT fail). ?

**\*\*ORDERING DISCREPANCY\*\***: The map's section 1A pipeline shows the validator order as:  
`FormatValidator` -> `ResolutionFPSValidator` -> `Blur` -> `SceneCut` -> `PTSContinuity` -> `Relevance`



But the actual registration order in `\_\_init\_\_.py` is:  
`FormatValidator -> ResolutionFPSValidator -> PTSContinuityValidator -> SceneCutDensityValidator  
-> BlurValidator -> RelevanceValidator`

So the map has Blur/SceneCut/PTSContinuity in the **\*\*wrong order\*\***. Since the map itself stresses "registration order = execution order," this is a substantive error to fix. Let me verify blur.py exists and the remaining files.

#### 58. user (2026-06-06T15:28:53.991Z)

```
1 import cv2
2 from backend.validators.base import VideoValidator, ValidationResult
3 from typing import Dict, Any
4
5 class BlurValidator(VideoValidator):
6 @property
7 def name(self) -> str:
8 return "blur_check"
9
10 @property
11 def description(self) -> str:
12 return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
13
14 def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15 cap = cv2.VideoCapture(video_path)
16 if not cap.isOpened():
17 return ValidationResult(
18 validator_name=self.name,
19 passed=False,
20 message="Failed to open video file for blur analysis."
21)
22
23 total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
24 if total_frames <= 0:
25 cap.release()
26 return ValidationResult(
27 validator_name=self.name,
28 passed=False,
29 message="Invalid frame count."
30)
31
32 # Sample 15 frames evenly distributed across the video
33 num_samples = 15
34 step = max(1, total_frames // num_samples)
35
36 variances = []
37
38 for i in range(num_samples):
39 frame_idx = i * step
40 cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
41 ret, frame = cap.read()
42 if not ret:
43 continue
44
45 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46 # Calculate the Laplacian variance (standard focus metric)
47 laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
48 variances.append(laplacian_var)
49
50 cap.release()
51
52 if not variances:
53 return ValidationResult(
```

```

54 validator_name=self.name,
55 passed=False,
56 message="Could not read frames to perform blur verification."
57)
58
59 avg_variance = sum(variances) / len(variances)
60
61 details = {
62 "avg_laplacian_variance": round(avg_variance, 2),
63 "min_laplacian_variance": round(min(variances), 2),
64 "max_laplacian_variance": round(max(variances), 2),
65 "sampled_frames_count": len(variances)
66 }
67
68 val_cfg = config.get("validation", {})
69 min_blur_threshold = val_cfg.get("min_blur_threshold", 100.0)
70
71 if avg_variance < min_blur_threshold:
72 return ValidationResult(
73 validator_name=self.name,
74 passed=False,
75 message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
76 details=details
77)
78
79 return ValidationResult(
80 validator_name=self.name,
81 passed=True,
82 message="Video sharpness meets focus requirements.",
83 details=details
84)
85

```

## 59. user (2026-06-06T15:28:54.931Z)

```

1 import os
2 import time
3 import json
4 import logging
5 import subprocess
6 import concurrent.futures
7 from typing import List, Dict, Any, Tuple
8 from backend.database import Database
9 from backend.indexer.base import BasePlaySegmenter, segmenter_registry
10 from backend.utils.video import get_video_duration, extract_video_chunk
11 from backend.sports import sport_registry
12 from backend.providers import provider_registry
13
14 logger = logging.getLogger(__name__)
15
16 class GeminiPlaySegmenter(BasePlaySegmenter):
17 @property
18 def name(self) -> str:
19 return "gemini"
20
21 @property
22 def description(self) -> str:
23 return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
24
25 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
26 """

```

```

27 Processes a video to detect play segments using Google Gemini Multimodal Video
API,
28 populates SQLite, and indexes detailed events.
29 """
30 # Get sport profile config
31 sport_name = self.config.get("sport", "badminton")
32 try:
33 sport_profile = sport_registry.get(sport_name)
34 except KeyError as e:
35 logger.error(str(e))
36 return []
37
38 # Get AI provider and model config
39 idx_cfg = self.config.get("indexing", {})
40 provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
41 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
42
43 # Get appropriate API key based on the provider
44 api_key_env_var = f"{provider_name.upper()}_API_KEY"
45 api_key = os.environ.get(api_key_env_var)
46 if not api_key:
47 logger.error(
48 f"{api_key_env_var} environment variable is not set! "
49 f"To run the {provider_name} segmenter, set your API key. "
50 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
51)
52 return []
53
54 try:
55 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
56 except KeyError as e:
57 logger.error(str(e))
58 return []
59
60 # A: Check for debug_duration to trim the video first
61 debug_duration = self.config.get("indexing", {}).get("debug_duration")
62 video_to_upload = video_path
63 is_trimmed = False
64 temp_trimmed_path = None
65 is_compressed = False
66 temp_compressed_path = None
67
68 if debug_duration:
69 logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
70 temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
71 os.makedirs("output", exist_ok=True)
72 if os.path.exists(temp_trimmed_path):
73 try:
74 os.remove(temp_trimmed_path)
75 except Exception:
76 pass
77
78 cmd = [
79 "ffmpeg", "-y",
80 "-ss", "0",
81 "-t", str(debug_duration),
82 "-i", video_path,
83 "-c", "copy",
84 temp_trimmed_path
85]
86 logger.debug(f"Running command: {' '.join(cmd)}")

```

```

87 try:
88 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
89 logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
90 video_to_upload = temp_trimmed_path
91 is_trimmed = True
92 except Exception as e:
93 logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
94
95
96
97 # Helper to clean up local temp files
98 def cleanup_temp_files():
99 if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
100 try:
101 os.remove(temp_trimmed_path)
102 except Exception:
103 pass
104
105 # 1. Determine actual video duration
106 video_duration = get_video_duration(video_to_upload)
107 if video_duration > 0:
108 logger.info(f"Detected video duration: {video_duration:.1f}s")
109 else:
110 logger.warning("Could not detect video duration. Prompt will not include
duration context.")
111
112 # 2. Decide whether to use chunked processing
113 idx_cfg = self.config.get("indexing", {})
114 chunk_duration = idx_cfg.get("chunk_duration", 300) # 5 minutes default
115 chunk_overlap = idx_cfg.get("chunk_overlap", 30) # 30 seconds overlap
116 use_chunking = video_duration > 0 and video_duration > chunk_duration
117
118 ai_segments = []
119 chunks_dir = os.path.join("output", "chunks")
120
121 if use_chunking:
122 # --- CHUNKED PROCESSING ---
123 step = chunk_duration - chunk_overlap
124 chunk_starts = []
125 t = 0.0
126 while t < video_duration:
127 chunk_starts.append(t)
128 t += step
129 num_chunks = len(chunk_starts)
130 logger.info(
131 f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
132 f"({chunk_duration}s each, {chunk_overlap}s overlap)."
133)
134
135 os.makedirs(chunks_dir, exist_ok=True)
136
137 def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
138 chunk_end = min(chunk_start + chunk_duration, video_duration)
139 actual_chunk_dur = chunk_end - chunk_start
140 chunk_label = f"Chunk {ci+1}/{num_chunks}"
141 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
142
143 # Extract chunk with ffmpeg
144 logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")

```

```

145 success = extract_video_chunk(video_to_upload, chunk_start,
actual_chunk_dur, chunk_path)
146 if not success:
147 logger.error(f"Failed to extract {chunk_label}. Skipping.")
148 return chunk_label, [], {}
149
150 # Measure chunk size
151 try:
152 chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
153 except Exception:
154 chunk_size_mb = 0.0
155
156 # Get prompt from sport profile
157 prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
158
159 # Process this chunk through the provider
160 chunk_segments, metrics = provider.analyze_video(
161 chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
162)
163 metrics["file_size_mb"] = chunk_size_mb
164 metrics["segments_found"] = len(chunk_segments)
165
166 # Offset timestamps back to full-video coordinates
167 for segment in chunk_segments:
168 if "start_time" in segment:
169 try:
170 segment["start_time"] = float(segment["start_time"]) +
chunk_start
171 except (ValueError, TypeError):
172 pass
173 if "end_time" in segment:
174 try:
175 segment["end_time"] = float(segment["end_time"]) +
chunk_start
176 except (ValueError, TypeError):
177 pass
178
179 logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
180
181 # Clean up chunk file
182 try:
183 os.remove(chunk_path)
184 except Exception:
185 pass
186
187 return chunk_label, chunk_segments, metrics
188
189 # Run chunks in parallel
190 all_metrics = {}
191 logger.info("Starting parallel processing with up to 5 concurrent
workers...")
192 t_chunking_start = time.time()
193 with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
194 futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
195 for future in concurrent.futures.as_completed(futures):
196 label, segments, metrics = future.result()
197 ai_segments.extend(segments)
198 if metrics:
199 all_metrics[label] = metrics
200
201 # Print Summary Log
202 summary_lines = [

```

```

203 "",
204 "=" * 60,
205 " GEMINI CHUNKING SUMMARY",
206 "=" * 60,
207 f" Total time: {time.time() - t_chunking_start:.1f}s",
208 f" Total chunks: {num_chunks}",
209 f" Total play segments found: {len(ai_segments)}",
210 "-" * 60,
211 f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}}",
212 "-" * 60,
213]
214 for i in range(num_chunks):
215 lbl = f"Chunk {i+1}/{num_chunks}"
216 m = all_metrics.get(lbl, {})
217 sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
218 upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
219 prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
220 gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
221 seg = m.get('segments_found', 0) if m else 0
222 summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
223 summary_lines.append("=" * 60)
224 logger.info("\n".join(summary_lines))
225
226 # Clean up local temp files
227 cleanup_temp_files()
228 try:
229 os.rmdir(chunks_dir)
230 except Exception:
231 pass
232
233 logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
234
235 else:
236 # --- SINGLE-FILE PROCESSING (short videos) ---
237 try:
238 file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
239 except Exception:
240 file_sz = 0.0
241
242 # Get prompt from sport profile
243 prompt = sport_profile.get_prompt(chunk_duration=video_duration)
244 ai_segments, metrics = provider.analyze_video(
245 video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
246)
247 summary_lines = [
248 "",
249 "=" * 50,
250 " GEMINI SINGLE FILE SUMMARY",
251 "=" * 50,
252 f" File Size: {file_sz:.1f} MB",
253 f" Upload time: {metrics.get('upload_time', 0):.1f}s",
254 f" API Processing time: {metrics.get('process_time', 0):.1f}s",
255 f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
256 f" Play segments found: {len(ai_segments)}",
257 "-" * 50,
258]
259 logger.info("\n".join(summary_lines))
260 cleanup_temp_files()
261
262 if not ai_segments:
263 return []

```

```

264
265 # Populate SQLite DB with parsed play segments
266 segments_data = []
267 logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
268
269 def parse_time(val) -> float:
270 """Converts a timestamp value to total float seconds.
271 Handles float/int pass-through, plain numeric strings, and
272 colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
273 when unexpected formats are encountered."""
274 if isinstance(val, (int, float)):
275 return float(val)
276 if isinstance(val, str):
277 val = val.strip()
278 if ":" in val:
279 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
280
281 # Convert it, but warn so the issue is visible in logs.
282 parts = val.split(":")
283 parts.reverse()
284 total = 0.0
285 for i, p in enumerate(parts):
286 try:
287 total += float(p) * (60 ** i)
288 except ValueError:
289 pass
290 logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
291 return total
292 try:
293 return float(val)
294 except ValueError:
295 logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
296 return 0.0
297 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
298 return 0.0
299
300 # 7. Parse timestamps and build candidate segment list
301 idx_cfg = self.config.get("indexing", {})
302 min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
303 max_segment_duration = 90.0 # Singles play segments almost never exceed this
304
305 candidates = []
306 for idx, segment in enumerate(ai_segments):
307 start = parse_time(segment.get("start_time", 0.0))
308 end = parse_time(segment.get("end_time", 0.0))
309 if start >= end:
310 logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
311 continue
312 candidates.append({
313 "idx": idx,
314 "start": start,
315 "end": end,
316 "raw": segment
317 })
318
319 # 8. Post-hoc validation pass
320 logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
321 validated = []
322 rejected = []

```

```

322
323 for c in candidates:
324 label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
325
326 # A: Reject segments that start beyond the video
327 if video_duration > 0 and c["start"] >= video_duration:
328 rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
329 continue
330
331 # B: Clamp end_time to video duration
332 if video_duration > 0 and c["end"] > video_duration:
333 logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
334 c["end"] = video_duration
335
336 dur = c["end"] - c["start"]
337
338 # C: Reject segments shorter than min_segment_duration
339 if dur < min_segment_duration:
340 rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
341 continue
342
343 # D: Flag suspiciously long segments
344 if dur > max_segment_duration:
345 logger.warning(
346 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
347 f"This may be a detection error. Keeping but flagging."
348)
349
350 validated.append(c)
351
352 # E: Resolve overlapping segments (keep the longer one)
353 if len(validated) > 1:
354 # Sort by start time
355 validated.sort(key=lambda r: r["start"])
356 non_overlapping = [validated[0]]
357 for curr in validated[1:]:
358 prev = non_overlapping[-1]
359 # Check if current segment overlaps with the previous one
360 if curr["start"] < prev["end"]:
361 prev_dur = prev["end"] - prev["start"]
362 curr_dur = curr["end"] - curr["start"]
363 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
364 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
365 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
366 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
367 if kept == curr:
368 non_overlapping[-1] = curr # Replace previous with current
369 else:
370 non_overlapping.append(curr)
371 validated = non_overlapping
372
373 # Log validation summary
374 if rejected:
375 reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
376 logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
377 logger.info(f"{len(validated)} play segments passed validation (out of

```



```

{len(ai_segments)} from Gemini).")
378
379 stitch_cfg = self.config.get("stitching", {})
380 begin_pad = stitch_cfg.get("begin_padding", 0.5)
381 end_pad = stitch_cfg.get("end_padding", 0.5)
382
383 total_highlight_dur = 0.0
384 for c in validated:
385 total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
386
387 mins = int(total_highlight_dur // 60)
388 secs = int(total_highlight_dur % 60)
389 logger.info(
390 f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
391 f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
392)
393
394 # 9. Populate SQLite DB with validated play segments
395 for c in validated:
396 start = c["start"]
397 end = c["end"]
398 segment = c["raw"]
399 dur = end - start
400
401 # Extract shot_count from Gemini or compute estimate if not present
402 shot_count = segment.get("shuttle_exchange_count")
403 if shot_count is None:
404 shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
405 else:
406 try:
407 shot_count = int(shot_count)
408 except Exception:
409 shot_count = max(3, int(dur / 0.8))
410
411 # Only store metrics we actually know ? no fabricated data
412 metadata = {
413 "shot_count": shot_count,
414 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
415 "detector": model_name
416 }
417
418 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
419
420 if segment_id:
421 segments_data.append({
422 "id": segment_id,
423 "start_time": start,
424 "end_time": end,
425 "duration": dur,
426 "metadata": metadata
427 })
428
429 return segments_data
430
431 # Register the segmenter
432 segmenter_registry.register(GeminiPlaySegmenter)
433

```

## 60. user (2026-06-06T15:28:55.097Z)

```

1 """Slice individual rally clips out of a (long) annotated video.

```

```

2
3 Use cases:
4 - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5 boundaries against the labels.
6 - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7 supervision the segmenter is tuned/adapted on for that specific video.
8
9 Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10 end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11 <out_dir>/<video_stem>__rally_<n>.mp4
12
13 Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14 shows the same lead-in/out the model is trained and evaluated against. The window
15 math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16 `slice_rallies`.
17 """
18 from __future__ import annotations
19
20 import argparse
21 import csv
22 import os
23 import subprocess
24 from dataclasses import dataclass
25 from typing import Dict, List, Optional, Tuple
26
27 from backend.eval.annotations import parse_timestamp
28
29 # Match VideoCompiler defaults so sliced clips frame a rally the same way.
30 BEGIN_PADDING = 0.5
31 END_PADDING = 1.0
32
33
34 @dataclass
35 class ClipWindow:
36 rally: str # kept as a string to allow inserted ids like "7.5"
37 start: float # padded, clamped
38 end: float # padded, clamped
39
40
41 def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
42 """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43 for key in (decimal_key, mmss_key):
44 val = (row.get(key) or "").strip()
45 if val:
46 try:
47 return parse_timestamp(val)
48 except ValueError:
49 continue
50 return None
51
52
53 def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54 """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56 Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57 end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58 """
59 labels: Dict[str, Tuple[float, float]] = {}
60 with open(csv_path, newline="", encoding="utf-8") as f:
61 reader = csv.DictReader(f)
62 reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63 [])]
64
65 for row in reader:
66 row = {(k or "").strip().lower(): v for k, v in row.items()}
67 rally = (row.get("rally_number") or "").strip()

```

```

66 if not rally:
67 continue
68 start = _read_time(row, "start_time", "start_mmss")
69 end = _read_time(row, "end_time", "end_mmss")
70 if start is None or end is None or end <= start:
71 continue # missing/degenerate (e.g. a not-yet-filled MISSED row)
72 labels[rally] = (start, end)
73 return labels
74
75
76 def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
77 rally_numbers: List[str],
78 begin_pad: float = BEGIN_PADDING,
79 end_pad: float = END_PADDING,
80 video_duration: Optional[float] = None) -> List[ClipWindow]:
81 """Apply padding and clamp to the video, for the requested rallies.
82
83 rally_numbers may be ["all"] to take every labeled rally (in time order).
84 Unknown rally ids are skipped (caller can diff to warn).
85 """
86 if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
87 wanted = sorted(labels, key=lambda r: labels[r][0])
88 else:
89 wanted = [r for r in rally_numbers if r in labels]
90
91 windows: List[ClipWindow] = []
92 for rally in wanted:
93 start, end = labels[rally]
94 ps = max(0.0, start - begin_pad)
95 pe = end + end_pad
96 if video_duration is not None and video_duration > 0:
97 if ps >= video_duration:
98 continue # entirely past the end
99 pe = min(pe, video_duration)
100 if pe <= ps:
101 continue
102 windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103 return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107 try:
108 out = subprocess.run(
109 ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110 "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111 capture_output=True, text=True, check=True,
112).stdout.strip()
113 return float(out)
114 except (subprocess.SubprocessError, OSError, ValueError):
115 return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119 out_dir: str, begin_pad: float = BEGIN_PADDING,
120 end_pad: float = END_PADDING) -> List[str]:
121 """Write one padded clip per requested rally; returns the clip paths written."""
122 if not os.path.exists(video_path):
123 raise FileNotFoundError(f"video not found: {video_path}")
124 labels = load_rally_labels(labels_csv)
125 if not labels:
126 raise ValueError(f"no usable rally rows in {labels_csv}")
127 os.makedirs(out_dir, exist_ok=True)
128
129 duration = _probe_duration(video_path)
130 windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)

```

```

131 if not windows:
132 raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134 stem = os.path.splitext(os.path.basename(video_path))[0]
135 written: List[str] = []
136 for w in windows:
137 safe_rally = w.rally.replace(".", "_")
138 out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139 cmd = [
140 "ffmpeg", "-y",
141 "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142 "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143 "-c:a", "aac", "-b:a", "128k",
144 out_path,
145]
146 try:
147 subprocess.run(cmd, capture_output=True, check=True)
148 except subprocess.CalledProcessError as e:
149 err = e.stderr.decode(errors="replace").strip()[-300:]
150 print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151 continue
152 if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153 written.append(out_path)
154 print(f"[slicer] rally {w.rally} -> {out_path}")
155 [{w.start:.3f}-{w.end:.3f}s}]
156 return written
157
158 def main() -> None:
159 p = argparse.ArgumentParser(description="Slice rally clips from an annotated
160 video.")
161 p.add_argument("--video", required=True, help="Source video file")
162 p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
163 p.add_argument("--rallies", required=True,
164 help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
165 p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
166 p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
167 p.add_argument("--end-pad", type=float, default=END_PADDING)
168 args = p.parse_args()
169
170 rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
171 written = slice_rallies(args.video, args.labels, rally_numbers,
172 args.out_dir, args.begin_pad, args.end_pad)
173 print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
174
175 if __name__ == "__main__":
176 main()
177

```

## 61. assistant (2026-06-06T15:29:02.220Z)

Confirmed:

```

- blur.py: `BlurValidator` (`blur_check`), Laplacian variance < threshold = fail,
`min_blur_threshold` default 100.0. ? Map says "blur 100.0 vs 20.0" for the config divergence ?
code default is 100.0 ?.
- gemini.py: `GeminiPlaySegmenter` (name 'gemini'), no P2, chunks only if
`duration>chunk_duration` ?, `max_workers=5` hardcoded (ignores ai_max_workers) ?,
`max_segment_duration=90.0` hardcoded ?, `debug_duration` truncates ?, `parse_time` accepts
colon timestamps (warns) ?. detector = bare `model_name` ?.
- rally_slicer.py: `compute_clip_windows` (pure), `load_rally_labels` (skips degenerate rows),
`BEGIN_PADDING=0.5`/`END_PADDING=1.0`, `_read_time -> parse_timestamp`, `slice_rallies`. ?

```

Now the entrypoints (run\_eval, run\_phase3\_eval, run\_regression, run\_process\_and\_stitch),  
config.json, setup.py, badminton, providers/gemini. Let me grep for key symbols and read the

entrypoints.

## 62. user (2026-06-06T15:29:03.170Z)

```
1 """CLI: regression-test the production tool against golden-set ground truth.
2
3 Obtains ground truth from an annotation provider (a tier), scores the video's
4 stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5 detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7 Examples
8 -----
9 Snapshot the current numbers as the baseline (first run / after a real improvement):
10 python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12 Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13 python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14 """
15 import os
16 import sys
17 import json
18 import argparse
19 import logging
20
21 from backend.database import Database
22 from backend.annotations import annotation_registry
23 from backend.eval import regression
24
25
26 def _default_db_path() -> str:
27 db_name = "sports_indexer.db"
28 if os.path.exists("config.json"):
29 try:
30 with open("config.json") as f:
31 db_name = json.load(f).get("output", {}).get("db_name", db_name)
32 except (json.JSONDecodeError, OSError):
33 pass
34 return os.path.join("output", db_name)
35
36
37 def build_parser() -> argparse.ArgumentParser:
38 p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
39 p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
40 p.add_argument("--tier", default="file",
41 help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
42 p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
43 p.add_argument("--stage", choices=("candidates", "final"), default="final")
44 p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
45 p.add_argument("--iou", type=float, default=0.5)
46 p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
47 p.add_argument("--db", default=None)
48 p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
49 p.add_argument("--update-baseline", action="store_true",
50 help="Snapshot the current numbers as the new baseline instead of
checking.")
51 p.add_argument("--json", dest="json_out", default=None)
52 return p
53
```

```

54
55 def main(argv=None) -> int:
56 logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
57 args = build_parser().parse_args(argv)
58
59 db_path = args.db or _default_db_path()
60 if not os.path.exists(db_path):
61 print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
62 return 2
63 db = Database(db_path)
64
65 result = regression.regress_with_provider(
66 db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
67 stage=args.stage, iou_threshold=args.iou, detector=args.detector,
68 tolerance=args.tolerance, baseline_path=args.baseline,
69)
70
71 print(regression.format_result(result))
72
73 if args.update_baseline:
74 regression.save_baseline(result.video_id, result.stage, result.precision,
75 result.recall, result.f1, path=args.baseline)
76 print(f"Baseline updated in {args.baseline}.")
77
78 if args.json_out:
79 with open(args.json_out, "w") as f:
80 json.dump(result.as_dict(), f, indent=2)
81 print(f"JSON written to {args.json_out}")
82
83 # Non-zero exit on regression so CI can gate on it.
84 return 1 if result.regressed else 0
85
86
87 if __name__ == "__main__":
88 raise SystemExit(main())
89

```

### 63. user (2026-06-06T15:29:03.649Z)

```

1 """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3 Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4 API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
6 Examples
7 -----
8 Create an empty annotation file to fill in:
9 python run_eval.py --scaffold --annotations rallies.csv
10
11 Score a video's detected rallies (identify it by file or by stored video_id):
12 python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
13 python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
14
15 Show which rallies were missed / spurious, and dump a JSON report:
16 python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
17 """
18
19 import os
20 import sys
21 import json
22 import hashlib
23 import argparse
24 import logging

```

```

25
26 from backend.database import Database
27 from backend.eval.annotations import load_annotations, write_scaffold
28 from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES
29
30
31 def get_file_md5(filepath: str) -> str:
32 """MD5 of a file, chunked ? matches how the pipeline derives video_id."""
33 hasher = hashlib.md5()
34 with open(filepath, "rb") as f:
35 for chunk in iter(lambda: f.read(8192), b''):
36 hasher.update(chunk)
37 return hasher.hexdigest()
38
39
40 def _default_db_path() -> str:
41 """Resolve the DB path from config.json (output.db_name) under output/."""
42 db_name = "sports_indexer.db"
43 if os.path.exists("config.json"):
44 try:
45 with open("config.json") as f:
46 db_name = json.load(f).get("output", {}).get("db_name", db_name)
47 except (json.JSONDecodeError, OSError):
48 pass
49 return os.path.join("output", db_name)
50
51
52 _EPILOG = """\
53 Prerequisite:
54 The harness scores predictions read from the SQLite DB ? it does NOT run the
55 pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
56 so its detections land in output/sports_indexer.db. Then point this tool at the
57 same video file (--video) or its stored MD5 (--video-id). If you get
58 "database not found" or zero predictions, the video hasn't been processed yet.
59
60 See docs/EVALUATION.md for the annotation format and how to read the output.
61 """
62
63
64 def build_parser() -> argparse.ArgumentParser:
65 p = argparse.ArgumentParser(
66 description="Evaluate detected rallies vs ground-truth annotations.",
67 epilog=_EPILOG,
68 formatter_class=argparse.RawDescriptionHelpFormatter,
69)
70 p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end per row).")
71 p.add_argument("--video", help="Path to the video file (its MD5 becomes the video_id).")
72 p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of --video.")
73 p.add_argument("--db", default=None, help="SQLite DB path (default: from config.json).")
74 p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
75 help="Which prediction stage(s) to score (default: both).")
76 p.add_argument("--detector", default=None,
77 help="Only score candidates from this detector (e.g. yolo_hybrid, tracknet_hybrid).")
78 p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default: 0.5).")
79 p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep (0.1..0.9) in the report.")
80 p.add_argument("--show-failures", action="store_true",
81 help="List missed (FN) and spurious (FP) rallies with timestamps.")
82 p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to

```

```

this path.")
83 p.add_argument("--scaffold", action="store_true",
84 help="Write an empty annotation CSV to --annotations and exit.")
85 return p
86
87
88 def main(argv=None) -> int:
89 logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
90 args = build_parser().parse_args(argv)
91
92 # Scaffold mode: just write the template and exit.
93 if args.scaffold:
94 if not args.annotations:
95 print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
96 return 2
97 write_scaffold(args.annotations)
98 print(f"Wrote empty annotation file: {args.annotations}")
99 return 0
100
101 if not args.annotations:
102 print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
103 return 2
104
105 # Resolve the video_id.
106 if args.video_id:
107 video_id = args.video_id
108 elif args.video:
109 if not os.path.exists(args.video):
110 print(f"ERROR: video not found: {args.video}", file=sys.stderr)
111 return 2
112 print("Computing video MD5...")
113 video_id = get_file_md5(args.video)
114 print(f"video_id = {video_id}")
115 else:
116 print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
117 return 2
118
119 db_path = args.db or _default_db_path()
120 if not os.path.exists(db_path):
121 print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
122 file=sys.stderr)
123 return 2
124
125 gt = load_annotations(args.annotations)
126 if not gt:
127 print(f"WARNING: no ground-truth rallies in {args.annotations}.",
128 file=sys.stderr)
129
130 stages = list(STAGES) if args.stage == "both" else [args.stage]
131 db = Database(db_path)
132 report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
133 detector=args.detector, with_pr_curve=args.pr_curve)
134
135 print(format_report_text(report, show_failures=args.show_failures))
136
137 if args.json_out:
138 with open(args.json_out, "w") as f:
139 json.dump(report_to_dict(report), f, indent=2)
140 print(f"\nJSON report written to {args.json_out}")
141
142 return 0
143
144 if __name__ == "__main__":
145 raise SystemExit(main())

```



**64. user (2026-06-06T15:29:04.042Z)**

```

1 """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3 Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4 processes each video through a segmenter (so detections land in the indexer DB),
5 scores each against its ground-truth rally CSV, and prints per-video plus
6 corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8 Examples
9 -----
10 Free CPU baseline on a single video (validate the loop):
11 python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
12 --segmenter motion --only shuttlestet_1
13
14 Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15 python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
16 --json output/phase3_report.json
17
18 Re-score only (videos already processed), no re-segmentation:
19 python run_phase3_eval.py --manifest ../manifest.json --score-only
20 """
21
22 import os
23 import sys
24 import json
25 import time
26 import hashlib
27 import argparse
28 import logging
29
30 from dotenv import load_dotenv
31 load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
32
33 from backend.database import Database
34 # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
35 from backend.indexer import segmenter_registry
36 from backend.eval.annotations import load_annotations
37 from backend.eval.harness import evaluate, format_report_text, report_to_dict
38 from backend.eval import batch
39
40 logger = logging.getLogger(__name__)
41
42
43 def get_file_md5(filepath: str) -> str:
44 hasher = hashlib.md5()
45 with open(filepath, "rb") as f:
46 for chunk in iter(lambda: f.read(65536), b''):
47 hasher.update(chunk)
48 return hasher.hexdigest()
49
50
51 def load_config(path: str = "config.json") -> dict:
52 if os.path.exists(path):
53 with open(path) as f:
54 return json.load(f)
55 return {}
56
57
58 def build_parser() -> argparse.ArgumentParser:
59 p = argparse.ArgumentParser(
60 description="Batch process + evaluate a collector export manifest (Phase 3).",

```

```

61 formatter_class=argparse.RawDescriptionHelpFormatter,
62)
63 p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
64 p.add_argument("--videos-root", default=None,
65 help="Root for resolving manifest relative local_paths "
66 "(default: the collector repo root, two levels above the
manifest).")
67 p.add_argument("--segmenter", default="motion",
68 help="Segmenter to run (default: motion ? free/CPU). "
69 "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
70 p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
71 default="auto",
72 help="Which prediction stage(s) to score (default: auto by
segmenter).")
73 p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
74 p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
75 p.add_argument("--only", action="append", default=None,
76 help="Only this video_id (repeatable).")
77 p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
78 p.add_argument("--max-frames", type=int, default=None,
79 help="Cap segmenter frames (fast validation).")
80 p.add_argument("--score-only", action="store_true",
81 help="Skip segmentation; only score detections already in the DB.")
82 p.add_argument("--reprocess", action="store_true",
83 help="Re-run segmentation even if the video already has segments.")
84 p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
85 p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
86 p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
87 return p
88
89 def main(argv=None) -> int:
90 logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
91 args = build_parser().parse_args(argv)
92
93 manifest_path = os.path.abspath(args.manifest)
94 if not os.path.exists(manifest_path):
95 print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
96 return 2
97 manifest_dir = os.path.dirname(manifest_path)
98 videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
99
100 with open(manifest_path) as f:
101 manifest = json.load(f)
102 entries = manifest.get("eval_sets", [])
103
104 cfg = load_config()
105 db_name = cfg.get("output", {}).get("db_name", "sports_indexer.db")
106 db_path = args.db or os.path.join("output", db_name)
107 os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
108 db = Database(db_path)
109
110 stages = batch.default_stages_for(args.segmenter, args.stage)
111
112 # Resolve the segmenter class up front (unless purely scoring).
113 segmenter = None
114 if not args.score_only:
115 seg_cls = segmenter_registry.get(args.segmenter)
116 if not seg_cls:

```

```

117 print(f"ERROR: segmenter '{args.segmenter}' not found. "
118 f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
119 return 2
120 segmenter = seg_cls(db, cfg)
121
122 # Filter entries.
123 targets = []
124 for e in entries:
125 if args.only and e["video_id"] not in args.only:
126 continue
127 targets.append(e)
128 if args.limit:
129 targets = targets[: args.limit]
130
131 per_video = []
132 skipped = []
133 print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
134
135 for e in targets:
136 vid = e["video_id"]
137 video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
138 csv_path = os.path.join(manifest_dir, e["csv"])
139
140 if not video_path or not os.path.exists(video_path):
141 skipped.append((vid, "no video file"))
142 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
143 continue
144 if not os.path.exists(csv_path):
145 skipped.append((vid, "no GT csv"))
146 print(f"[skip] {vid}: GT csv missing ({csv_path})")
147 continue
148
149 video_id = get_file_md5(video_path)
150
151 # Segment unless score-only or already present (and not --reprocess).
152 if not args.score_only:
153 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
154 if already and not args.reprocess:
155 print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
156 else:
157 print(f"[proc] {vid}: segmenting with {args.segmenter}...")
158 db.add_video(video_id, video_path, "processed", [])
159 t0 = time.time()
160 segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
161 print(f" -> {len(segs)} segments in {time.time() - t0:.0f}s")
162
163 gt = load_annotations(csv_path)
164 report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
165 print(format_report_text(report, show_failures=args.show_failures))
166 print()
167 d = report_to_dict(report)
168 d["collector_video_id"] = vid
169 per_video.append(d)
170
171 agg = batch.aggregate(per_video, stages)
172 print(batch.format_aggregate(agg))
173 if skipped:
174 print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))

```

```

175
176 if args.json_out:
177 with open(args.json_out, "w") as f:
178 json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
179 "aggregate": agg, "per_video": per_video,
180 "skipped": skipped}, f, indent=2)
181 print(f"\nJSON report written to {args.json_out}")
182
183 return 0
184
185
186 if __name__ == "__main__":
187 raise SystemExit(main())
188

```

## 65. assistant (2026-06-06T15:29:13.860Z)

All three entrypoints confirmed:

```

- run_regression.py: `main`, `regress_with_provider` vs baseline, exit codes
0=ok/1=regression/2=missing DB ?, `--update-baseline` runs AFTER compare (snapshots even
regressing run) ?. `get_file_md5` ? wait, run_regression doesn't compute MD5, it takes `--video-
id` directly. The map says run_regression `--video-id` ?.
- run_eval.py: `main`, `--stage{candidates,final,both}`, exit 2=arg/I/O, pure scorer,
`get_file_md5` uses 8192 (8KB) ?.
- run_phase3_eval.py: `main`, walks manifest, segment-if-needed, score, aggregate,
`load_dotenv()` at import ?, DB key = file MD5 (`get_file_md5(video_path)`) NOT manifest's
`video_id` (used as label, `d["collector_video_id"]`) ?, `get_file_md5` uses 65536 (64KB).

```

Wait ? the map's gotcha about MD5 chunk sizes says "64KB main, 1MB run\_process, 8KB run\_eval, 64KB phase3". I've verified: main.py=65536 (64KB) ?, run\_eval=8192 (8KB) ?, phase3=65536 (64KB) ?. Need to verify run\_process\_and\_stitch uses 1MB. Let me read it, plus config.json, setup.py, badminton, providers/gemini.

## 66. user (2026-06-06T15:29:14.805Z)

```

1 import os
2 import sys
3 import json
4 import hashlib
5 import logging
6 from dotenv import load_dotenv
7
8 load_dotenv()
9
10 # Configure root logger ? all backend modules use logging.getLogger(__name__)
11 logging.basicConfig(
12 level=logging.INFO,
13 format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
14 datefmt="%H:%M:%S",
15)
16
17 from backend.database import Database
18 from backend.validators.base import registry
19 from backend.indexer import segmenter_registry
20 from backend.stitcher.compiler import VideoCompiler
21
22 def get_file_md5(filepath: str) -> str:
23 total_size = os.path.getsize(filepath)
24 hasher = hashlib.md5()
25 processed = 0
26 with open(filepath, "rb") as f:
27 # Read in 1MB chunks for faster IO on huge files
28 for chunk in iter(lambda: f.read(1024 * 1024), b''):
29 hasher.update(chunk)
30 processed += len(chunk)

```

```

31 if processed % (1024 * 1024 * 50) == 0 or processed == total_size:
32 print(f"\r[MD5] Processed {processed/(1024**3):.2f} GB /
{total_size/(1024**3):.2f} GB ({(processed/total_size)*100:.1f}%)", end="", flush=True)
33 print()
34 return hasher.hexdigest()
35
36 def main():
37 video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
38 output_dir = r"C:\Users\avidu\Videos\Testing"
39 output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
40
41 print(f"Loading config...")
42 if os.path.exists("config.json"):
43 with open("config.json", "r") as f:
44 config = json.load(f)
45 else:
46 config = {}
47
48 print(f"Verifying files exist...")
49 if not os.path.exists(video_path):
50 print(f"Error: Video file not found at {video_path}")
51 return
52
53 os.makedirs("output", exist_ok=True)
54 db_path = os.path.join("output", "sports_indexer.db")
55 db = Database(db_path)
56
57 # Calculate MD5 of the video to uniquely identify it
58 import time
59 print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
60 t0 = time.time()
61 video_id = get_file_md5(video_path)
62 print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
63
64 stitching_config = config.get("stitching", {})
65 use_cache = stitching_config.get("use_cache", True)
66
67 segments = []
68 cached_video = db.get_video(video_id)
69 cache_hit = False
70
71 if cached_video and cached_video.get("status") == "processed":
72 segments = db.query_play_segments(video_id, {})
73 if len(segments) > 0:
74 print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
75 choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
76 if choice == '1':
77 print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
78 cache_hit = True
79 else:
80 print("[CACHE] Ignoring cache...", flush=True)
81 else:
82 print(f"[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
83
84 if not cache_hit:
85 print("[CACHE] Initializing clean indexing...", flush=True)
86 db.clear_video_data(video_id)
87
88 print("Bypassing OpenCV validations for performance on massive files...",
flush=True)

```

```

89 db.add_video(video_id, video_path, "processed", [])
90
91 print("Running segmenter & indexing...", flush=True)
92 seg_name = config.get("indexing", {}).get("default_segmenter", "yolo_hybrid")
93 segmenter_cls = segmenter_registry.get(seg_name)
94 if not segmenter_cls:
95 print(f"[FAIL] Segmenter '{seg_name}' not found.")
96 return
97 print(f"Using segmenter: {seg_name}", flush=True)
98 segmenter = segmenter_cls(db, config)
99 segments = segmenter.process_video(video_id, video_path)
100 print(f"Successfully indexed {len(segments)} play segments.", flush=True)
101
102 if not segments:
103 print("No play segments detected. Cannot compile highlights.")
104 return
105
106 end_padding = stitching_config.get("end_padding", 1.0)
107 begin_padding = stitching_config.get("begin_padding", 0.5)
108 min_shot_count = stitching_config.get("min_shot_count", 0)
109
110 if cache_hit:
111 print("\n--- Stitching Customizations ---")
112 try:
113 bpad_input = input(f"Enter begin_padding in seconds (default
114{begin_padding}): ").strip()
115 if bpad_input:
116 begin_padding = float(bpad_input)
117
118 pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
119").strip()
120 if pad_input:
121 end_padding = float(pad_input)
122
123 shot_input = input(f"Enter min_shot_count to filter by (default
124{min_shot_count}): ").strip()
125 if shot_input:
126 min_shot_count = int(shot_input)
127 except ValueError:
128 print("[WARNING] Invalid input detected, falling back to defaults from
129config.json.")
130
131 print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
132end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
133
134 # Filter segments where shot_count >= min_shot_count
135 time_pairs = []
136 skipped_count = 0
137 for s in segments:
138 meta = s.get("metadata", {})
139 if isinstance(meta, str):
140 try:
141 meta = json.loads(meta)
142 except Exception:
143 meta = {}
144 shot_count = meta.get("shot_count", 0)
145 if shot_count >= min_shot_count:
146 time_pairs.append((s["start_time"], s["end_time"]))
147 else:
148 skipped_count += 1
149
150 if skipped_count > 0:
151 print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
152{min_shot_count} shots.", flush=True)

```

```

148 print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)
149
150 if not time_pairs:
151 print("No play segments passed the filtering criteria. Cannot compile
highlights.")
152 return
153
154 compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding)
155 try:
156 out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
157 print(f"Stitched highlight video saved to: {out_path}", flush=True)
158 except Exception as e:
159 print(f"Error during stitching: {e}", flush=True)
160
161 if __name__ == "__main__":
162 main()
163

```

#### 67. user (2026-06-06T15:29:15.499Z)

```

1 {
2 "sport": "badminton",
3 "ai_provider": "gemini",
4 "ai_model": "gemini-2.5-flash",
5 "validation": {
6 "min_resolution_width": 1280,
7 "min_resolution_height": 720,
8 "min_fps": 29.9,
9 "min_blur_threshold": 20.0,
10 "max_scene_cuts_per_minute": 0.5,
11 "relevance": {
12 "court_green_lower": [
13 35,
14 40,
15 40
16],
17 "court_green_upper": [
18 85,
19 255,
20 255
21],
22 "court_pixels_min_ratio": 0.05
23 }
24 },
25 "indexing": {
26 "default_segmenter": "gemini",
27 "use_gpu": true,
28 "motion_threshold": 0.08,
29 "min_segment_duration": 3.0,
30 "dead_time_merge_gap": 2.0,
31 "chunk_duration": 90,
32 "chunk_overlap": 10,
33 "detector_model": "fasterrcnn_resnet50_fpn",
34 "detector_score_threshold": 0.5,
35 "yolo_velocity_threshold": 0.15,
36 "yolo_frame_skip": 3,
37 "yolo_tracker": "bytetrack.yaml",
38 "yolo_parallel_chunks": false,
39 "yolo_chunk_workers": 2,
40 "min_action_window_duration": 2.0,
41 "log_yolo_candidates": true,
42 "store_yolo_candidates": true,
43 "ai_handoff_padding": 2.0,

```

```

44 "ai_max_workers": 5,
45 "tracknet": {
46 "wsl_distro": "Ubuntu",
47 "repo_dir": "~/models/TrackNetV4",
48 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
49 "conda_env": "TrackNetV4",
50 "weights_path": "",
51 "queue_length": 5,
52 "stage_in_wsl": true,
53 "wsl_stage_dir": "~/clips",
54 "timeout_sec": 0,
55 "velocity_threshold": 0.02
56 }
57 },
58 "output": {
59 "default_highlights_name": "highlights.mp4",
60 "db_name": "sports_indexer.db"
61 },
62 "stitching": {
63 "begin_padding": 0.5,
64 "end_padding": 0.5,
65 "use_cache": false,
66 "min_shot_count": 1
67 }
68 }

```

#### 68. user (2026-06-06T15:29:15.753Z)

```

1 from backend.sports.base import SportProfile
2
3 class BadmintonSportProfile(SportProfile):
4 @property
5 def name(self) -> str:
6 return "badminton"
7
8 def get_prompt(self, chunk_duration: float = 0.0) -> str:
9 base_prompt = """System Role & Objective
10 You are a highly precise computer vision data extraction engine specializing in
11 badminton match analysis. Your exclusive objective is to parse a full-length match video and
12 generate a flawless JSON array containing exact rally boundaries and shot metrics for automated
13 highlight generation.
14
15 Core Directives
16 Detect every single active rally in the video. Do not miss any live gameplay.
17 Extract high-precision timestamps as TOTAL DECIMAL SECONDS from the start of the video
18 for exact frame-accurate cuts.
19 Estimate the number of shots exchanged (net crossings) and provide a confidence score
20 for that count, acknowledging frame-sampling limitations.
21
22 CRITICAL TIMESTAMP FORMAT RULES:
23 - All start_time and end_time values MUST be expressed as TOTAL SECONDS from the
24 beginning of the video, as a decimal number.
25 - Example: A rally starting at 1 minute 14 seconds into the video MUST be written as
26 74.0, NOT as "1:14" or 114.
27 - Example: A rally starting at 2 minutes 30.5 seconds MUST be written as 150.5, NOT as
28 "2:30.5" or 230.5.
29 - NEVER use MM:SS or HH:MM:SS colon-separated format. NEVER concatenate minutes and
30 seconds into a single number without converting.
31 - The downstream pipeline will BREAK if timestamps are not in total decimal seconds.
32
33 Positive Constraints: Rally Boundaries
34 Start Condition: The exact fractional second the server's racket makes contact with the
35 shuttlecock.
36 End Condition: The rally concludes ONLY when both of the following physical criteria are
37 met sequentially: First, the shuttlecock hits the court surface, strikes the net and drops, or

```



lands visibly out of bounds. Second, a 0.3 to 0.5-second buffer has elapsed where both players have visibly ceased all kinetic momentum (no lunging, sliding, or racket follow-through).

```
27
28 Negative Constraints: Anomaly Rejection (CRITICAL)
29 Replay Rejection: You must strictly ignore and filter out all slow-motion instant
replays. Replays are not live rallies.
30 Dead-Time Rejection: Ignore all Hawkeye/line-call challenges, interval coaching, towel
breaks, court mopping, changing shuttles, and player celebrations.
31 Pre-serve Rituals: Do not start the timestamp during the players' setup or stance. Wait
for physical racket-to-shuttle contact.
32
33 Shot Tracking & Data Confidence
34 Shot Definition: Count every successful traverse of the shuttlecock across the net
(serves, clears, smashes, drops, drives). Do not count the final grounding or pre-net faults.
35 Confidence Scoring: Because the shuttlecock moves at extreme velocities that may fall
between sampled frames, provide a shot_count_confidence metric (Low, Medium, High). Base your
estimate on player biomechanics, positioning, and racket swings if the shuttle itself is
blurred.
36
37 Strict Output Schema
38 Output strictly in the following JSON array format. Do not include markdown formatting,
introductory text, conversational filler, or explanations. Any deviation from this schema will
break the downstream pipeline.
39 Remember: start_time and end_time are TOTAL SECONDS (decimal), not MM:SS.
40
41 [
42 {
43 "start_time": 74.0,
44 "end_time": 81.3,
45 "shuttle_exchange_count": 11,
46 "shot_count_confidence": "High"
47 },
48 {
49 "start_time": 150.5,
50 "end_time": 174.2,
51 "shuttle_exchange_count": 24,
52 "shot_count_confidence": "Medium"
53 }
54]
55 """
56 if chunk_duration > 0.0:
57 duration_context = (
58 f"IMPORTANT VIDEO CONTEXT: This video segment is exactly
{chunk_duration:.1f} seconds long. "
59 f"All timestamps MUST be between 0 and {chunk_duration:.1f}. "
60 f"Any timestamp beyond {chunk_duration:.1f}s is INVALID.\n\n"
61)
62 return duration_context + base_prompt
63 return base_prompt
64
65 def get_relevance_config(self) -> dict:
66 return {
67 "court_green_lower": [35, 40, 40],
68 "court_green_upper": [85, 255, 255],
69 "court_pixels_min_ratio": 0.05
70 }
71
```

## 69. user (2026-06-06T15:29:16.486Z)

```
1 import time
2 import json
3 import logging
4 from typing import Tuple
5 from google import genai
```

```

6 from google.genai import types
7 from backend.providers.base import AIVideoProvider
8
9 logger = logging.getLogger(__name__)
10
11 class GeminiProvider(AIVideoProvider):
12 def __init__(self, api_key: str, model_name: str):
13 super().__init__(api_key, model_name)
14 # Initialize client lazily to avoid connection/auth issues during class
instantiation
15 self._client = None
16
17 @property
18 def client(self):
19 if self._client is None:
20 self._client = genai.Client(api_key=self.api_key)
21 return self._client
22
23 def analyze_video(self, video_path: str, prompt: str,
24 chunk_duration: float = 0.0,
25 label: str = "") -> Tuple[list, dict]:
26 log_prefix = f"Gemini {label}" if label else "Gemini"
27
28 metrics = {
29 "upload_time": 0.0,
30 "process_time": 0.0,
31 "gen_time": 0.0
32 }
33
34 # 1. Upload
35 logger.info(f"[{log_prefix}] Uploading {video_path}...")
36 t_upload_start = time.time()
37 try:
38 video_file = self.client.files.upload(file=video_path)
39 metrics["upload_time"] = time.time() - t_upload_start
40 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
41 except Exception as e:
42 logger.error(f"[{log_prefix}] Upload failed: {e}")
43 return [], metrics
44
45 # 2. Wait for processing on Google servers
46 t_process_start = time.time()
47 try:
48 while video_file.state.name == "PROCESSING":
49 logger.info(f"[{log_prefix}] Processing on Google servers. Elapsed:
{time.time()-t_process_start:.1f}s. Retrying in 10s...")
50 time.sleep(10)
51 video_file = self.client.files.get(name=video_file.name)
52 if video_file.state.name == "FAILED":
53 logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
54 try:
55 self.client.files.delete(name=video_file.name)
56 except Exception:
57 pass
58 return [], metrics
59 except Exception as e:
60 logger.error(f"[{log_prefix}] Status check failed: {e}")
61 try:
62 self.client.files.delete(name=video_file.name)
63 except Exception:
64 pass
65 return [], metrics
66

```

```

67 metrics["process_time"] = time.time() - t_process_start
68 logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
69
70 # 3. Call Gemini
71 logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
72 t_gen_start = time.time()
73 raw_text = None
74 try:
75 response = self.client.models.generate_content(
76 model=self.model_name,
77 contents=[video_file, prompt],
78 config=types.GenerateContentConfig(
79 response_mime_type="application/json",
80 temperature=0.0
81)
82)
83 raw_text = response.text
84 metrics["gen_time"] = time.time() - t_gen_start
85 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
86 except Exception as e:
87 logger.error(f"[{log_prefix}] API call failed: {e}")
88 finally:
89 try:
90 self.client.files.delete(name=video_file.name)
91 except Exception:
92 pass
93
94 if not raw_text:
95 return [], metrics
96
97 # 4. Parse JSON
98 try:
99 segments = json.loads(raw_text)
100 if not isinstance(segments, list):
101 logger.error(f"[{log_prefix}] Response is not a JSON array.")
102 return [], metrics
103 return segments, metrics
104 except Exception as e:
105 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
106 return [], metrics
107

```

## 70. assistant (2026-06-06T15:29:27.739Z)

All confirmed:

- run\_process\_and\_stitch.py: `main`, `get\_file\_md5` uses 1MB chunks (`f.read(1024\*1024)`) ?, hardcoded paths, interactive `input()` ?, `default\_segmenter` fallback `yolo\_hybrid` ?, skips validation ("Bypassing OpenCV validations") ?.
- config.json: ships `default\_segmenter: gemini` ?, sections validation/indexing/output/stitching + root sport/ai\_provider/ai\_model ?, has `indexing.tracknet` but NO `indexing.wasb` block. The map says config has `indexing (+tracknet, .wasb)`. Actually there's NO `wasb` block in config.json ? wasb config falls back to WasbConfig defaults. Minor imprecision in the map. Also config has `relevance` block (the map's setup.py gotcha is about setup.py's generated config lacking relevance/tracknet ? that's a separate claim I'll verify). blur threshold in config = 20.0, code default = 100.0 ?.
- badminton.py: `BadmintonSportProfile`, prompt emits the JSON array schema with `start\_time/end\_time/shuttle\_exchange\_count/shot\_count\_confidence`, actively defends against MM:SS, HSV relevance cfg. ?
- providers/gemini.py: `GeminiProvider` (upload->poll->generate JSON temp 0->delete), blocking `time.sleep(10)` ?, returns `([], metrics)` on any failure ?, lazy client ?.

Now let me verify the two remaining claims: setup.py (generated config divergence, GPU-aware

pip) and detectors/\_\_init\_\_.py docstring-only. Let me also confirm config has no wasb block via grep and check the WASB external dep claims aren't verifiable (external repo). Let me read setup.py and detectors/\_\_init\_\_.

## 71. user (2026-06-06T15:29:28.685Z)

```
1 import sys
2 import os
3 import subprocess
4 import shutil
5 import json
6
7 def check_command(cmd):
8 """Check if a system command is available."""
9 return shutil.which(cmd) is not None
10
11 def run_pip_install():
12 """Install required Python packages."""
13 print("Installing Python packages from requirements.txt...")
14
15 # Check if NVIDIA GPU is present
16 has_gpu = check_command("nvidia-smi")
17
18 pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
19 "requirements.txt"]
20
21 if has_gpu:
22 print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
23 wheels.")
24 pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
25 else:
26 print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
27 pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
28
29 try:
30 subprocess.check_call(pip_cmd)
31 print("[SUCCESS] Python packages installed.")
32 return True
33 except subprocess.CalledProcessError as e:
34 print(f"[ERROR] Failed to install Python packages: {e}")
35 return False
36
37 def init_default_config():
38 """Initialize a default config.json if it doesn't exist."""
39 config_path = "config.json"
40 if os.path.exists(config_path):
41 print(f"[INFO] {config_path} already exists. Skipping initialization.")
42 return
43
44 default_config = {
45 "sport": "badminton",
46 "ai_provider": "gemini",
47 "ai_model": "gemini-2.5-flash",
48 "validation": {
49 "min_resolution_width": 1280,
50 "min_resolution_height": 720,
51 "min_fps": 29.9,
52 "min_blur_threshold": 100.0, # Laplacian variance threshold
53 "max_scene_cuts_per_minute": 0.5 # Continuous raw files should have near 0
54 },
55 "indexing": {
56 "default_segmenter": "yolo_hybrid",
57 "use_gpu": True,
58 "motion_threshold": 0.08,
```

```

57 "min_segment_duration": 3.0,
58 "dead_time_merge_gap": 2.0,
59 "chunk_duration": 90,
60 "chunk_overlap": 10,
61 "detector_model": "fasterrcnn_resnet50_fpn",
62 "detector_score_threshold": 0.5,
63 "yolo_velocity_threshold": 0.15,
64 "yolo_frame_skip": 3,
65 "yolo_tracker": "bytetrack.yaml",
66 "yolo_parallel_chunks": False,
67 "yolo_chunk_workers": 2,
68 "min_action_window_duration": 2.0,
69 "ai_handoff_padding": 2.0,
70 "ai_max_workers": 5
71 },
72 "output": {
73 "default_highlights_name": "highlights.mp4",
74 "db_name": "sports_indexer.db"
75 },
76 "stitching": {
77 "begin_padding": 0.5,
78 "end_padding": 0.5,
79 "use_cache": False,
80 "min_shot_count": 1
81 }
82 }
83
84 try:
85 with open(config_path, "w") as f:
86 json.dump(default_config, f, indent=4)
87 print("[SUCCESS] config.json initialized with default settings.")
88 except Exception as e:
89 print(f"[ERROR] Failed to create config.json: {e}")
90
91 def run_diagnostics():
92 """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
93 print("=" * 50)
94 print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
95 print("=" * 50)
96
97 # 1. Check Python Version
98 print(f"Python Version: {sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
99 if sys.version_info.major < 3 or (sys.version_info.major == 3 and
100 sys.version_info.minor < 8):
101 print("[WARNING] Python 3.8+ is highly recommended!")
102 else:
103 print("[PASS] Python version check passed.")
104
105 print("-" * 50)
106
107 # 2. Check FFmpeg / FFprobe
108 ffmpeg_ok = check_command("ffmpeg")
109 ffprobe_ok = check_command("ffprobe")
110
111 if ffmpeg_ok:
112 print("[PASS] FFmpeg is installed and available in PATH.")
113 else:
114 print("[FAIL] FFmpeg is NOT found in your PATH.")
115
116 if ffprobe_ok:
117 print("[PASS] FFprobe is installed and available in PATH.")
118 else:
119 print("[FAIL] FFprobe is NOT found in your PATH.")

```

```

120 if not (ffmpeg_ok and ffprobe_ok):
121 print("\n" + "!" * 50)
122 print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
123 print("How to install FFmpeg on Windows:")
124 print(" Option 1: Open PowerShell as Administrator and run:")
125 print(" winget install Gyan.FFmpeg")
126 print(" Option 2: Download the build from https://ffmpeg.org/download.html")
127 print(" and add its 'bin' directory to your System PATH environment
variable.")
128 print("!" * 50 + "\n")
129 else:
130 print("[SUCCESS] All essential media command line tools are ready.")
131
132 print("-" * 50)
133
134 # 3. Check ML / GPU (Optional but good)
135 try:
136 import torch
137 cuda_ok = torch.cuda.is_available()
138 if cuda_ok:
139 print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
140 else:
141 print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
142 except ImportError:
143 print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
144
145 print("=" * 50)
146 return ffmpeg_ok and ffprobe_ok
147
148 if __name__ == "__main__":
149 init_default_config()
150 media_ok = run_diagnostics()
151 run_pip_install()
152 print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
153

```

## 72. user (2026-06-06T15:29:29.160Z)

```

1 """Low-level moving-object detectors that run outside the main Python process.
2
3 Currently houses the TrackNet WSL runner ? it shells out to a TensorFlow
4 TrackNetV4 environment living inside WSL2 (see docs/DECISIONS.md, ADR-001) and
5 returns the shuttle trajectory back to the Windows-side indexer. Keeping the
6 heavy TF dependency quarantined in WSL is deliberate: the main project env stays
7 free of TensorFlow/CUDA.
8 """
9

```

## 73. user (2026-06-06T15:29:30.009Z)

No matches found

## 74. assistant (2026-06-06T15:29:37.849Z)

All confirmed:

- setup.py: writes default config (skips if present) ?, GPU-aware pip (`nvidia-smi` -> cu124 else cpu) ?, ffmpeg/torch/CUDA diagnostics ?. Generated config: `default\_segmenter: yolo\_hybrid` ?, NO `relevance` block (config.json has it) ?, NO `tracknet` block ?, blur 100.0 vs config.json's 20.0 ?. All the setup.py divergence claims hold.
- detectors/\_\_\_init\_\_\_.py: docstring only, refs `docs/DECISIONS.md` ADR-001 ?, does NOT re-export runners ?.

- config.json has no `wasb` block confirmed (grep no matches). The map's claim `indexing (+.tracknet, .wasb)` overstates ? config.json has `.tracknet` but no `.wasb`. I'll fix that to note wasb is consumed but not in the checked-in config.

Now let me quickly verify the test map and a couple of remaining symbol-level claims (the WSL contract is an external repo ? not verifiable in this repo, so I'll keep its claims as-is but note they're unverifiable). Let me confirm test files exist matching the test map.

#### 75. user (2026-06-06T15:29:38.190Z)

```
tests\test_base.py
tests\test_database.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_harness.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_gemini.py
tests\test_geometry.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_video.py
tests\test_yolo_hybrid.py
tests\test_annotation_providers.py
tests\test_automated_provider.py
tests\test_eval_regression.py
tests\test_rally_slicer.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_wasb_cache.py
tests\test_export_wasb_segments.py
```

#### 76. assistant (2026-06-06T15:29:49.236Z)

All test files in the test map exist and match. Note: the map's TEST MAP claims `test\_wasb\_cache.py` covers `detectors.wasb\_infer` resumable cache ? confirmed earlier that wasb\_infer is the cache module. There's no `test\_wasb\_runner.py` or `test\_wasb\_hybrid.py`, consistent with the map (it lists test\_wasb\_cache and notes wasb\_hybrid is the twin tested only via tracknet ? actually the map doesn't list a wasb\_hybrid test, and indeed there's none; the "no dedicated test" row covers validators/stitcher/main/run\_\*).

I have now verified a representative sample across every section. Let me compile findings and produce the corrected final map. The substantive errors found:

1. **\*\*Validator execution order WRONG\*\*** (section 1A): map says `Format -> ResolutionFPS -> Blur -> SceneCut -> PTSContinuity -> Relevance`; actual registration/execution order is `Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance`. This is substantive since the map itself stresses order = execution.
2. **\*\*config.json `.wasb` block\*\*** (section 2.0): map says `indexing (+.tracknet, .wasb)`; config.json has `.tracknet` only, no `.wasb` block (WASB uses WasbConfig defaults).
3. **\*\*training.py gotcha imprecise\*\*** (section 2.3): map says raw JSON str -> all-zero; actually `stats.get` on a str would raise AttributeError (crash), not all-zero. Softening.

Minor (not flipping accurate, but fixing inline):

4. scene\_cut "first ~100 frames" ? 100 sampled frames at 2fps (~first 50s).
5. MatchResult field names abbreviated (`unmatched\_pred` vs actual `unmatched\_pred\_indices`) ? clarify.

Let me produce the corrected final map.

## 77. user (2026-06-06T15:32:39.478Z)

Structured output provided successfully

## 78. assistant (2026-06-06T15:32:51.770Z)

I verified the CODE\_MAP against the repo across every section and found it largely accurate, with a few substantive errors corrected inline. The map is returned via the structured output.

Summary of what I found:

**\*\*Substantive errors fixed (why accurate=false):\*\***

1. **\*\*Validator execution order was wrong\*\*** (section 1A). The map had `Format ? ResolutionFPS ? Blur ? SceneCut ? PTSContinuity ? Relevance`, but `@backend/validators/\_\_init\_\_.py` registers (and `run\_all` executes) in the order `Format ? ResolutionFPS ? PTSContinuity ? SceneCut ? Blur ? Relevance`. Load-bearing because the map itself states "registration order = execution order."
2. **\*\*config.json `.wasb` block claim\*\*** (section 2.0). The map claimed `indexing (+.tracknet, .wasb)`; the checked-in config has `indexing.tracknet` but no `indexing.wasb` ? WASB runs fall back to `WasbConfig` defaults.
3. **\*\*training.extract\_yolo\_features gotcha was incorrect\*\*** (section 2.3). The map said a raw-JSON-str `stats` yields an all-zero vector; the code does `stats.get(...)` , so a string would raise `AttributeError` (crash), not zero-fill.

**\*\*Minor fixes:\*\*** scene\_cut's "first ~100 frames" ? 100 \*sampled\* frames at 2fps (~first 50s); `MatchResult` field names corrected to `unmatched\_pred\_indices`/`unmatched\_gt\_indices`.

**\*\*Confirmed accurate:\*\*** all registry singletons and plugin patterns (segmenter/provider/sport/annotation/validator), the 4-table DB schema + migration ladder, MD5 chunk-size divergence (64KB/1MB/8KB/64KB), the three-default-segmenter divergence, regression.py's two-"annotations" import, the wasb\_infer resumable-cache contract, and all the \$data-contract shapes (candidate window, det\_raw.jsonl, manifest.json, Score). Sections 4 and 5 describe external repos not present here ? I flagged them as unverifiable against source but confirmed the repo-side caller (`wasb\_infer.py`) is consistent with the documented WASB interface.